

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN ĐIỆN TỬ
BỘ MÔN KỸ THUẬT ĐIỆN TỬ - THÔNG TIN**



HCMUTE

ĐỒ ÁN TỐT NGHIỆP

**THIẾT KẾ VÀ XÂY DỰNG
PHƯƠNG PHÁP KIỂM CHỨNG UVM CHO
RUNTIME-CONFIGURABLE AXI-TO-APB BRIDGE**

NGÀNH CÔNG NGHỆ KỸ THUẬT ĐIỆN TỬ - VIỄN THÔNG

**Sinh viên: NGUYỄN VĂN ANH
MSSV: 22161219**

**ĐOÀN LÊ NHÂN
MSSV: 22161298**

Hướng dẫn: TS. PHẠM VĂN KHOA

TP. HỒ CHÍ MINH – 06/2026

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN ĐIỆN TỬ
BỘ MÔN KỸ THUẬT ĐIỆN TỬ - THÔNG TIN**



HCMUTE

ĐỒ ÁN TỐT NGHIỆP

**THIẾT KẾ VÀ XÂY DỰNG
PHƯƠNG PHÁP KIỂM CHỨNG UVM CHO
RUNTIME-CONFIGURABLE AXI-TO-APB BRIDGE**

NGÀNH CÔNG NGHỆ KỸ THUẬT ĐIỆN TỬ - VIỄN THÔNG

Sinh viên: NGUYỄN VĂN ANH

MSSV: 22161219

ĐOÀN LÊ NHÂN

MSSV: 22161298

Hướng dẫn: TS. PHẠM VĂN KHOA

TP. HỒ CHÍ MINH – 06/2026

BẢN NHẬN XÉT KHÓA LUẬN TỐT NGHIỆP
(Dành cho giảng viên hướng dẫn)

Đề tài: THIẾT KẾ VÀ XÂY DỰNG PHƯƠNG PHÁP KIỂM CHỨNG
UUM CHO RUNTIME - CONFIGURABLE AXI-TO-APB BRIDGE

Sinh viên: + Nguyễn Văn Anh
+ Đoàn Lê Nhân
Hướng dẫn: T.S. Phạm Văn Khoa

MSSV: 201610219
MSSV: 201610298

Nhận xét bao gồm các nội dung sau đây:

1. Tính hợp lý trong cách đặt vấn đề và giải quyết vấn đề; ý nghĩa khoa học và thực tiễn:

Đặt vấn đề rõ ràng, mục tiêu cụ thể; đề tài có tính mới, cấp thiết; đề tài có khả năng ứng dụng, tính sáng tạo.

2. Phương pháp thực hiện/ phân tích/ thiết kế:

Phương pháp hợp lý và tin cậy dựa trên cơ sở lý thuyết; có phân tích và đánh giá phù hợp; có tính mới và tính sáng tạo.

3. Kết quả thực hiện/ phân tích và đánh giá kết quả/ kiểm định thiết kế:

Phù hợp với mục tiêu đề tài; phân tích và đánh giá/ kiểm thử thiết kế hợp lý; có tính sáng tạo/ kiểm định chặt chẽ và đảm bảo độ tin cậy.

4. Kết luận và đề xuất:

Kết luận phù hợp với cách đặt vấn đề, đề xuất mang tính cải tiến và thực tiễn; kết luận có đóng góp mới mẻ, đề xuất sáng tạo và thuyết phục.

5. Hình thức trình bày và bố cục báo cáo:

Văn phong trôi chảy, bố cục hợp lý, cấu trúc rõ ràng, dùng định dạng mẫu; có tính hấp dẫn, thể hiện năng lực tốt, văn bản trang trọng.

6. Kỹ năng chuyên nghiệp và tính sáng tạo:

Thể hiện các kỹ năng giao tiếp, kỹ năng làm việc nhóm, và các kỹ năng chuyên nghiệp khác trong việc thực hiện đề tài.

7. Tài liệu trích dẫn

Tính trung thực trong việc trích dẫn tài liệu tham khảo; tính phù hợp của các tài liệu trích dẫn; trích dẫn theo đúng chỉ dẫn APA.

8. Đánh giá về sự trùng lặp của đề tài

Cần khẳng định đề tài có trùng lặp hay không? Nếu có, đề nghị ghi rõ mức độ, tên đề tài, nơi công bố, năm công bố của đề tài đã trùng lặp.

9. Những nhược điểm và thiếu sót, những điểm cần được bổ sung và chỉnh sửa*

10. Nhận xét tinh thần, thái độ học tập, nghiên cứu của sinh viên

Nghiêm túc, tinh thần học hỏi cao, làm việc hiệu quả

Đề nghị của giảng viên hướng dẫn

Ghi rõ: "Báo cáo đạt/ không đạt yêu cầu của khóa luận tốt nghiệp kỹ sư, và được phép/ không được phép bàn về khóa luận tốt nghiệp"

Cho biết các (8,5)

Tp. HCM, ngày 22 tháng 06 năm 2026

Người nhận xét
(Ký và ghi rõ họ tên)

Phạm Văn Khoa

* Giáo viên hướng dẫn có thể ghi tiếp vào mặt sau

PHIẾU GIAO NHIỆM VỤ ĐỒ ÁN TỐT NGHIỆP

1. Thông tin sinh viên

Họ và tên: Nguyễn Văn Anh

MSSV: 22161219

Họ và tên: Đoàn Lê Nhân

MSSV: 22161298

2. Thông tin đề tài

Tên đề tài: Thiết kế và xây dựng phương pháp kiểm chứng UVM cho Runtime-Configurable AXI-To-APB Bridge

Mục đích đề tài: Xây dựng và đánh giá quy trình Design Verification dựa trên phương pháp UVM theo hướng tiếp cận công nghiệp đối với các register-based IP, sử dụng Runtime-Configurable AXI-to-APB Bridge làm đối tượng triển khai và đánh giá.

3. Nội dung thực hiện

- Nghiên cứu giao thức AXI, APB, phương pháp UVM và quy trình Design Verification theo hướng tiếp cận công nghiệp.
- Sử dụng Verilog để mở rộng AXI-to-APB Bridge thành Runtime-Configurable AXI-to-APB Bridge
- Xây dựng môi trường kiểm chứng dựa trên phương pháp UVM, bám sát quy trình Design Verification chuẩn; thực hiện mô phỏng, regression trên phần mềm QuestaSim.
- Đánh giá mức độ hoàn thành của quá trình kiểm chứng thông qua Functional Coverage và Code Coverage.

4. Lời cam đoan của sinh viên

Chúng tôi cam đoan đồ án tốt nghiệp này là kết quả nghiên cứu và thực hiện của nhóm dưới sự hướng dẫn của giảng viên hướng dẫn. Các kết quả, số liệu và nội dung trình bày trong đồ án là trung thực, được trích dẫn đầy đủ theo quy định và chưa từng được sử dụng để bảo vệ bất kỳ học vị nào trước đây. Chúng tôi hoàn toàn chịu trách nhiệm về nội dung cam đoan này.

TP.HCM, ngày 19 tháng 6 năm 2026

Sinh viên thực hiện

Nguyễn Văn Anh

Đoàn Lê Nhân

LỜI CẢM ƠN

Trong quá trình thực hiện đồ án tốt nghiệp, nhóm thực hiện đã nhận được nhiều sự quan tâm, hướng dẫn và hỗ trợ quý báu từ quý thầy cô và nhà trường.

Trước hết, nhóm xin gửi lời cảm ơn chân thành đến thầy TS. Phạm Văn Khoa đã tận tình hướng dẫn, định hướng và hỗ trợ trong suốt quá trình thực hiện đề tài. Những góp ý, kinh nghiệm chuyên môn và sự động viên của thầy đã góp phần quan trọng giúp đề tài được hoàn thành.

Nhóm cũng xin chân thành cảm ơn quý thầy cô Khoa Điện – Điện tử, Bộ môn Kỹ thuật Máy tính – Viễn thông, Trường Đại học Công Nghệ Kỹ thuật Thành phố Hồ Chí Minh đã truyền đạt kiến thức và tạo điều kiện thuận lợi trong suốt quá trình học tập và nghiên cứu.

Thông qua đề tài này, nhóm có cơ hội củng cố kiến thức chuyên ngành, tiếp cận quy trình Design Verification theo hướng công nghiệp và tích lũy thêm nhiều kinh nghiệm thực tiễn trong lĩnh vực thiết kế vi mạch số.

Mặc dù đã nỗ lực hoàn thành đồ án với tinh thần nghiêm túc và trách nhiệm, báo cáo khó tránh khỏi những hạn chế nhất định. Nhóm rất mong nhận được những ý kiến đóng góp từ quý thầy cô để đề tài được hoàn thiện hơn.

Xin chân thành cảm ơn!

TÓM TẮT

Trong những năm gần đây, cùng với sự phát triển của ngành công nghiệp bán dẫn, Universal Verification Methodology (UVM) đã trở thành một trong những phương pháp kiểm chứng được sử dụng rộng rãi trong quá trình phát triển IP và System on Chip (SoC). Việc nắm vững quy trình Design Verification dựa trên UVM hiện là một yêu cầu quan trọng đối với lĩnh vực kiểm chứng thiết kế số, đồng thời nhận được sự quan tâm lớn từ cả môi trường học thuật và công nghiệp.

Xuất phát từ thực tế đó, luận văn tập trung nghiên cứu và xây dựng một quy trình Design Verification theo phương pháp UVM. Để triển khai và đánh giá quy trình này, một kiến trúc AXI-to-APB Bridge được lựa chọn làm đối tượng minh họa và được mở rộng bằng hệ thống thanh ghi cấu hình cho phép thay đổi động bản đồ địa chỉ của các APB Slave trong thời gian chạy, hình thành Runtime-Configurable AXI-to-APB Bridge.

Thông qua đề tài, tác giả hướng tới việc xây dựng một quy trình verification có tính hệ thống, bám sát quy trình phát triển IP trong thực tế và có khả năng tham khảo áp dụng cho các register-based IP trong lĩnh vực thiết kế vi mạch số.

MỤC LỤC

PHIẾU GIAO NHIỆM VỤ ĐỒ ÁN TỐT NGHIỆP	ii
LỜI CẢM ƠN.....	iii
TÓM TẮT.....	iv
MỤC LỤC	v
DANH MỤC HÌNH.....	viii
DANH MỤC BẢNG	x
CÁC TỪ VIẾT TẮT.....	xii
CHƯƠNG 1: TỔNG QUAN	1
1.1 Lý do chọn đề tài	1
1.2 Mục tiêu nghiên cứu	2
1.3 Đối tượng nghiên cứu	3
1.4 Phạm vi thực hiện	3
1.5 Giới hạn đề tài	4
1.6 Phương pháp nghiên cứu	4
1.7 Bố cục của đề tài.....	5
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT.....	7
2.1 Tổng quan về System on Chip và AMBA.....	7
2.2 Giao thức AMBA AXI	7
2.2.1 Các kênh truyền dữ liệu trong AXI	8
2.2.2 Chu trình đọc và ghi của giao thức AXI.....	11
2.2.3 Cơ chế bắt tay của AXI	13
2.2.4 Cơ chế burst của AXI	14
2.3 Giao thức AMBA APB.....	16
2.3.1 Tín hiệu của giao thức APB	16
2.3.2 Chu trình ghi và đọc của giao thức APB	17
2.3.2.1 Giao dịch ghi	17
2.3.2.2 Giao dịch đọc.....	18
2.3.3 Trạng thái hoạt động của giao thức APB	20
2.4 Tổng quan về phương pháp kiểm chứng UVM.....	20
2.4.1 Cấu trúc UVM Testbench.....	21
2.4.2 Các lớp cấu trúc UVM Testbench	22
2.4.2.1 UVM Testbench Top	22
2.4.2.2 UVM Test	23

2.4.2.3 UVM Environment.....	24
2.4.2.4 UVM Agent.....	25
2.4.2.5 UVM Sequencer.....	27
2.4.2.6 UVM Sequence.....	27
2.4.2.7 UVM Driver.....	28
2.4.2.8 UVM Monitor.....	29
2.4.2.9 UVM Scoreboard.....	29
2.4.3 Verification IP (VIP).....	30
2.4.4 Cơ chế UVM.....	31
2.4.4.1 Cơ chế Pha.....	31
2.4.4.2 Cơ chế Factory.....	32
2.4.4.3 Cơ chế Config_db.....	32
CHƯƠNG 3: PHÂN TÍCH THIẾT KẾ VÀ XÂY DỰNG MÔI TRƯỜNG	
KIỂM CHỨNG UVM.....	34
3.1 Yêu cầu thiết kế.....	34
3.2 Đặc tả và nguyên lý hoạt động.....	34
3.2.1 Tổng quan kiến trúc.....	34
3.2.2 Sơ đồ khối.....	35
3.2.2.1 AXI Clock Domain.....	36
3.2.2.2 APB Clock Domain.....	39
3.2.3 Mô tả các thanh ghi cấu hình.....	43
3.2.4 Phân tích các nội dung cần kiểm chứng.....	45
3.3 Thiết kế môi trường kiểm tra UVM.....	46
3.3.1 Luồng kiểm chứng tổng thể.....	46
3.3.2 Xây dựng kế hoạch kiểm chứng Verification Plan.....	47
3.3.3 Thiết kế cấu trúc dự án và kiến trúc Testbench.....	48
3.3.4 Xây dựng các thành phần của UVM Testbench.....	49
3.3.4.1 Xây dựng Verification IP (VIP).....	49
3.3.4.2 Tích hợp và kết nối các thành phần UVM Testbench.....	51
3.3.4.3 Cơ chế xử lý giao dịch và kiểm tra kết quả trong UVM.....	62
3.3.4.4 Các luồng giao dịch điển hình trong môi trường verify.....	75
3.3.5 Thiết kế Testcase.....	79
3.3.5.1 Kiểm tra phân hệ thanh ghi cấu hình nội bộ.....	79
3.3.5.2 Kiểm tra logic giải mã và tính toán địa chỉ AXI Burst.....	80
3.3.5.3 Kiểm tra cơ chế cấu hình lại bản đồ địa chỉ runtime.....	81
3.3.5.4 Kiểm tra cơ chế quản lý ngắt và lỗi giải mã.....	82
3.3.5.5 Kiểm tra truyền dẫn phản hồi lỗi từ Slave ngoại vi.....	83
CHƯƠNG 4: KẾT QUẢ MÔ PHỎNG VÀ KIỂM CHỨNG.....	84

4.1 Kết quả kiểm chứng mức giao thức.....	84
4.1.1 AXI Write Protocol	85
4.1.2 AXI Read Protocol	86
4.1.3 APB Write Protocol.....	87
4.1.4 APB Read Protocol.....	88
4.1.5 AXI-to-APB Transaction Flow	89
4.2 Kết quả kiểm chứng theo Verification Plan	90
4.2.1 Nhóm A – Kiểm tra thanh ghi cấu hình APB.....	90
4.2.2 Nhóm B – Kiểm tra giao dịch ghi AXI	91
4.2.3 Nhóm C – Kiểm tra giao dịch đọc AXI.....	94
4.2.4 Nhóm D – Kiểm tra stress ngẫu nhiên kết hợp đọc và ghi.....	96
4.2.5 Nhóm E – Kiểm tra cơ chế ngắt lỗi giải mã địa chỉ	97
4.2.6 Nhóm F – Kiểm tra cấu hình lại bản đồ địa chỉ động	98
4.3 Kết quả Regression.....	99
4.4 Các lỗi phát hiện trong quá trình kiểm chứng	99
4.5 Xây dựng và phân tích Coverage	109
CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	113
5.1 Kết luận.....	113
5.2 Hướng phát triển.....	114
TÀI LIỆU THAM KHẢO	115

DANH MỤC HÌNH

Hình 2.1: Các kênh giao tiếp giữa Master và Slave trong giao thức AXI.....	8
Hình 2.2: Hoạt động của giao dịch đọc trong giao thức AXI.....	12
Hình 2.3: Hoạt động của giao dịch ghi trong giao thức AXI.....	12
Hình 2.4: Tín hiệu VALID tích cực sau tín hiệu READY	13
Hình 2.5: Tín hiệu VALID tích cực trước tín hiệu READY	14
Hình 2.6: Tín hiệu VALID tích cực cùng lúc với tín hiệu READY	14
Hình 2.7: Minh họa cơ chế burst trong giao thức AXI4.....	14
Hình 2.8: Chu trình ghi không có trạng thái chờ	18
Hình 2.9: Chu trình ghi có trạng thái chờ	18
Hình 2.10: Chu trình đọc không có trạng thái chờ	19
Hình 2.11: Chu trình đọc có trạng thái chờ	19
Hình 2.12: Máy trạng thái mô tả hoạt động của giao thức APB.....	20
Hình 2.13. Cấu trúc điển hình của UVM testbench.....	21
Hình 2.14. Quan hệ kế thừa giữa các lớp trong UVM.....	22
Hình 2.15: Vị trí của UVM Testbench Top trong cấu trúc UVM Testbench	23
Hình 2.17. Vị trí của UVM Environment trong cấu trúc UVM Testbench	25
Hình 2.18. Vị trí của UVM Agent trong cấu trúc UVM Testbench	26
Hình 2.19. Vị trí của UVM Sequence trong cấu trúc UVM Testbench.....	27
Hình 2.20. Vị trí của UVM Driver trong cấu trúc UVM Testbench.....	28
Hình 2.21. Vị trí của UVM Monitor trong cấu trúc UVM Testbench.....	29
Hình 2.22. Vị trí của UVM Scoreboard trong cấu trúc UVM Testbench.....	30
Hình 2.23. Cơ chế pha trong UVM	32
Hình 3.1: Sơ đồ khối tổng thể của Runtime-Configurable AXI-to-APB Bridge	36
Hình 3.2: Sơ đồ khối AXI Clock Domain	37
Hình 3.3: Sơ đồ khối Read Address FIFO (AR FIFO).....	37
Hình 3.4: Sơ đồ khối Write Address FIFO (AW FIFO).....	38
Hình 3.5: Sơ đồ khối Write Data FIFO (W FIFO)	38
Hình 3.7: Sơ đồ khối Buffer Response	39
Hình 3.8: Sơ đồ khối APB Clock Domain.....	40
Hình 3.9: Sơ đồ khối APB_MASTER.....	40
Hình 3.10: Sơ đồ trạng thái FSM của khối RW_SELECT	41

Hình 3.11: Nguyên lý tính địa chỉ của ADDR_DECODER theo loại burst.....	42
Hình 3.12: Sơ đồ khối APB_REGISTER.....	42
Hình 3.13: Ví dụ cấu hình Base Address cho APB Slave 0	44
Hình 3.14: Cấu trúc phân cấp thư mục và các thành phần chính của môi trường ...	48
Hình 3.15: Kiến trúc tổng thể UVM Testbench của AXI-to-APB Bridge	49
Hình 3.16: Kiến trúc AXI VIP.....	50
Hình 3.17: Kiến trúc APB Slave VIP	50
Hình 3.18: Kiến trúc APB Master VIP	51
Hình 3.19: Cấu trúc interface cơ bản.....	52
Hình 3.21: Khai báo interface và kết nối DUT.....	53
Hình 3.22: Testbench tạo Clock và Reset.....	54
Hình 3.23: Testbench đăng ký UVM Config DB và khởi động UVM.....	54
Hình 3.24: Kiến trúc Testcase	55
Hình 3.25: Cấu trúc Base Test.....	56
Hình 3.26: Cấu trúc kịch bản test cụ thể.....	57
Hình 3.27: Kiến trúc Environment sau khi tích hợp VIP và Scoreboard	58
Hình 3.28: Environment khởi tạo các thành phần trong build_phase	59
Hình 3.29: Environment nối dây luồng dữ liệu (connect_phase).....	60
Hình 3.30: Agent khởi tạo các thành phần	61
Hình 3.31: Agent kết nối Driver với Sequencer	61
Hình 3.32: Kiến trúc UVM Testbench sau khi hoàn tất kết nối	62
Hình 3.33: Flowchart hoạt động của AXI Driver	63
Hình 3.34: Flowchart hoạt động của AXI Monitor	65
Hình 3.35: Flowchart hoạt động của APB Slave Driver	67
Hình 3.36: Flowchart hoạt động của APB Slave Monitor.....	68
Hình 3.37: Flowchart hoạt động của APB Master Driver	70
Hình 3.38: Flowchart hoạt động của APB Master Monitor.....	71
Hình 3.39: Flowchart hoạt động của Scoreboard	73
Hình 3.40: Sequence diagram luồng AXI WRITE burst → APB WRITE	76
Hình 3.41: Sequence diagram luồng ghi thanh ghi qua RAL Model	77
Hình 3.42: Sequence diagram luồng Decode Error và DecErrIntr.....	78
Hình 4.1: Dạng sóng AXI4 Write Burst (INCR, len=3, size=2Byte).....	86
Hình 4.2: Dạng sóng AXI4 Read Burst (WRAP, len=3, size=2Byte).....	87

Hình 4.3. Dạng sóng APB4 Write Transaction với Wait States – giao thức đúng..	88
Hình 4.4. Dạng sóng APB4 Read Transaction với Wait States – giao thức đúng...	88
Hình 4.5. Dạng sóng chuyển đổi giao dịch AXI4–APB4 qua Bridge	90
Hình 4.8: Dạng sóng chi tiết PRDATA tại vùng đọc BIR sau reset (Bug #1)	100
Hình 4.9: Mã RTL gốc tại dòng 153 – DecErrEn <= 0 (lỗi)	101
Hình 4.10: Mã RTL sau khi sửa tại dòng 153 – DecErrEn <= 1	101
Hình 4.11: Đặc tả thanh ghi BAMS1	102
Hình 4.12: Scoreboard báo lỗi	102
Hình 4.13: Dạng sóng chi tiết xác nhận ghi 00/11 vào bit [1:0] BAMS1	102
Hình 4.14: Mã RTL gốc dòng 106 – điều kiện sai BAMS2_ADDR.....	103
Hình 4.15: Dạng sóng/log bằng chứng BVALID không phản hồi	104
Hình 4.16: Mã RTL gốc – bresp_in chưa được đồng bộ hóa	104
Hình 4.17: Mã RTL sau khi sửa – thêm tầng synchronizer cho bresp_in	105
Hình 4.18: Dạng sóng kênh R – RDATA thay đổi khi RVALID=1 RREADY=0	105
Hình 4.19: Dữ liệu RDATA bị capture sai	106
Hình 4.20: Mã RTL gốc – chưa chỉnh logic RVALID	106
Hình 4.21: Mã RTL sau khi sửa logic RVALID	107
assign rden = (!rd_empty) && RREADY; và RVALID_r <= (!rd_empty).....	107
Hình 4.22: Mã RTL gốc – chưa sửa logic RLAST	108
Hình 4.23: Mã RTL sau khi sửa logic RLAST	109
Hình 4.24: Kiến trúc UVM Testbench sau khi tích hợp khối Coverage	110
Hình 4.25: Kết quả tổng hợp Functional Coverage của UVM Testbench.....	111
Hình 4.26: Kết quả tổng hợp Code Coverage trên RTL của AXI-to-APB Bridge	111

DANH MỤC BẢNG

Bảng 2.1: Các tín hiệu của kênh địa chỉ ghi	9
Bảng 2.2: Các tín hiệu của kênh dữ liệu ghi.....	10
Bảng 2.3: Các tín hiệu của kênh phản hồi ghi	10
Bảng 2.4: Các tín hiệu của kênh địa chỉ đọc.....	10
Bảng 2.5: Các tín hiệu của kênh dữ liệu đọc	11
Bảng 2.6: Các cặp tín hiệu bắt tay trong giao thức AXI4.....	13
Bảng 2.7: Các loại burst tương ứng với giá trị AxBURST.....	15
Bảng 2.8: Kích thước dữ liệu tương ứng với giá trị AxSIZE	16
Bảng 2.9: Tín hiệu của giao thức APB	17
Bảng 3.1: Bản đồ thanh ghi nội bộ của APB_REGISTER.....	43
Bảng 3.2: Mã hóa kích thước vùng địa chỉ APB Slave	43
Bảng 3.3: Cấu trúc thanh ghi BAMS0.....	44
Bảng 3.4: Cấu trúc thanh ghi BAMS1	44
Bảng 3.5: Cấu trúc thanh ghi BAMS2.....	45
Bảng 3.6: Cấu trúc thanh ghi BIR	45
Bảng 3.7: Các giai đoạn trong quy trình Design Verification	46
Bảng 3.8: Verification Plan	47
Bảng 3.9: Kịch bản kiểm tra thanh ghi cấu hình nội bộ	79
Bảng 3.10: Kịch bản kiểm tra logic giải mã và tính toán địa chỉ AXI Burst.....	80
Bảng 3.11: Kịch bản kiểm tra cơ chế cấu hình lại bản đồ địa chỉ động	81
Bảng 3.12: Kịch bản kiểm tra cơ chế quản lý ngắt và lỗi giải mã.....	82
Bảng 3.13: Kịch bản kiểm tra khả năng truyền dẫn phản hồi lỗi ngoại vi	83
Bảng 4.1: Cấu hình môi trường mô phỏng	84
Bảng 4.2: Kết quả kiểm chứng nhóm A – Thanh ghi cấu hình APB.....	91
Bảng 4.3: Kết quả kiểm chứng nhóm B – Giao dịch ghi AXI.....	92
Bảng 4.4: Kết quả kiểm chứng nhóm C – Giao dịch đọc AXI.....	94
Bảng 4.5: Kết quả kiểm chứng nhóm D – Stress ngẫu nhiên đọc/ghi	96
Bảng 4.6: Kết quả kiểm chứng nhóm E – Cơ chế ngắt Decode Error.....	97
Bảng 4.7: Kết quả kiểm chứng nhóm F – Cấu hình lại địa chỉ động.....	98
Bảng 4.8: Kết quả Regression tổng hợp theo nhóm tính năng	99

CÁC TỪ VIẾT TẮT

AMBA	Advanced Microcontroller Bus Architecture
APB	Advanced Peripheral Bus
AXI	Advanced eXtensible Interface
ASIC	Application-Specific Integrated Circuit
BAMS	Base Address & Memory Size
BIR	Bridge Interrupt Register
CDC	Clock Domain Crossing
CPU	Central Processing Unit
DMA	Direct Memory Access
DUT	Design Under Test
EDA	Electronic Design Automation
FC	Functional Coverage
FIFO	First In First Out
FPGA	Field-Programmable Gate Array
FSM	Finite State Machine
GPIO	General Purpose Input/Output
I2C	Inter-Integrated Circuit
IP	Intellectual Property
RAL	Register Abstraction Layer
RTL	Register Transfer Level
SoC	System on Chip
SPI	Serial Peripheral Interface
SVA	SystemVerilog Assertions
TLM	Transaction Level Modeling
UART	Universal Asynchronous Receiver Transmitter
UVM	Universal Verification Methodology
VIP	Verification IP
VPlan	Verification Plan

CHƯƠNG 1: TỔNG QUAN

1.1 LÝ DO CHỌN ĐỀ TÀI

Trong những năm gần đây, cùng với sự gia tăng độ phức tạp của các hệ thống System on Chip, hoạt động kiểm chứng chức năng ngày càng đóng vai trò quan trọng trong quá trình phát triển vi mạch. Nhiều nghiên cứu cho thấy verification đang trở thành một trong những công đoạn tiêu tốn nhiều nguồn lực nhất và có thể chiếm gần 70% tổng công sức phát triển sản phẩm [12]. Điều này đặt ra yêu cầu về các phương pháp kiểm chứng có tính hệ thống, khả năng tái sử dụng và dễ mở rộng.

Để đáp ứng yêu cầu đó, Universal Verification Methodology (UVM) được phát triển nhằm xây dựng một phương pháp kiểm chứng chuẩn hóa, hỗ trợ phát triển các môi trường verification có tính mô-đun, khả năng mở rộng và tái sử dụng [1]. Nhờ những đặc điểm này, UVM đã trở thành một trong những phương pháp phổ biến trong lĩnh vực Design Verification đối với các thiết kế số hiện đại.

Trong khi UVM đã trở thành phương pháp kiểm chứng được sử dụng rộng rãi trong ngành công nghiệp bán dẫn, việc xây dựng và áp dụng một quy trình verification hoàn chỉnh theo hướng tiếp cận công nghiệp vẫn là một nội dung có giá trị nghiên cứu và thực hành đối với các IP số hiện đại. Trong số đó, AXI-to-APB Bridge là một IP phổ biến trong các hệ thống SoC, đồng thời tích hợp nhiều đặc điểm đặc trưng của quá trình verification như xử lý giao thức AXI và APB, chuyển đổi giao dịch giữa hai bus, quản lý nhiều kênh dữ liệu và xử lý các trường hợp lỗi giao tiếp. Điều này giúp AXI-to-APB Bridge trở thành một đối tượng phù hợp để nghiên cứu và áp dụng quy trình kiểm chứng theo phương pháp UVM.

Bên cạnh đó, các AXI-to-APB Bridge truyền thống chủ yếu thực hiện chức năng chuyển đổi giao thức. Việc bổ sung cơ chế cấu hình động thông qua hệ thống thanh ghi không chỉ làm tăng tính linh hoạt của thiết kế mà

còn mở rộng phạm vi kiểm chứng với các chức năng liên quan đến cấu hình, ánh xạ địa chỉ và quản lý giao dịch.

Xuất phát từ những lý do trên, đề tài “THIẾT KẾ VÀ XÂY DỰNG PHƯƠNG PHÁP KIỂM CHỨNG UVM CHO RUNTIME-CONFIGURABLE AXI-TO-APB BRIDGE” được thực hiện nhằm nghiên cứu, thiết kế và xây dựng môi trường kiểm chứng dựa trên phương pháp UVM cho một kiến trúc AXI-to-APB Bridge có khả năng cấu hình động trong thời gian chạy. Đề tài không chỉ góp phần minh họa quy trình Design Verification theo hướng tiếp cận công nghiệp mà còn cung cấp một tài liệu tham khảo cho việc phát triển và kiểm chứng các register-based IP trong lĩnh vực thiết kế vi mạch số.

1.2 MỤC TIÊU NGHIÊN CỨU

Mục tiêu của luận văn là xây dựng một quy trình kiểm chứng chức năng dựa trên phương pháp UVM cho Runtime-Configurable AXI-to-APB Bridge, từ đó đánh giá tính đúng đắn của thiết kế và khả năng áp dụng của phương pháp kiểm chứng đối với các IP số có cấu trúc tương tự.

Các mục tiêu chính của luận văn bao gồm:

- Nghiên cứu, kế thừa và mở rộng kiến trúc AXI-to-APB Bridge hiện có [13] bằng cách bổ sung hệ thống thanh ghi cấu hình và cơ chế ánh xạ địa chỉ động, từ đó phát triển Runtime-Configurable AXI-to-APB Bridge.
- Xây dựng quy trình kiểm chứng theo hướng tiếp cận công nghiệp, bao gồm tổ chức cấu trúc testbench, xây dựng hệ thống script tự động hóa và phát triển môi trường kiểm chứng dựa trên phương pháp UVM.
- Thực hiện kiểm chứng chức năng cho Runtime-Configurable AXI-to-APB Bridge theo quy trình kiểm chứng đã xây dựng nhằm đánh giá tính đúng đắn và mức độ đáp ứng đặc tả của thiết kế.

Thông qua các mục tiêu trên, luận văn góp phần minh họa quy trình áp dụng phương pháp UVM trong kiểm chứng các IP số, đồng thời cung cấp một tài liệu

tham khảo cho quá trình phát triển và kiểm chứng các register-based IP trong lĩnh vực thiết kế vi mạch.

1.3 ĐỐI TƯỢNG NGHIÊN CỨU

Đối tượng nghiên cứu của luận văn là quy trình Design Verification dựa trên phương pháp UVM đối với các IP số có giao thức truyền thông và hệ thống thanh ghi cấu hình. Quy trình này bao gồm các giai đoạn phân tích thiết kế, xây dựng Verification Plan (VPlan), phát triển môi trường kiểm chứng, xây dựng testcase, thu thập coverage và thực hiện regression.

Để triển khai và đánh giá quy trình trên, luận văn sử dụng Runtime-Configurable AXI-to-APB Bridge làm thiết kế mục tiêu (Design Under Test – DUT). Thiết kế được phát triển từ một kiến trúc AXI-to-APB Bridge có sẵn và được mở rộng bằng hệ thống thanh ghi cấu hình cho phép thay đổi động bản đồ địa chỉ của các APB Slave trong thời gian chạy. Việc lựa chọn thiết kế này giúp tạo ra một đối tượng kiểm chứng có đầy đủ các đặc trưng của một register-based IP, từ đó làm cơ sở để xây dựng và đánh giá quy trình verification theo phương pháp UVM.

1.4 PHẠM VI THỰC HIỆN

Đề tài tập trung nghiên cứu, thiết kế và kiểm chứng Runtime-Configurable AXI-to-APB Bridge theo quy trình Design Verification dựa trên phương pháp UVM. Phạm vi thực hiện của đề tài bao gồm:

- Nghiên cứu kiến trúc và nguyên lý hoạt động của AXI-to-APB Bridge.
- Kế thừa thiết kế AXI-to-APB Bridge có sẵn [13] và thiết kế thêm hệ thống thanh ghi cấu hình cho phép thay đổi động bản đồ địa chỉ của các APB Slave trong thời gian chạy.
- Xây dựng Verification Plan cho thiết kế.
- Xây dựng hệ thống Script và môi trường kiểm chứng dựa trên phương pháp UVM.

- Xây dựng Register Abstraction Layer (RAL) phục vụ kiểm chứng hệ thống thanh ghi cấu hình.
- Phát triển các testcase directed và constrained-random nhằm kiểm tra các chức năng của thiết kế.
- Thực hiện mô phỏng, phân tích kết quả kiểm chứng, thu thập coverage và thực hiện regression trên Questasim.

1.5 GIỚI HẠN ĐỀ TÀI

Luận văn tập trung nghiên cứu và xây dựng quy trình kiểm chứng chức năng dựa trên phương pháp UVM cho Runtime-Configurable AXI-to-APB Bridge trong phạm vi một IP số có giao thức truyền thông và hệ thống thanh ghi cấu hình rõ ràng. Quy trình kiểm chứng [17] được đề xuất hướng tới các register-based IP thường gặp trong thiết kế vi mạch như UART, SPI, I2C, GPIO, Timer hoặc các IP sử dụng giao thức AMBA.

Do đó, phạm vi nghiên cứu không hướng tới các IP có độ phức tạp rất thấp và không có cơ chế cấu hình thông qua thanh ghi, cũng như các hệ thống xử lý phức tạp như bộ xử lý RISC-V, bộ xử lý đa lõi hoặc các hệ thống SoC hoàn chỉnh. Đối với các đối tượng này, quy trình verification thường đòi hỏi các phương pháp, mô hình tham chiếu và chiến lược kiểm chứng chuyên biệt vượt ra ngoài phạm vi của luận văn.

Bên cạnh đó, Runtime-Configurable AXI-to-APB Bridge được nghiên cứu trong luận văn chỉ tập trung vào các cơ chế truyền dữ liệu cơ bản của giao thức AXI. Các đặc tính nâng cao như AXI ID, multiple outstanding transaction, out-of-order transaction hoặc các cơ chế tối ưu hiệu năng phức tạp khác không được xem xét trong phạm vi thiết kế và kiểm chứng của đề tài.

1.6 PHƯƠNG PHÁP NGHIÊN CỨU

Để thực hiện luận văn, đề tài sử dụng phương pháp nghiên cứu kết hợp giữa khảo sát lý thuyết, thiết kế hệ thống và kiểm chứng chức năng. Trước hết, các kiến thức nền liên quan đến kiến trúc SoC, chuẩn AMBA, giao thức AXI, APB

và phương pháp UVM được nghiên cứu, phân tích và tổng hợp nhằm làm cơ sở cho quá trình phát triển thiết kế và xây dựng môi trường kiểm chứng.

Trên cơ sở đó, luận văn tiến hành nghiên cứu, kế thừa và mở rộng kiến trúc AXI-to-APB Bridge hiện có để phát triển Runtime-Configurable AXI-to-APB Bridge thông qua cơ chế cấu hình động bản đồ địa chỉ APB Slave bằng hệ thống thanh ghi nội bộ.

Tiếp theo, quy trình kiểm chứng được xây dựng theo hướng tiếp cận công nghiệp, bao gồm phân tích đặc tả thiết kế, xác định phạm vi kiểm chứng, xây dựng Verification Plan, phát triển môi trường kiểm chứng dựa trên phương pháp UVM và xây dựng các kịch bản kiểm chứng phù hợp với mục tiêu đề ra.

Cuối cùng, quá trình kiểm chứng được thực hiện thông qua mô phỏng và phân tích kết quả nhằm đánh giá tính đúng đắn và mức độ đáp ứng đặc tả của Runtime-Configurable AXI-to-APB Bridge.

1.7 BỐ CỤC CỦA ĐỀ TÀI

Luận văn được tổ chức thành năm chương chính như sau:

Chương 1. Tổng quan: Trình bày bối cảnh nghiên cứu, mục tiêu nghiên cứu, giới hạn của đề tài, phương pháp nghiên cứu và bố cục tổng thể của luận văn.

Chương 2. Cơ sở lý thuyết: Trình bày các kiến thức nền phục vụ cho quá trình thiết kế và kiểm chứng, bao gồm kiến trúc SoC, chuẩn AMBA, giao thức AXI, APB, phương pháp Functional Verification và các thành phần quan trọng của phương pháp UVM.

Chương 3. Phân tích thiết kế và xây dựng môi trường kiểm chứng UVM: Trình bày quá trình phân tích thiết kế, kiến trúc RTL của Runtime-Configurable AXI-to-APB Bridge, cơ chế cấu hình động bản đồ địa chỉ APB Slave và quy trình xây dựng môi trường kiểm chứng theo phương pháp UVM, bao gồm VPlan, kiến trúc testbench và các thành phần kiểm chứng chính.

Chương 4. Kết quả mô phỏng và kiểm chứng: Trình bày kết quả thực hiện kiểm chứng chức năng đối với Runtime-Configurable AXI-to-APB Bridge, phân

tích kết quả mô phỏng và đánh giá mức độ đáp ứng đặc tả của thiết kế dựa trên các tiêu chí kiểm chứng đã đề ra.

Chương 5. Kết luận và hướng phát triển: Tổng kết các kết quả đạt được của luận văn, những hạn chế còn tồn tại và đề xuất các hướng nghiên cứu, phát triển trong tương lai.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1 TỔNG QUAN VỀ SYSTEM ON CHIP VÀ AMBA

SoC là một vi mạch tích hợp hầu hết hoặc toàn bộ các thành phần chức năng của một hệ thống điện tử trên cùng một đế silicon, bao gồm bộ xử lý, bộ nhớ, các giao diện truyền thông và các khối xử lý chuyên dụng khác [15]. Việc tích hợp nhiều thành phần trên cùng một vi mạch giúp giảm kích thước hệ thống, nâng cao hiệu năng và tối ưu mức tiêu thụ năng lượng. Nhờ đó, SoC được sử dụng rộng rãi trong các hệ thống nhúng, thiết bị di động, IoT và nhiều ứng dụng điện tử hiện đại.

Trong một hệ thống SoC, các IP thường có yêu cầu khác nhau về tốc độ truyền dữ liệu và cơ chế giao tiếp. Để đảm bảo khả năng kết nối và tương tác giữa các thành phần, ARM đã phát triển chuẩn AMBA (Advanced Microcontroller Bus Architecture), cung cấp các giao thức bus chuẩn hóa cho việc truyền dữ liệu trong hệ thống. Trong đó, AXI được sử dụng cho các khối yêu cầu hiệu năng cao, còn APB phù hợp với các ngoại vi đơn giản và các thanh ghi điều khiển [5], [6].

Do sự khác biệt về đặc tính hoạt động giữa AXI và APB, các hệ thống SoC thường sử dụng AXI-to-APB Bridge để thực hiện chuyển đổi giao thức giữa hai miền giao tiếp này. Trong phạm vi luận văn, AXI-to-APB Bridge được lựa chọn làm đối tượng nghiên cứu và phát triển thành Runtime-Configurable AXI-to-APB Bridge, làm nền tảng cho việc xây dựng và áp dụng quy trình kiểm chứng dựa trên phương pháp UVM.

2.2 GIAO THỨC AMBA AXI

Các nội dung trình bày trong mục này được tổng hợp và diễn giải dựa trên tài liệu AMBA AXI Protocol Specification của ARM.

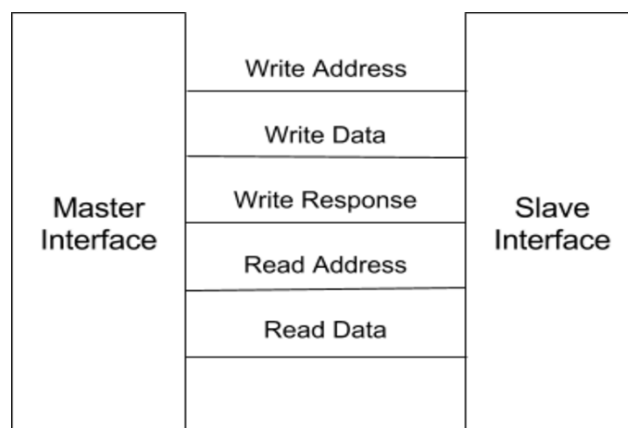
AXI (Advanced eXtensible Interface) là giao thức bus thuộc chuẩn AMBA do ARM phát triển, được thiết kế nhằm đáp ứng nhu cầu truyền dữ liệu tốc độ

cao trong các hệ thống SoC hiện đại. So với các thế hệ bus trước đó trong họ AMBA, AXI cung cấp khả năng truyền dữ liệu hiệu quả hơn nhờ hỗ trợ băng thông lớn, độ trễ thấp và khả năng mở rộng linh hoạt. Vì vậy, AXI thường được sử dụng để kết nối các thành phần yêu cầu hiệu năng cao như bộ xử lý trung tâm (CPU), bộ điều khiển DMA, bộ nhớ và các IP xử lý dữ liệu chuyên dụng [5].

Một trong những đặc điểm nổi bật của AXI là việc tách biệt các kênh địa chỉ, dữ liệu và phản hồi thành các đường truyền độc lập, cho phép nhiều giao dịch được thực hiện song song nhằm nâng cao hiệu suất truyền dữ liệu. Bên cạnh đó, AXI còn hỗ trợ cơ chế truyền dữ liệu theo burst, giúp giảm số lần truy cập bus và tối ưu băng thông hệ thống. Nhờ những đặc điểm này, AXI hiện là một trong những giao thức bus được sử dụng phổ biến nhất trong các hệ thống SoC dựa trên kiến trúc AMBA [5].

2.2.1 Các kênh truyền dữ liệu trong AXI

Trong giao thức AXI, mỗi lần truy cập đọc hoặc ghi đều được gọi là một giao dịch. AXI hoạt động dựa trên năm kênh độc lập, mỗi kênh đảm nhiệm một chức năng riêng biệt. Mặc dù hoạt động độc lập, các kênh này phối hợp chặt chẽ để đảm bảo tính toàn vẹn và hiệu quả của toàn bộ giao dịch. Hình 2.1 mô tả hai khối Master và Slave được kết nối qua năm kênh này.



Hình 2.1: Các kênh giao tiếp giữa Master và Slave trong giao thức AXI

Năm kênh giao tiếp của AXI bao gồm:

- Kênh địa chỉ ghi (Write Address Channel): truyền thông tin địa chỉ và các tham số điều khiển liên quan đến giao dịch ghi từ Master đến Slave.
- Kênh dữ liệu ghi (Write Data Channel): truyền dữ liệu cần ghi từ Master đến Slave.
- Kênh phản hồi ghi (Write Response Channel): truyền thông tin xác nhận kết quả giao dịch ghi từ Slave về Master sau khi hoàn tất.
- Kênh địa chỉ đọc (Read Address Channel): truyền thông tin địa chỉ và các tham số điều khiển của giao dịch đọc từ Master đến Slave.
- Kênh dữ liệu đọc (Read Data Channel): truyền dữ liệu đọc và thông tin phản hồi từ Slave về Master.

Các tín hiệu trong từng kênh được giao thức AXI quy định rõ ràng và trình bày chi tiết trong các Bảng 2.1 đến 2.5.

Bảng 2.1: Các tín hiệu của kênh địa chỉ ghi

TÍN HIỆU	NGUỒN
AWID	Master
AWADDR	Master
AWLEN	Master
AWSIZE	Master
AWBURST	Master
AWLOCK	Master
AWCACHE	Master
AWPROT	Master
AWQOS	Master
AWREGION	Master
AWUSER	Master
AWVALID	Master
AWREADY	Slave

Bảng 2.1 liệt kê các tín hiệu thuộc kênh địa chỉ ghi. Master sử dụng các tín hiệu này để gửi địa chỉ đích và các thuộc tính bổ sung như độ dài burst, kích thước dữ liệu, kiểu truy cập, bảo mật và chất lượng dịch vụ đến Slave. Hầu hết

tín hiệu đều xuất phát từ Master, ngoại trừ AWREADY do Slave điều khiển để báo hiệu trạng thái sẵn sàng nhận thông tin.

Bảng 2.2: Các tín hiệu của kênh dữ liệu ghi

TÍN HIỆU	NGUỒN
WID	Master
WDATA	Master
WSTRB	Master
WLAST	Master
WUSER	Master
WVALID	Master
WREADY	Slave

Bảng 2.2 mô tả các tín hiệu thuộc kênh dữ liệu ghi. Master truyền dữ liệu thực tế đến Slave thông qua kênh này. Các tín hiệu WDATA,WSTRB và WLAST lần lượt mang nội dung dữ liệu, thông tin byte enable và dấu hiệu kết thúc burst. Tín hiệu WREADY do Slave điều khiển để xác nhận khả năng tiếp nhận dữ liệu.

Bảng 2.3: Các tín hiệu của kênh phản hồi ghi

TÍN HIỆU	NGUỒN
BID	Slave
BRESP	Slave
BUSER	Slave
BVALID	Slave
BREADY	Master

Bảng 2.3 trình bày các tín hiệu thuộc kênh phản hồi ghi. Sau khi giao dịch ghi hoàn tất, Slave sử dụng kênh này để thông báo kết quả về cho Master thông qua tín hiệu BRESP. Master điều khiển tín hiệu BREADY để xác nhận đã nhận được phản hồi.

Bảng 2.4: Các tín hiệu của kênh địa chỉ đọc

TÍN HIỆU	NGUỒN
ARID	Master
ARADDR	Master
ARLEN	Master
ARSIZE	Master

ARBURST	Master
ARLOCK	Master
ARCACHE	Master
ARPROT	Master
ARQOS	Master
ARREGION	Master
ARUSER	Master
ARVALID	Master
ARREADY	Slave

Bảng 2.4 mô tả các tín hiệu của kênh địa chỉ đọc. Kênh này chịu trách nhiệm truyền thông tin địa chỉ và các tham số điều khiển từ Master đến Slave để khởi tạo giao dịch đọc. Hầu hết tín hiệu đều do Master điều khiển; tín hiệu ARREADY do Slave điều khiển để báo sẵn sàng nhận yêu cầu.

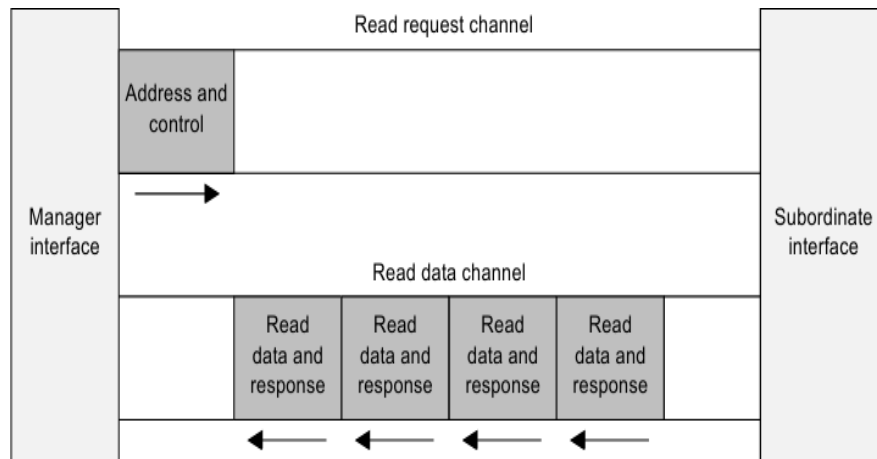
Bảng 2.5: Các tín hiệu của kênh dữ liệu đọc

TÍN HIỆU	NGUỒN
RID	Slave
RDATA	Slave
RRESP	Slave
RLAST	Slave
RUSER	Slave
RVALID	Slave
RREADY	Master

Bảng 2.5 liệt kê các tín hiệu của kênh dữ liệu đọc. Sau khi Slave nhận được yêu cầu đọc, nó truyền dữ liệu về Master qua kênh này. Các tín hiệu RDATA, RRESP và RLAST lần lượt mang dữ liệu, mã phản hồi và dấu hiệu kết thúc burst. Master điều khiển RREADY để xác nhận khả năng tiếp nhận.

2.2.2 Chu trình đọc và ghi của giao thức AXI

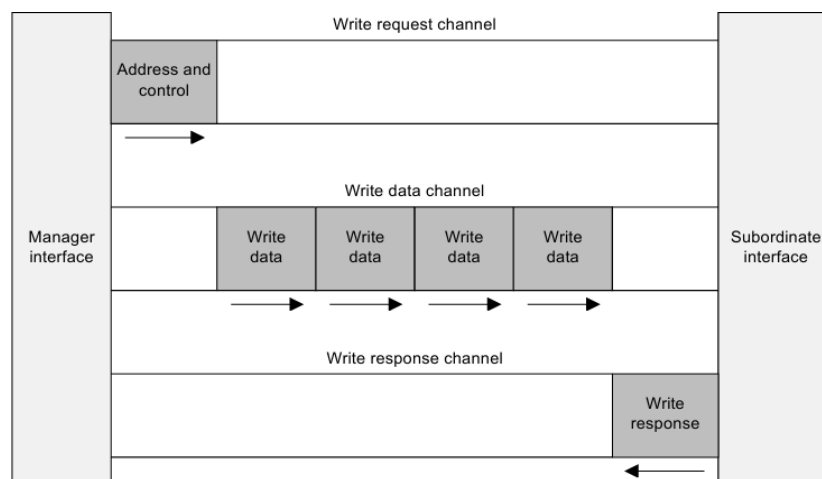
Trong giao thức AXI, mỗi truy cập đọc hoặc ghi đều được xem là một giao dịch độc lập. Một giao dịch đọc được thực hiện qua hai kênh: kênh địa chỉ đọc và kênh dữ liệu đọc. Hình 2.2 minh họa trình tự hoạt động của một giao dịch đọc.



Hình 2.2: Hoạt động của giao dịch đọc trong giao thức AXI

Một giao dịch đọc diễn ra theo hai bước chính. Đầu tiên, Master gửi địa chỉ cần đọc cùng các thông tin điều khiển lên kênh địa chỉ đọc để khởi động giao dịch. Tiếp theo, Slave truyền dữ liệu và mã phản hồi về cho Master thông qua kênh dữ liệu đọc; số lượng dữ liệu được truyền phụ thuộc vào các tham số điều khiển đã được gửi ở bước đầu.

Tương tự giao dịch đọc, một giao dịch ghi được thực hiện qua ba kênh: kênh địa chỉ ghi, kênh dữ liệu ghi và kênh phản hồi ghi. Hình 2.3 minh họa trình tự hoạt động tương ứng.



Hình 2.3: Hoạt động của giao dịch ghi trong giao thức AXI

Một giao dịch ghi gồm ba bước. Bước một, Master gửi địa chỉ và thông tin điều khiển lên kênh địa chỉ ghi. Bước hai, dữ liệu cần ghi được truyền từ Master

đến Slave qua kênh dữ liệu ghi theo số lượng beat đã quy định. Bước ba, Slave gửi tín hiệu phản hồi về cho Master qua kênh phản hồi ghi để xác nhận kết quả toàn bộ giao dịch.

2.2.3 Cơ chế bắt tay của AXI

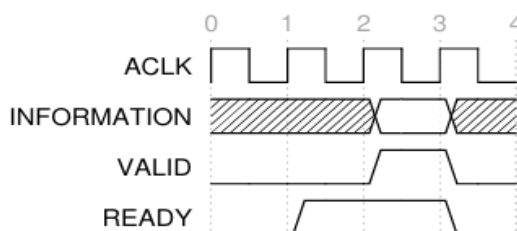
Giao thức AXI hoạt động dựa trên cơ chế bắt tay hai chiều, trong đó quá trình truyền dữ liệu trên mỗi kênh được điều phối thông qua cặp tín hiệu VALID và READY. Bên phát kích hoạt VALID khi có dữ liệu hợp lệ và duy trì tín hiệu này cho đến khi bên nhận kích hoạt READY. Quá trình truyền chỉ xảy ra tại thời điểm cả VALID và READY cùng ở mức tích cực trên cạnh lên của xung nhịp.

Cơ chế bắt tay của AXI4 cho phép ba tình huống phát sinh tự nhiên trong quá trình vận hành. Bảng 2.6 trình bày các cặp tín hiệu bắt tay tương ứng với từng kênh.

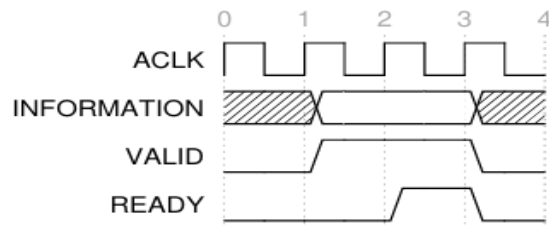
Bảng 2.6: Các cặp tín hiệu bắt tay trong giao thức AXI4

KÊNH	CẶP TÍN HIỆU BẮT TAY
Kênh địa chỉ ghi	AWVALID – AWREADY
Kênh dữ liệu ghi	WVALID – WREADY
Kênh phản hồi ghi	BVALID – BREADY
Kênh địa chỉ đọc	ARVALID – ARREADY
Kênh dữ liệu đọc	RVALID – RREADY

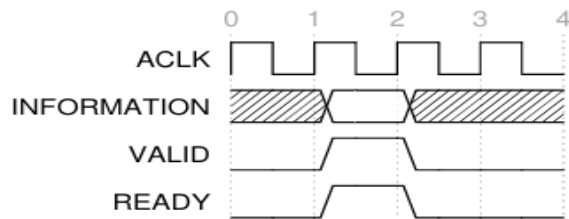
Trong thực tế vận hành, ba tình huống bắt tay phổ biến có thể xảy ra tùy theo thời điểm hai phía sẵn sàng. Hình 2.4, 2.5 và 2.6 lần lượt minh họa các trường hợp VALID tích cực sau READY, VALID tích cực trước READY và cả hai cùng tích cực đồng thời.



Hình 2.4: Tín hiệu VALID tích cực sau tín hiệu READY



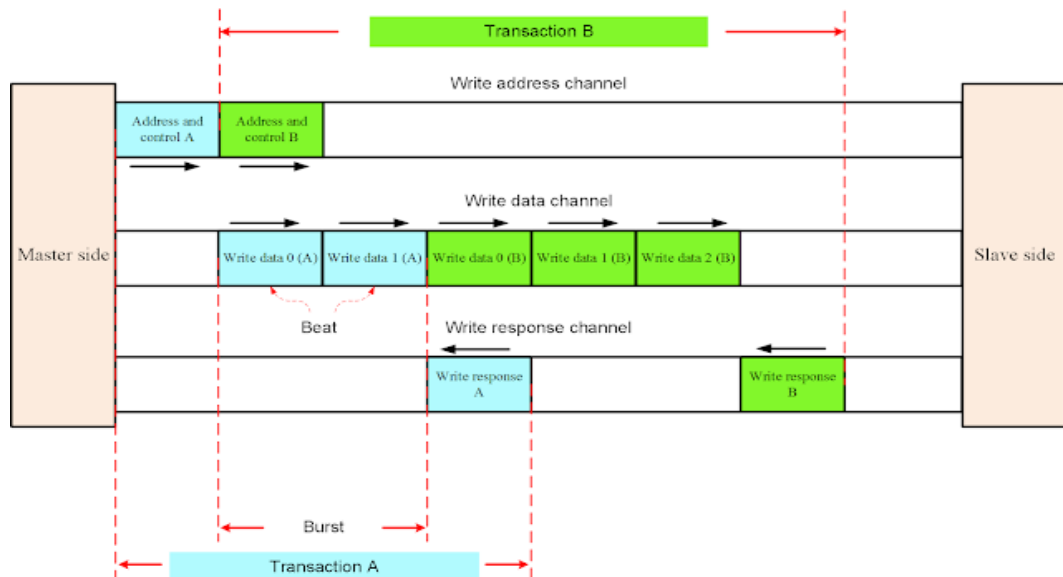
Hình 2.5: Tín hiệu VALID tích cực trước tín hiệu READY



Hình 2.6: Tín hiệu VALID tích cực cùng lúc với tín hiệu READY

2.2.4 Cơ chế burst của AXI

Giao thức AXI sử dụng cơ chế truyền dữ liệu theo kiểu burst, cho phép truyền nhiều beat dữ liệu liên tiếp trong cùng một giao dịch mà không cần phát lại địa chỉ cho từng beat. Hình 2.7 minh họa tổng quan cơ chế burst trong AXI.



Hình 2.7: Minh họa cơ chế burst trong giao thức AXI4

Loại burst được xác định thông qua tín hiệu AxBURST. Giao thức AXI4 hỗ trợ ba loại burst chính:

– FIXED: Địa chỉ của tất cả các beat trong burst đều giống nhau, không thay đổi theo từng lần truyền. Loại burst này thường dùng cho các giao dịch truy cập lặp lại vào cùng một địa chỉ.

– INCR: Địa chỉ tăng tuyến tính sau mỗi beat theo kích thước transfer được quy định bởi AxSIZE. Công thức tính địa chỉ:

$$Aligned_Addr = INT(Start_Addr / Size) \times Size \quad (2.1)$$

$$Address_N = Start_Address + (N - 1) \times Size \quad (2.2)$$

Trong đó: Start_Addr là địa chỉ bắt đầu; INT(x) là phép làm tròn xuống; N là số thứ tự beat; Size = 2^{AxSIZE} là kích thước mỗi beat tính theo byte.

– WRAP: Địa chỉ tăng tuyến tính tương tự INCR, nhưng khi đạt đến biên cuộn (wrap boundary) sẽ quay về đầu vùng địa chỉ căn chỉnh và tiếp tục tăng. Công thức tính:

$$Wrap_Boundary = INT(Start_Addr / (Size \times Length)) \times Size \times Length \quad (2.3)$$

$$Address_N = Aligned_Addr + (N - 1) \times Size \quad (khi\ chưa\ vượt\ biên) \quad (2.4)$$

$$Address_N = Start_Addr + (N - 1) \times Size - (Size \times Length) \quad (2.5)$$

(khi vượt biên)

Trong đó: Length = AxLEN + 1 là tổng số beat trong burst.

Bảng 2.7: Các loại burst tương ứng với giá trị AxBURST

AxBURST[1:0]	Loại burst	Mô tả
2'b00	FIXED	Địa chỉ không đổi qua các beat
2'b01	INCR	Địa chỉ tăng tuyến tính mỗi beat
2'b10	WRAP	Địa chỉ tăng và quay vòng trong vùng căn chỉnh
2'b11	Reserved	Dự phòng, không sử dụng

Độ dài một burst được tính theo công thức:

$$Số\ beat\ trong\ burst = AxLEN + 1 \quad (2.6)$$

Bảng 2.8: Kích thước dữ liệu tương ứng với giá trị AxSIZE

AxSIZE[2:0]	Số byte mỗi beat	Ghi chú
3'b000	1 byte	
3'b001	2 byte	
3'b010	4 byte	Phổ biến với bus 32-bit
3'b011	8 byte	
3'b100	16 byte	
3'b101	32 byte	
3'b110	64 byte	
3'b111	128 byte	

2.3 GIAO THỨC AMBA APB

Các nội dung trình bày trong mục này được tổng hợp và diễn giải dựa trên tài liệu AMBA APB Protocol Specification của ARM [6].

APB (Advanced Peripheral Bus) là giao thức bus thuộc chuẩn AMBA do ARM phát triển, được thiết kế cho các thiết bị ngoại vi có yêu cầu băng thông thấp và cấu trúc điều khiển đơn giản [6]. So với AXI, APB không hỗ trợ pipeline, burst transfer hoặc nhiều giao dịch đồng thời, nhờ đó giúp giảm độ phức tạp phần cứng và tiết kiệm tài nguyên hệ thống.

Trong các hệ thống SoC, APB thường được sử dụng để kết nối các ngoại vi như UART, SPI, I2C, GPIO, Timer hoặc các khối thanh ghi điều khiển. APB thường được đặt phía sau các bus hiệu năng cao thông qua các cầu chuyển đổi giao thức nhằm đảm bảo sự cân bằng giữa hiệu năng truyền dữ liệu và chi phí phần cứng [6].

2.3.1 Tín hiệu của giao thức APB

Giao thức APB sử dụng tập tín hiệu đơn giản để thực hiện các giao dịch đọc và ghi giữa Master và Slave. Bảng 2.9 liệt kê các tín hiệu chính cùng mô tả chức năng tương ứng.

Bảng 2.9: Tín hiệu của giao thức APB

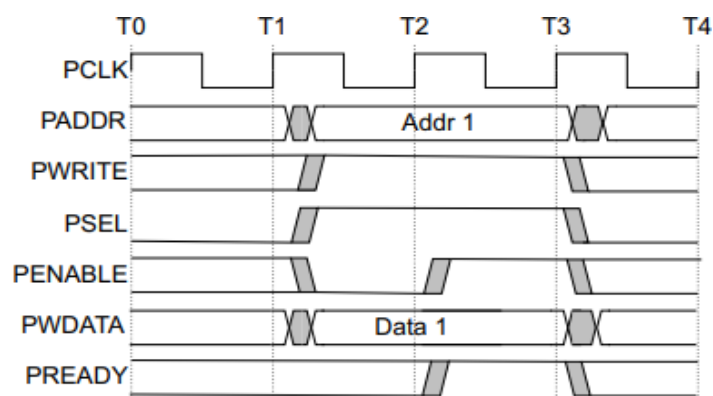
TÍN HIỆU	MÔ TẢ
PCLK	Tín hiệu xung nhịp, tất cả tín hiệu được lấy mẫu theo cạnh lên
PRESETn	Tín hiệu reset tích cực mức thấp, dùng để khởi tạo lại hệ thống
PADDR	Địa chỉ thanh ghi hoặc ngoại vi mà Master muốn truy cập
PSELx	Tín hiệu chọn Slave; mỗi Slave tương ứng một tín hiệu PSEL riêng
PENABLE	Tín hiệu báo hiệu giai đoạn ACCESS đang diễn ra
PWRITE	Xác định hướng giao dịch: 1 = Ghi, 0 = Đọc
PWDATA	Dữ liệu ghi từ Master đến Slave
PRDATA	Dữ liệu đọc từ Slave trả về Master
PREADY	Tín hiệu từ Slave báo hiệu đã sẵn sàng hoàn tất giao dịch
PSLVERR	Tín hiệu báo lỗi, kích hoạt khi Slave phát hiện lỗi giao dịch

2.3.2 Chu trình ghi và đọc của giao thức APB

Giao thức APB hỗ trợ hai loại giao dịch cơ bản là ghi (write) và đọc (read). Mỗi giao dịch đều được thực hiện qua hai giai đoạn chính là SETUP và ACCESS. Trong giai đoạn SETUP, Master lựa chọn Slave thông qua tín hiệu PSELx và thiết lập các tín hiệu địa chỉ, dữ liệu và điều khiển cần thiết. Sang giai đoạn ACCESS, tín hiệu PENABLE được kích hoạt để bắt đầu quá trình trao đổi dữ liệu giữa Master và Slave. Tùy thuộc vào trạng thái của tín hiệu PREADY, giao dịch có thể hoàn tất ngay hoặc bị kéo dài thêm một hoặc nhiều chu kỳ xung nhịp.

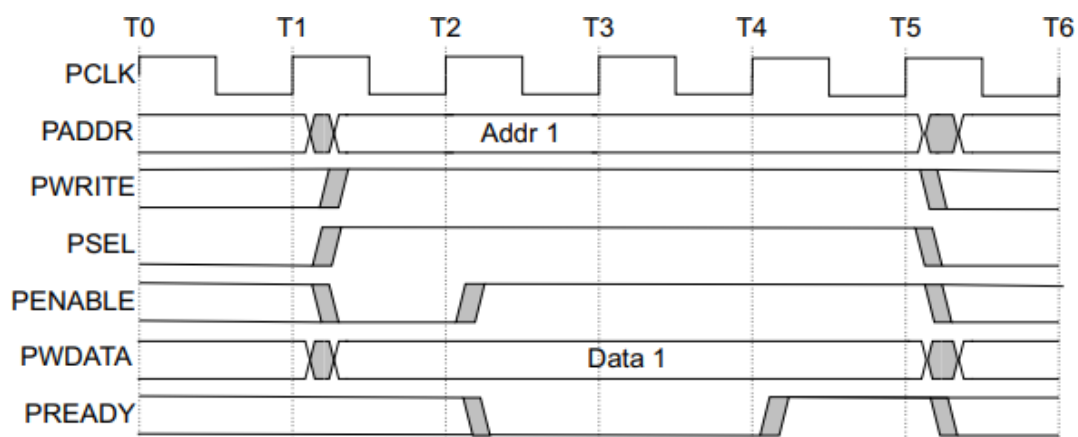
2.3.2.1 Giao dịch ghi

Hình 2.8 minh họa chu trình ghi không có trạng thái chờ trong giao thức APB. Sau khi Master lựa chọn Slave và cung cấp địa chỉ truy cập thông qua tín hiệu PADDR cùng dữ liệu ghi trên tín hiệu PWDATA, giao dịch chuyển sang giai đoạn ACCESS khi tín hiệu PENABLE được kích hoạt. Trong trường hợp Slave luôn sẵn sàng xử lý yêu cầu, tín hiệu PREADY được kích hoạt ngay, cho phép giao dịch hoàn tất chỉ sau một chu kỳ ACCESS.



Hình 2.8: Chu trình ghi không có trạng thái chờ

Hình 2.9 minh họa trường hợp giao dịch ghi có trạng thái chờ. Trong trường hợp này, Slave chưa sẵn sàng tiếp nhận dữ liệu nên tín hiệu PREADY được giữ ở mức thấp trong một hoặc nhiều chu kỳ xung nhịp. Khi đó, Master phải duy trì ổn định các tín hiệu địa chỉ, dữ liệu và điều khiển cho đến khi Slave kích hoạt PREADY. Sau khi PREADY được xác nhận, giao dịch ghi được hoàn tất và bus trở về trạng thái chờ để chuẩn bị cho giao dịch tiếp theo.

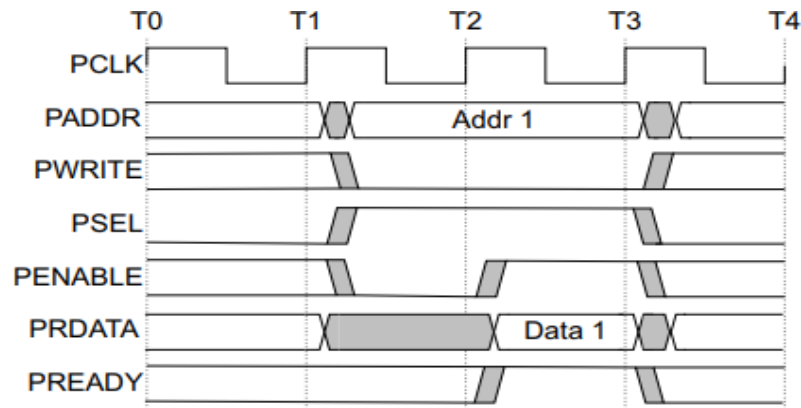


Hình 2.9: Chu trình ghi có trạng thái chờ

2.3.2.2 Giao dịch đọc

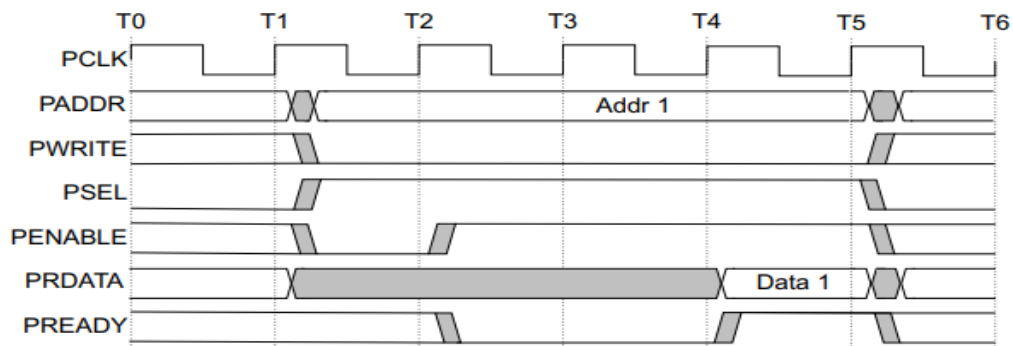
Tương tự giao dịch ghi, giao dịch đọc cũng có thể xảy ra với hoặc không có trạng thái chờ. Hình 2.10 minh họa chu trình đọc không có trạng thái chờ. Sau khi Master gửi địa chỉ cần truy cập trong giai đoạn SETUP, giao dịch chuyển sang giai đoạn ACCESS. Slave phản hồi ngay bằng cách cung cấp dữ liệu trên tín

hiệu PRDATA và kích hoạt PREADY, cho phép Master nhận dữ liệu và hoàn tất giao dịch.



Hình 2.10: Chu trình đọc không có trạng thái chờ

Hình 2.11 minh họa trường hợp giao dịch đọc có trạng thái chờ. Khi dữ liệu chưa sẵn sàng, Slave giữ tín hiệu PREADY ở mức thấp, làm kéo dài giai đoạn ACCESS. Trong thời gian này, các tín hiệu địa chỉ và điều khiển vẫn phải được giữ ổn định. Khi dữ liệu đã sẵn sàng, Slave đưa dữ liệu lên tín hiệu PRDATA và kích hoạt PREADY để hoàn tất giao dịch đọc.

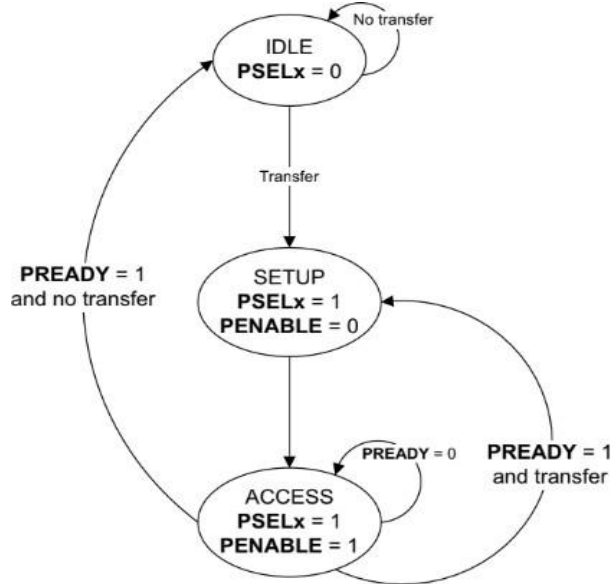


Hình 2.11: Chu trình đọc có trạng thái chờ

Từ các trường hợp trên có thể thấy tín hiệu PREADY đóng vai trò điều khiển thời điểm hoàn tất giao dịch trong giao thức APB. Cơ chế này cho phép APB hỗ trợ các ngoại vi có tốc độ xử lý khác nhau mà vẫn đảm bảo tính đơn giản của giao thức.

2.3.3 Trạng thái hoạt động của giao thức APB

Giao thức APB hoạt động đồng bộ theo cạnh lên của PCLK và được mô tả bởi máy trạng thái ba trạng thái: IDLE, SETUP và ACCESS, như minh họa tại



Hình 2.12: Máy trạng thái mô tả hoạt động của giao thức APB

Trạng thái IDLE là trạng thái mặc định khi bus APB không có giao dịch nào đang thực hiện. Lúc này, PSELx ở mức thấp và không có Slave nào được chọn.

Khi có yêu cầu truy cập mới, hệ thống chuyển sang trạng thái SETUP: PSELx được kích hoạt, đồng thời các tín hiệu PADDR, PWRITE, PWDATA (với giao dịch ghi) hoặc PRDATA (với giao dịch đọc) được thiết lập ổn định. PENABLE vẫn giữ ở mức thấp trong trạng thái này.

Ở chu kỳ xung nhịp tiếp theo, hệ thống chuyển sang trạng thái ACCESS: PENABLE được kích hoạt mức cao. Giao dịch hoàn tất khi Slave assert PREADY. Nếu PREADY ở mức thấp, trạng thái ACCESS được kéo dài thêm cho đến khi PREADY tích cực. Sau khi hoàn tất, hệ thống trở về trạng thái IDLE để chuẩn bị cho giao dịch tiếp theo.

2.4 TỔNG QUAN VỀ PHƯƠNG PHÁP KIỂM CHỨNG UVM

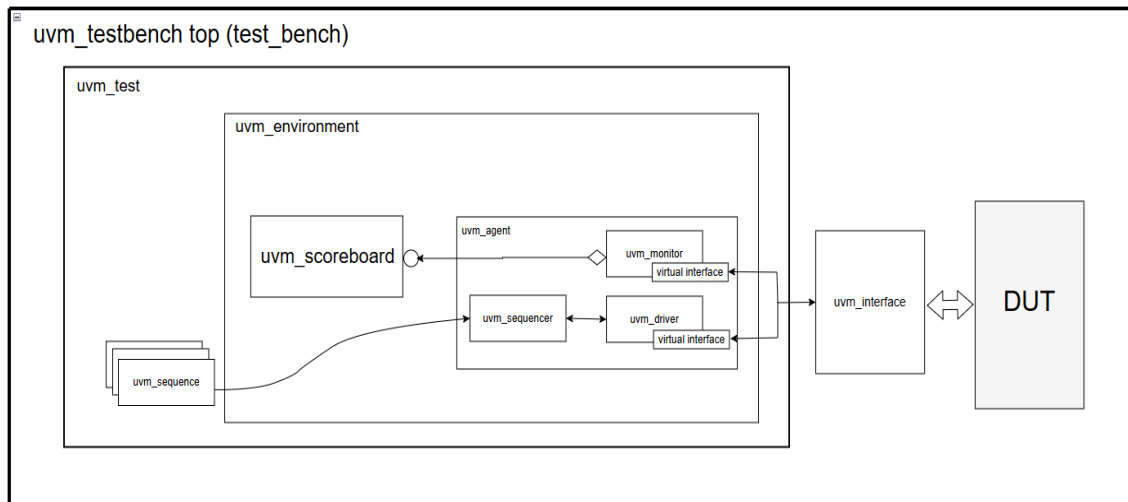
UVM là phương pháp kiểm chứng chức năng dựa trên ngôn ngữ SystemVerilog, được phát triển bởi Accellera nhằm chuẩn hóa quy trình xây

dụng môi trường verification cho các thiết kế số phức tạp. UVM cung cấp một bộ thư viện chuẩn hỗ trợ xây dựng các môi trường kiểm chứng có tính tái sử dụng, khả năng mở rộng và tương thích cao giữa các công cụ EDA khác nhau. Nhờ những ưu điểm này, UVM hiện là một trong những phương pháp kiểm chứng được sử dụng phổ biến trong lĩnh vực Design Verification của ngành công nghiệp bán dẫn [1], [2].

Trong thực tế, UVM được ứng dụng rộng rãi trong việc kiểm chứng các giao thức AMBA và nhiều IP phức tạp nhờ khả năng hỗ trợ kiểm chứng hướng đối tượng, Constrained Random Verification và Coverage-Driven Verification [3], [4]

2.4.1 Cấu trúc UVM Testbench

UVM cung cấp các thư viện lớp cần thiết để xây dựng môi trường kiểm chứng có tính linh hoạt và tái sử dụng cao [1], [2]. Thư viện bao gồm các tiện ích về cấu hình, giao dịch và giao tiếp, cho phép tổ chức các thành phần kiểm chứng theo một cấu trúc phân cấp rõ ràng. Hình 2.13 minh họa cấu trúc điển hình của một UVM testbench.



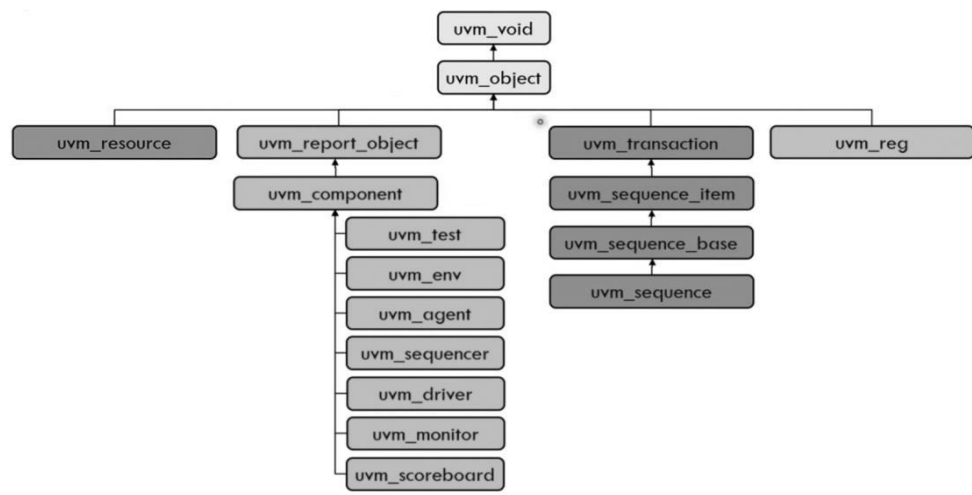
Hình 2.13. Cấu trúc điển hình của UVM testbench

Trong một UVM testbench, các thành phần được xây dựng từ lớp cơ sở `uvm_component` tạo nên cấu trúc phân cấp ổn định và được duy trì xuyên suốt quá trình mô phỏng. Các thành phần này bao gồm `uvm_test`, `uvm_env`,

uvm_agent, uvm_driver, uvm_monitor và uvm_scoreboard, đóng vai trò điều phối toàn bộ hoạt động kiểm chứng từ sinh kích thích đến kiểm tra kết quả.

Ngược lại, các đối tượng như uvm_sequence_item và uvm_sequence được kế thừa từ lớp uvm_object và chỉ tồn tại trong thời gian thực hiện các kịch bản kiểm tra. Các đối tượng này được tạo động trong quá trình mô phỏng để mô tả các giao dịch và kích thích cần gửi tới DUT.

Hình 2.14 minh họa mối quan hệ kế thừa giữa các lớp cơ sở trong thư viện UVM. Sự phân tách giữa uvm_component và uvm_object giúp UVM xây dựng môi trường kiểm chứng theo mô hình phân cấp rõ ràng, đồng thời hỗ trợ việc trao đổi dữ liệu ở mức giao dịch một cách linh hoạt và hiệu quả [1], [2].



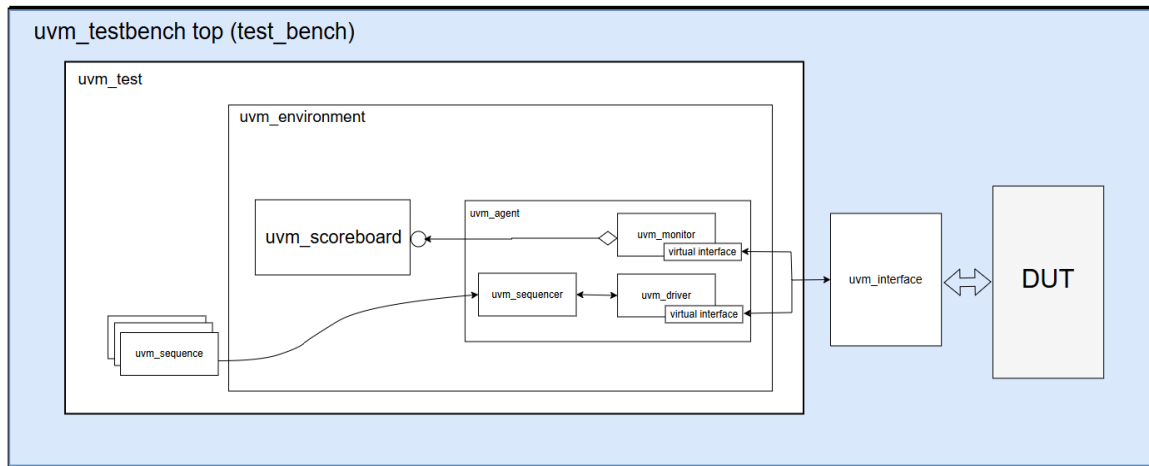
Hình 2.14. Quan hệ kế thừa giữa các lớp trong UVM

2.4.2 Các lớp cấu trúc UVM Testbench

2.4.2.1 UVM Testbench Top

UVM Testbench Top là mô-đun ở mức cao nhất của môi trường kiểm chứng, đóng vai trò kết nối giữa DUT và các thành phần UVM. Thành phần này chịu trách nhiệm khởi tạo DUT, các interface và môi trường UVM, đồng thời kích hoạt quá trình mô phỏng thông qua lệnh run_test().

Hình 2.15 minh họa vị trí của UVM Testbench Top trong cấu trúc tổng thể của một UVM testbench.



Hình 2.15: Vị trí của UVM Testbench Top trong cấu trúc UVM Testbench

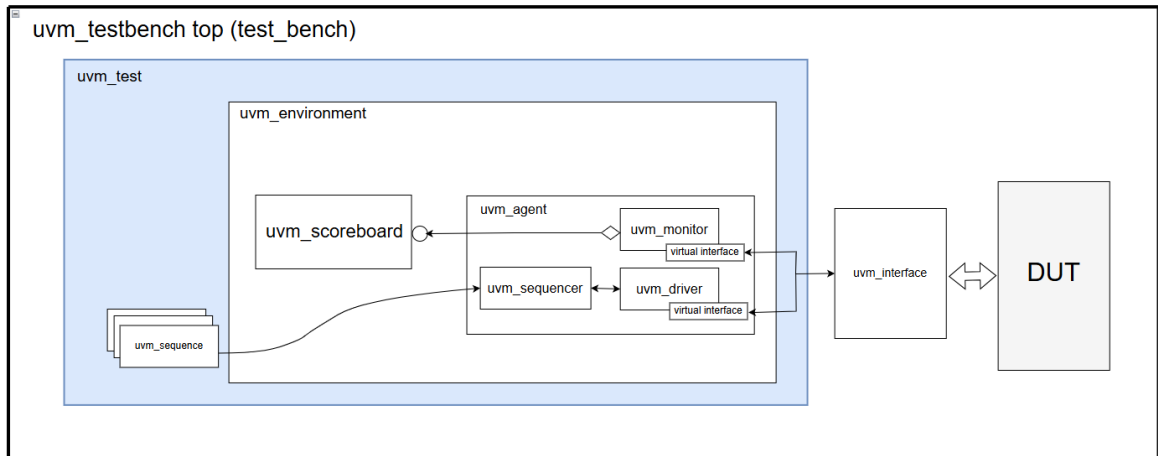
UVM Testbench Top thường thực hiện các chức năng chính sau:

- Tạo xung nhịp.
- Khởi tạo và giải phóng tín hiệu reset.
- Khởi tạo các interface kết nối với DUT.
- Khởi tạo DUT và các thành phần cần thiết của testbench.
- Kích hoạt thực thi testcase thông qua cơ chế run_test().

Thông qua các interface, các thành phần như driver và monitor có thể truy cập tín hiệu của DUT để thực hiện việc phát kích thích và thu thập dữ liệu phục vụ cho quá trình kiểm chứng. Nhờ đó, UVM Testbench Top đóng vai trò là cầu nối giữa môi trường kiểm chứng và thiết kế cần kiểm chứng [1], [2].

2.4.2.2 UVM Test

UVM Test là thành phần ở mức cao nhất trong cấu trúc phân cấp của môi trường kiểm chứng UVM. Thành phần này chịu trách nhiệm khởi tạo và cấu hình environment, đồng thời điều phối việc thực thi các kịch bản kiểm tra thông qua việc khởi động các sequence phù hợp. Hình 2.16 minh họa vị trí của UVM Test trong cấu trúc tổng thể của một UVM testbench.



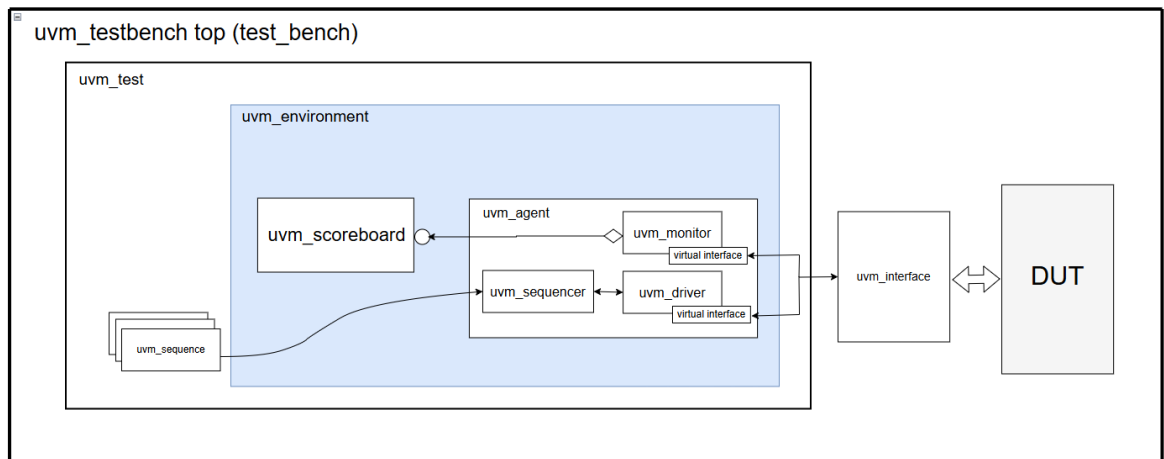
Hình 2.16: Vị trí của UVM Test trong cấu trúc UVM Testbench

Trong thực tế, UVM Test thường chứa các đối tượng cấu hình của môi trường kiểm chứng và thực hiện việc khởi động các sequence cần thiết cho từng kịch bản kiểm tra. Để tránh lặp lại mã nguồn giữa nhiều testcase khác nhau, thông thường một lớp `base_test` sẽ được xây dựng để chứa các cấu hình và thành phần dùng chung của testbench như environment, configuration object hoặc các thiết lập mặc định.

Từ lớp `base_test`, các testcase cụ thể có thể được kế thừa và mở rộng nhằm xây dựng các kịch bản kiểm tra khác nhau theo Verification Plan. Cách tổ chức này giúp nâng cao khả năng tái sử dụng, giảm khối lượng mã nguồn cần bảo trì và đơn giản hóa quá trình phát triển môi trường kiểm chứng.

2.4.2.3 UVM Environment

UVM Environment là thành phần dùng để tổ chức và quản lý các khối kiểm chứng có liên quan trong một môi trường verification. Environment thường chứa các thành phần như agent, scoreboard, coverage collector và các module hỗ trợ khác. Hình 2.17 minh họa vị trí của UVM Environment trong cấu trúc tổng thể của một UVM testbench.



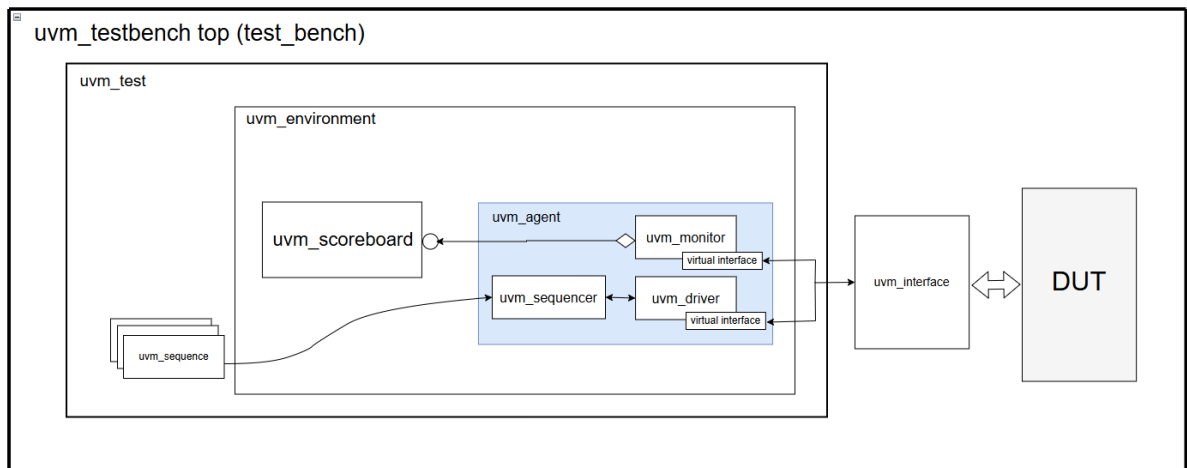
Hình 2.17. Vị trí của UVM Environment trong cấu trúc UVM Testbench

Trong môi trường UVM, Environment đóng vai trò là lớp trung gian giữa Test và các thành phần kiểm chứng ở mức thấp hơn. Thành phần này chịu trách nhiệm khởi tạo, cấu hình và kết nối các agent, scoreboard hoặc các khối chức năng khác cần thiết cho quá trình kiểm chứng. Nhờ đó, Test chỉ cần tương tác với Environment mà không phải quản lý trực tiếp từng thành phần riêng lẻ.

Đối với các hệ thống phức tạp, một Environment có thể chứa nhiều Environment con nhằm hỗ trợ việc xây dựng các môi trường kiểm chứng có tính phân cấp và khả năng tái sử dụng cao.

2.4.2.4 UVM Agent

UVM Agent là thành phần đại diện cho một giao diện cụ thể của DUT trong môi trường kiểm chứng. Agent có nhiệm vụ tạo kích thích, điều khiển tín hiệu và giám sát hoạt động của giao diện tương ứng. Hình 2.18 minh họa vị trí của UVM Agent trong cấu trúc tổng thể của một UVM testbench.



Hình 2.18. Vị trí của UVM Agent trong cấu trúc UVM Testbench

Một UVM Agent thường bao gồm ba thành phần chính:

- Sequencer: Quản lý và phân phối các transaction được sinh ra từ sequence.
- Driver: Chuyển đổi transaction thành các tín hiệu thực tế để tác động lên giao diện DUT.
- Monitor: Giám sát tín hiệu trên giao diện DUT và thu thập thông tin phục vụ cho quá trình kiểm tra.

Các thành phần bên trong Agent được kết nối với nhau thông qua cơ chế Transaction Level Modeling (TLM), cho phép trao đổi dữ liệu ở mức giao dịch thay vì mức tín hiệu. Nhờ đó, môi trường kiểm chứng có thể được xây dựng theo hướng trừu tượng hóa và dễ dàng tái sử dụng.

Tùy theo mục đích sử dụng, Agent có thể hoạt động ở hai chế độ:

- Active Agent: Bao gồm sequencer, driver và monitor, được sử dụng để sinh kích thích và điều khiển DUT.
- Passive Agent: Chỉ bao gồm monitor, được sử dụng để quan sát và thu thập dữ liệu mà không tác động trực tiếp lên DUT.

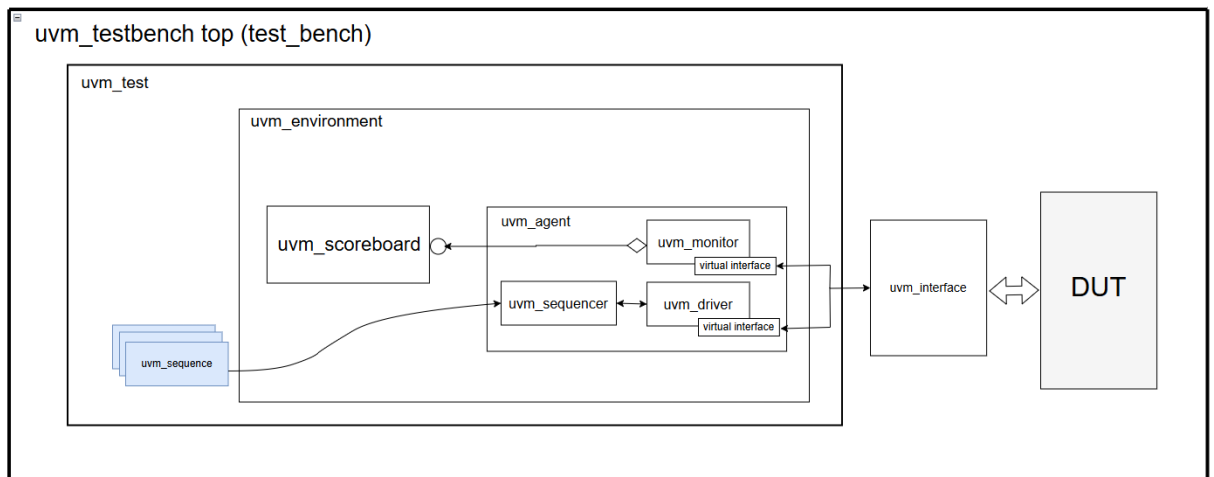
Nhờ khả năng đóng gói toàn bộ logic điều khiển và giám sát của một giao diện, Agent trở thành thành phần cốt lõi trong việc xây dựng các Verification IP (VIP) và các môi trường kiểm chứng UVM.

2.4.2.5 UVM Sequencer

UVM Sequencer đóng vai trò điều phối luồng giao dịch, quản lý và phân xử các transaction được sinh ra từ một hoặc nhiều sequence trước khi chuyển đến driver. Sequencer đảm bảo các transaction được truyền đến driver theo đúng thứ tự và không bị xung đột khi có nhiều sequence hoạt động đồng thời.

2.4.2.6 UVM Sequence

UVM Sequence là thành phần chịu trách nhiệm sinh các transaction phục vụ cho quá trình kiểm chứng. Hình 2.19 minh họa vị trí của UVM Sequence trong cấu trúc tổng thể của một UVM testbench.



Hình 2.19. Vị trí của UVM Sequence trong cấu trúc UVM Testbench

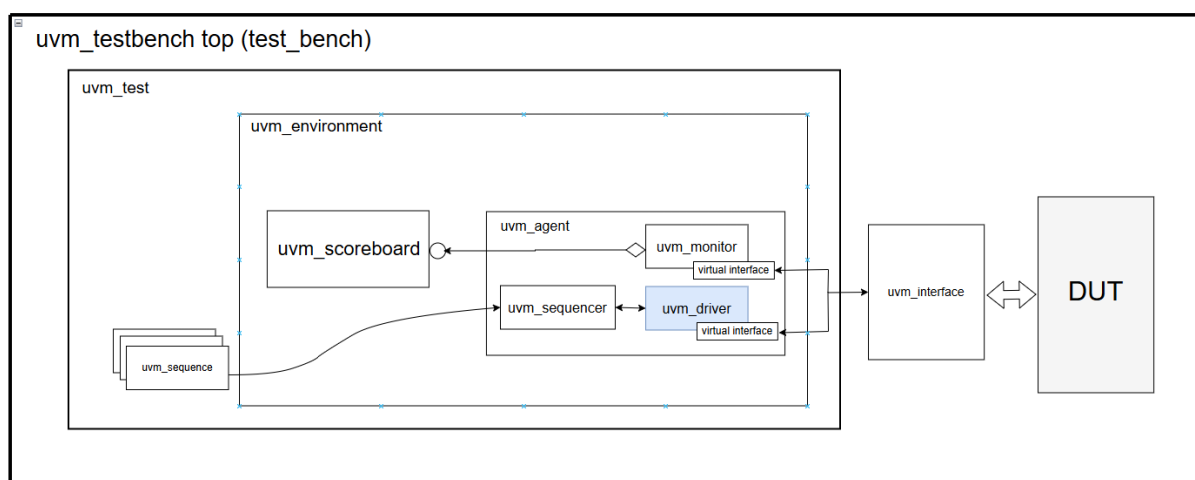
Khác với các thành phần như Test, Environment hay Agent được xây dựng từ lớp `uvm_component`, Sequence được kế thừa từ lớp `uvm_object` thông qua `uvm_sequence`. Do đó, Sequence không thuộc cấu trúc phân cấp cố định của testbench mà chỉ được tạo ra trong quá trình mô phỏng khi cần thực hiện một kịch bản kiểm tra cụ thể.

Thành phần quan trọng nhất của Sequence là hàm `body()`, nơi định nghĩa luồng sinh transaction và các thao tác kiểm tra tương ứng. Trong quá trình thực thi, Sequence tạo ra các sequence item và gửi chúng đến Sequencer để chuyển tiếp cho Driver thực hiện việc điều khiển DUT ở mức tín hiệu.

Nhờ cơ chế này, các kịch bản kiểm tra có thể được mô tả ở mức giao dịch thay vì phải thao tác trực tiếp trên các tín hiệu của DUT, giúp đơn giản hóa quá trình phát triển và nâng cao khả năng tái sử dụng của môi trường kiểm chứng.

2.4.2.7 UVM Driver

UVM Driver là thành phần chịu trách nhiệm chuyển đổi các transaction nhận được từ sequencer thành các tín hiệu thực tế để tác động lên DUT. Hình 2.20 minh họa vị trí của UVM Driver trong cấu trúc tổng thể của một UVM testbench.



Hình 2.20. Vị trí của UVM Driver trong cấu trúc UVM Testbench

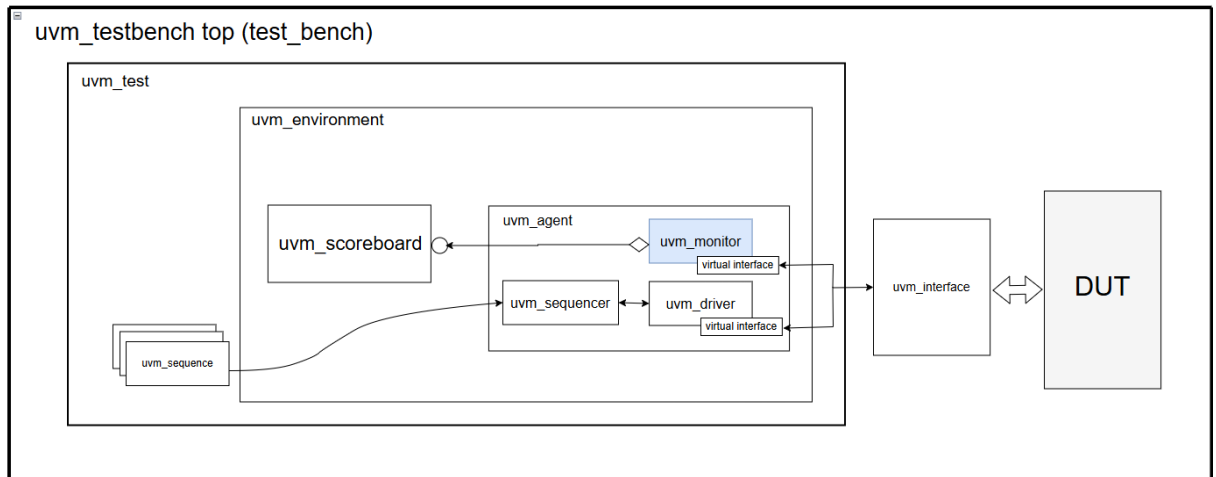
Driver nhận các transaction từ sequencer thông qua cơ chế TLM và chuyển đổi chúng thành hoạt động ở mức tín hiệu trên giao diện của DUT. Để thực hiện việc điều khiển tín hiệu, Driver thường truy cập DUT thông qua virtual interface được truyền từ môi trường kiểm chứng.

Trong quá trình mô phỏng, Driver đóng vai trò cầu nối giữa mức giao dịch và mức tín hiệu. Nhờ đó, các sequence chỉ cần tập trung mô tả kịch bản kiểm tra ở mức trừu tượng, trong khi việc điều khiển các tín hiệu cụ thể của giao thức được Driver thực hiện.

Đối với một số giao thức, Driver cũng có thể thu thập các tín hiệu phản hồi từ DUT nhằm hoàn tất quá trình thực hiện transaction trước khi tiếp tục xử lý các giao dịch tiếp theo.

2.4.2.8 UVM Monitor

UVM Monitor là thành phần có nhiệm vụ giám sát các tín hiệu trên giao diện của DUT và chuyển đổi chúng thành các transaction ở mức trừu tượng. Hình 2.21 minh họa vị trí của UVM Monitor trong cấu trúc tổng thể của một UVM testbench.



Hình 2.21. Vị trí của UVM Monitor trong cấu trúc UVM Testbench

Monitor là thành phần hoạt động hoàn toàn thụ động, không tác động trực tiếp đến DUT. Thông qua virtual interface, Monitor quan sát các tín hiệu trên giao diện DUT, thu thập thông tin giao dịch và chuyển đổi chúng thành các transaction hoặc sequence item tương ứng.

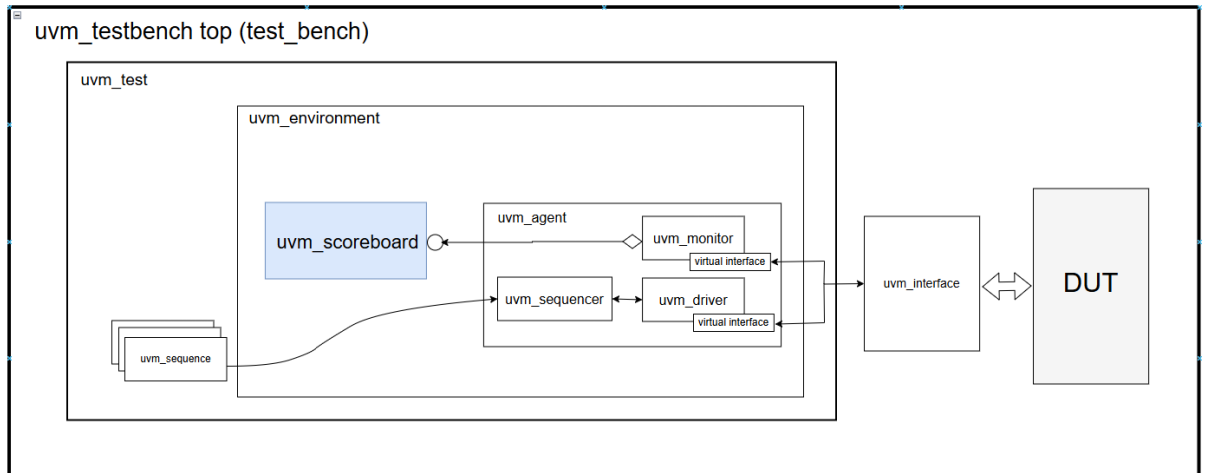
Các transaction được Monitor gửi đến các thành phần khác như Scoreboard hoặc Coverage Collector thông qua cơ chế TLM để phục vụ cho việc kiểm tra chức năng và thu thập độ bao phủ kiểm chứng.

Để đảm bảo tính tái sử dụng, Monitor thường chỉ thực hiện các kiểm tra giao thức cơ bản và chức năng thu thập dữ liệu. Các kiểm tra chức năng ở mức hệ thống hoặc so sánh kết quả kỳ vọng thường được thực hiện bởi Scoreboard.

2.4.2.9 UVM Scoreboard

UVM Scoreboard là thành phần chịu trách nhiệm kiểm tra tính đúng đắn của DUT bằng cách so sánh dữ liệu thực tế thu được từ DUT với dữ liệu kỳ vọng.

Hình 2.22 minh họa vị trí của UVM Scoreboard trong cấu trúc tổng thể của một UVM testbench.



Hình 2.22. Vị trí của UVM Scoreboard trong cấu trúc UVM Testbench

Scoreboard thường nhận các transaction từ Monitor thông qua analysis port. Dựa trên các transaction đầu vào, Scoreboard có thể sử dụng mô hình tham chiếu hoặc cơ chế dự đoán để tạo ra kết quả kỳ vọng tương ứng.

Sau đó, dữ liệu thực tế thu được từ DUT sẽ được so sánh với dữ liệu kỳ vọng. Nếu hai kết quả khớp nhau, giao dịch được xem là hợp lệ; ngược lại, Scoreboard sẽ ghi nhận lỗi và cung cấp thông tin phục vụ cho quá trình debug.

Nhờ khả năng tự động kiểm tra kết quả mô phỏng, Scoreboard là một trong những thành phần quan trọng nhất của môi trường kiểm chứng UVM, góp phần nâng cao mức độ tự động hóa và độ tin cậy của quá trình verification.

2.4.3 Verification IP (VIP)

Trong lĩnh vực thiết kế vi mạch, Intellectual Property (IP) là một khối chức năng phần cứng được thiết kế để thực hiện một chức năng xác định và có thể tái sử dụng trong nhiều dự án khác nhau [17]. Trong khi đó, VIP là tập hợp các thành phần kiểm chứng được xây dựng nhằm mô phỏng, điều khiển, giám sát và kiểm tra hoạt động của một giao thức hoặc giao diện cụ thể [16].

VIP được sử dụng trong testbench để mô phỏng hành vi của các thành phần ngoại vi mà thiết kế đang được kiểm thử DUT tương tác thông qua các giao thức

phổ biến như AXI, APB, AHB, PCIe, Ethernet, DDR, SPI, I2C và nhiều chuẩn giao tiếp khác. Việc sử dụng VIP giúp giảm đáng kể thời gian xây dựng môi trường kiểm chứng và nâng cao khả năng tái sử dụng giữa các dự án [16].

Trong môi trường UVM, một VIP thường được xây dựng từ các thành phần như Agent, Sequencer, Driver, Monitor, Interface, Transaction và các cơ chế kiểm tra giao thức. Nhờ khả năng đóng gói các chức năng kiểm chứng thành một khối độc lập và có khả năng tái sử dụng cao, VIP hiện được sử dụng rộng rãi trong quy trình Design Verification của ngành công nghiệp bán dẫn [16].

2.4.4 Cơ chế UVM

2.4.4.1 Cơ chế Pha

Để đảm bảo luồng thực thi của testbench được tổ chức chặt chẽ và nhất quán, UVM sử dụng cơ chế pha nhằm điều phối và sắp xếp các hoạt động trong suốt quá trình mô phỏng. Các pha được chia thành ba nhóm chính.

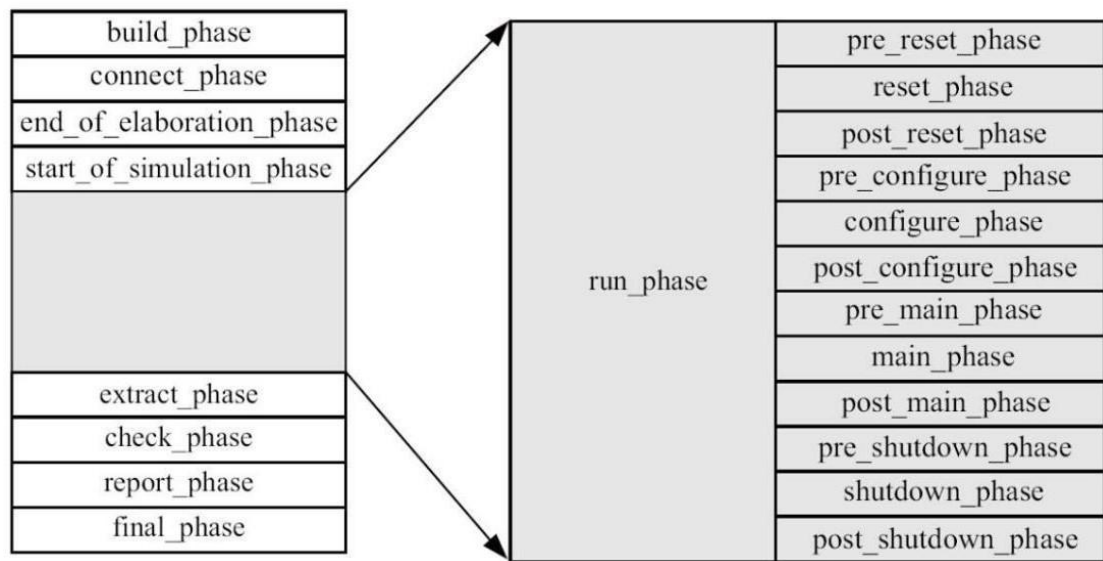
- Pha khởi tạo: Bao gồm các bước từ `build_phase` đến `start_of_simulation_phase`. Đây là các function phase không tiêu tốn thời gian mô phỏng, chủ yếu phục vụ việc xây dựng cấu trúc, cấu hình tham số và kết nối các thành phần trong UVM.

- Pha thực thi: Là giai đoạn tiêu tốn thời gian mô phỏng thực sự, trong đó các kịch bản kiểm tra được thực thi. Hình 2.23 minh họa các giai đoạn của pha thực thi. Có hai cách tổ chức: theo chuỗi từ `pre_reset_phase` đến `post_shutdown_phase`, hoặc thông qua `run_phase` — hai cách có ý nghĩa tương đương và không nên dùng đồng thời.

- Pha kết thúc: Bao gồm các bước từ `extract_phase` đến `final_phase`, phục vụ thu thập kết quả, tổng hợp dữ liệu và xuất báo cáo kiểm tra.

Lớp cơ sở `uvm_component` cung cấp các phương thức ảo tương ứng với từng pha, cho phép người dùng định nghĩa hành vi của mỗi thành phần theo đúng vai trò của chúng. Tuân thủ cơ chế phasing giúp các thành phần kiểm chứng được

phát triển độc lập nhưng vẫn phối hợp chính xác với nhau trong quá trình mô phỏng.



Hình 2.23. Cơ chế pha trong UVM

2.4.4.2 Cơ chế Factory

Cơ chế Factory được sử dụng để tạo và quản lý các đối tượng trong môi trường kiểm tra một cách linh hoạt thông qua một cơ chế trung gian, thay vì khởi tạo trực tiếp theo lớp cụ thể. Cách tiếp cận này tách biệt cấu trúc testbench khỏi kiểu đối tượng được sử dụng, cho phép thay thế hoặc mở rộng chức năng các thành phần mà không cần sửa đổi mã nguồn gốc.

Để sử dụng Factory, các lớp phải được đăng ký thông qua các macro chuẩn: `uvm_component_utils` đối với `component` và `uvm_object_utils` đối với `object`. Nhờ cơ chế này, Factory góp phần nâng cao tính tái sử dụng, khả năng mở rộng và đơn giản hóa quá trình phát triển môi trường kiểm chứng, đặc biệt với các thiết kế quy mô lớn.

2.4.4.3 Cơ chế Config_db

Cơ chế cơ sở dữ liệu cấu hình (`config_db`) được sử dụng để truyền các tham số cấu hình giữa các thành phần trong môi trường kiểm tra một cách linh hoạt và có hệ thống. Cơ chế này cho phép thiết lập giá trị cấu hình từ mức cao như test

hoặc environment, sau đó tự động phân phối xuống các thành phần con như agent, driver hoặc monitor mà không cần kết nối trực tiếp giữa các đối tượng.

Config_db cung cấp hai phương thức chính: set() để ghi giá trị cấu hình vào database và get() để các thành phần con lấy giá trị tương ứng. Cơ chế này giúp tách biệt cấu hình khỏi cấu trúc testbench, đơn giản hóa việc tùy chỉnh hành vi từng thành phần theo từng kịch bản kiểm tra mà không cần thay đổi mã nguồn.

CHƯƠNG 3: PHÂN TÍCH THIẾT KẾ VÀ XÂY DỰNG MÔI TRƯỜNG KIỂM CHỨNG UVM

3.1 YÊU CẦU THIẾT KẾ

Trong đề tài này, phần RTL của AXI-to-APB Bridge được kế thừa từ dự án mã nguồn mở *Design and Verify AXI4-to-APB4 Bridge using UVM* [13]. Trên cơ sở thiết kế gốc, luận văn tập trung mở rộng kiến trúc bridge bằng cách bổ sung khối APB Register cho phép cấu hình động bản đồ địa chỉ của các APB Slave trong thời gian chạy (runtime address remapping), đồng thời bổ sung cơ chế xử lý Decode Error và hệ thống ngắt tương ứng. Các chức năng mở rộng này làm tăng tính linh hoạt của bridge nhưng cũng đặt ra yêu cầu kiểm chứng phức tạp hơn so với thiết kế ban đầu. Mục tiêu của đề tài được xác định như sau:

- Kế thừa và mở rộng thiết kế cầu nối AXI-APB theo đúng đặc tả, hỗ trợ các kiểu burst AXI, cơ chế cấu hình động bản đồ địa chỉ, xử lý lỗi và ngắt tương ứng.
- Xây dựng môi trường kiểm chứng dựa trên phương pháp UVM nhằm kiểm tra đầy đủ các chức năng của thiết kế.
- Đảm bảo toàn bộ testcase trong Verification Plan thực thi thành công (100% PASS) sau khi hoàn tất quá trình debug.
- Đạt 100% Functional Coverage trên các covergroup đã định nghĩa và đạt trên 90% Code Coverage.

3.2 ĐẶC TẢ VÀ NGUYÊN LÝ HOẠT ĐỘNG

3.2.1 Tổng quan kiến trúc

Runtime-Configurable AXI-to-APB Bridge là cầu nối giữa bus AXI4 hiệu năng cao và bus APB4 dành cho các ngoại vi tốc độ thấp. Bridge đóng vai trò AXI Slave và APB Master, thực hiện chuyển đổi các giao dịch AXI sang giao dịch APB tương ứng.

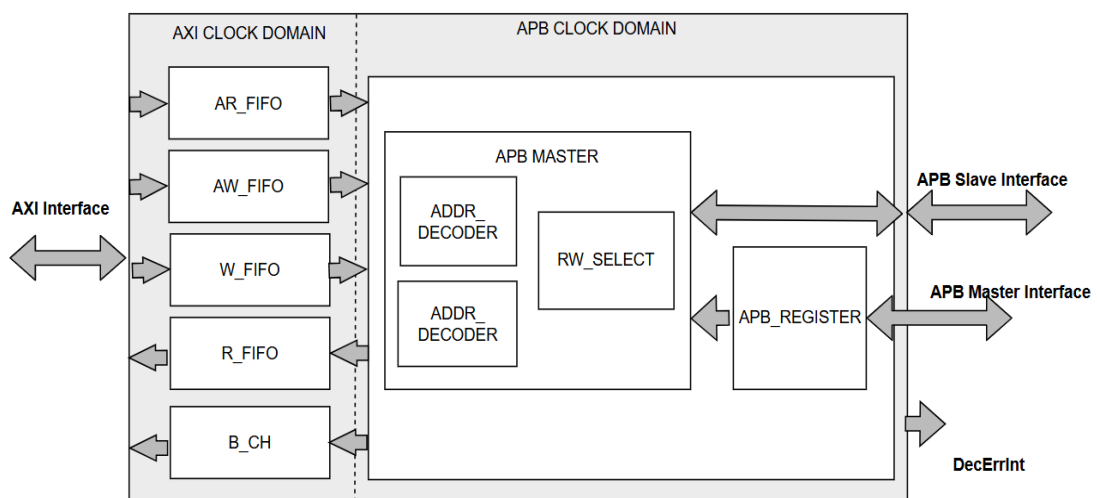
Các đặc tính chính của Bridge bao gồm:

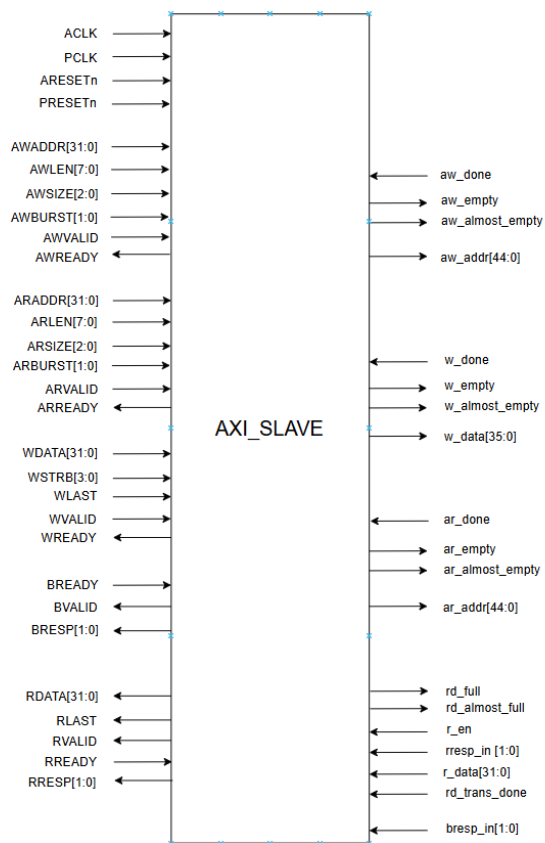
- Hỗ trợ chuẩn AMBA AXI4 và APB4.
- Tuân thủ đặc tả giao thức AMBA AXI.
- Hỗ trợ tối đa 3 APB Slave.
- Hỗ trợ độ rộng dữ liệu AXI cấu hình được (8/16/32-bit).
- Hỗ trợ Runtime Address Remapping thông qua các thanh ghi cấu hình khi APB MASTER IDLE.
- Không hỗ trợ các cấu hình vùng địa chỉ chồng lấn giữa nhiều APB Slave
- Hỗ trợ phát hiện lỗi APB Slave Error (PSLVERR).
- Hỗ trợ phát hiện lỗi giải mã địa chỉ (Decode Error).
- Hỗ trợ cơ chế sinh ngắt khi xảy ra Decode Error.
- Hỗ trợ đồng bộ dữ liệu giữa hai miền xung nhịp thông qua Asynchronous FIFO.

So với AXI-to-APB Bridge thông thường, điểm nổi bật của thiết kế là khả năng Runtime Address Remapping. Thông qua các thanh ghi cấu hình BAMS, phần mềm có thể thay đổi bản đồ địa chỉ của các APB Slave trong thời gian chạy mà không cần sửa đổi hoặc tổng hợp lại phần cứng.

3.2.2 Sơ đồ khối

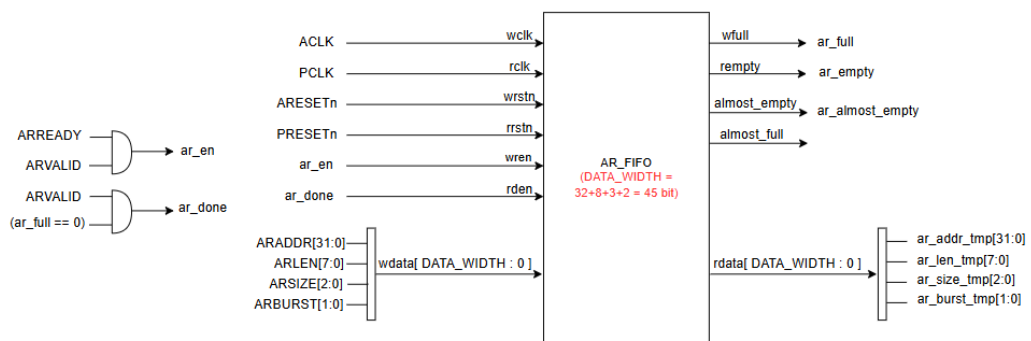
Kiến trúc tổng thể của Runtime-Configurable AXI-to-APB Bridge được minh họa trong Hình 3.1.





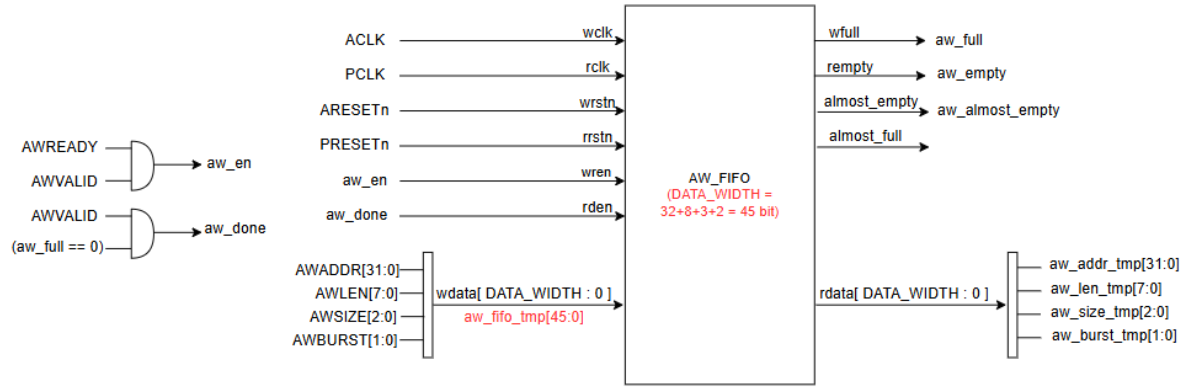
Hình 3.2: Sơ đồ khối AXI Clock Domain

AR FIFO lưu trữ thông tin địa chỉ của các giao dịch đọc từ kênh AR trước khi chuyển sang miền PCLK.



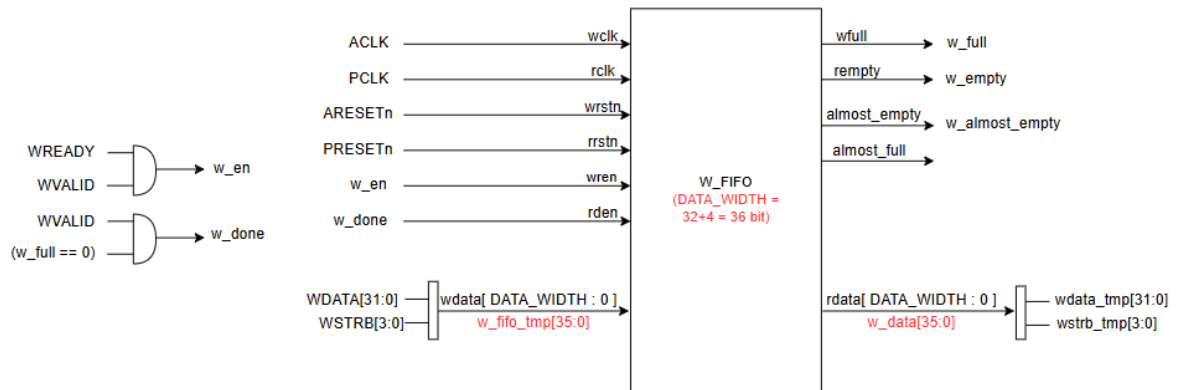
Hình 3.3: Sơ đồ khối Read Address FIFO (AR FIFO)

AW FIFO lưu trữ thông tin địa chỉ của các giao dịch ghi từ kênh AW trước khi được xử lý bởi APB Clock Domain..



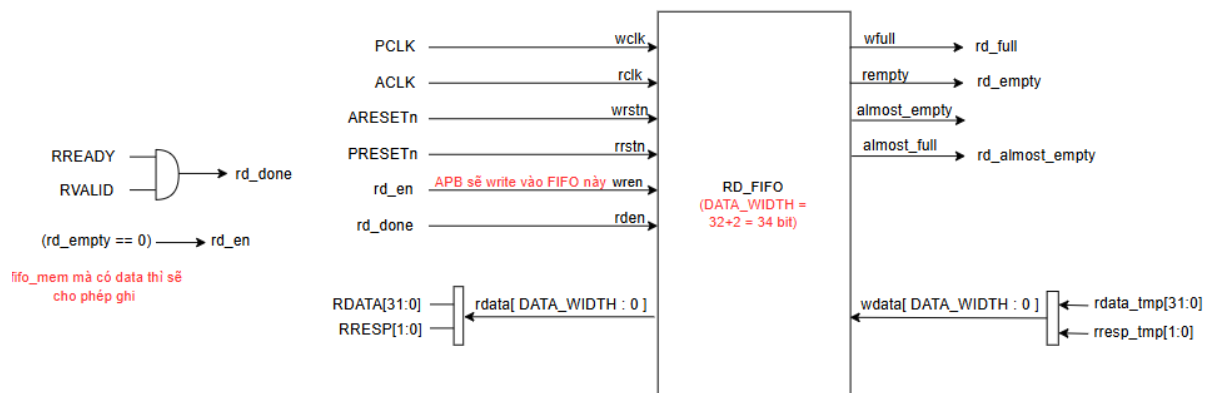
Hình 3.4: Sơ đồ khối Write Address FIFO (AW FIFO)

W FIFO lưu trữ dữ liệu ghi và byte enable của các giao dịch AXI Write.



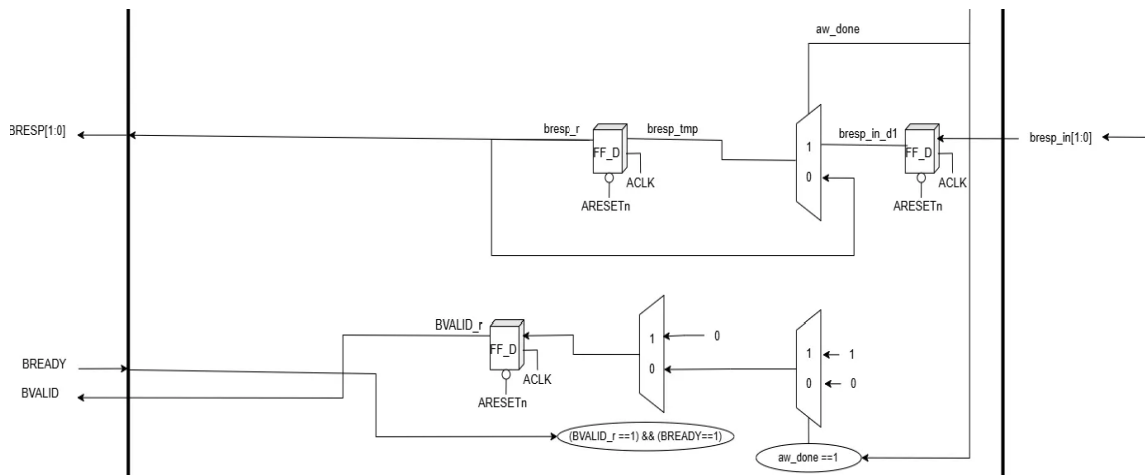
Hình 3.5: Sơ đồ khối Write Data FIFO (W FIFO)

RD FIFO lưu trữ dữ liệu đọc trả về từ APB trước khi chuyển sang giao diện AXI Read Data.



Hình 3.6: Sơ đồ khối Read Data FIFO (RD FIFO)

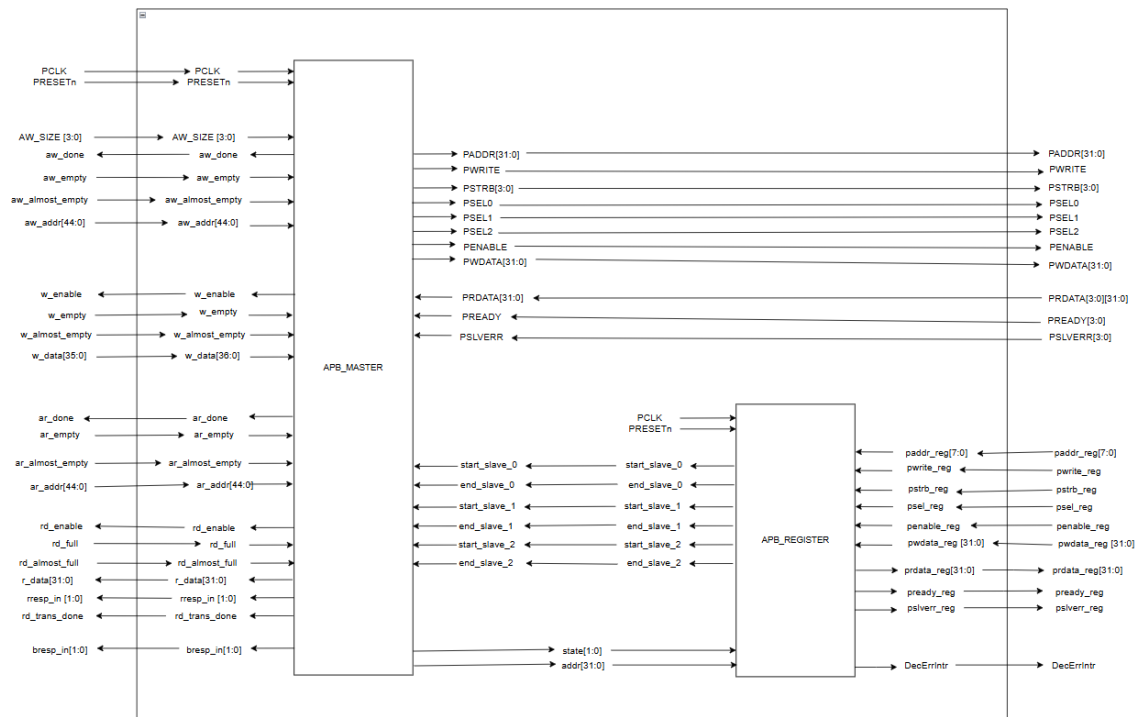
Buffer Response lưu trữ và xử lý phản hồi ghi từ phía APB, sau đó chuyển đổi thành phản hồi BRESP tương ứng cho giao diện AXI.



Hình 3.7: Sơ đồ khối Buffer Response

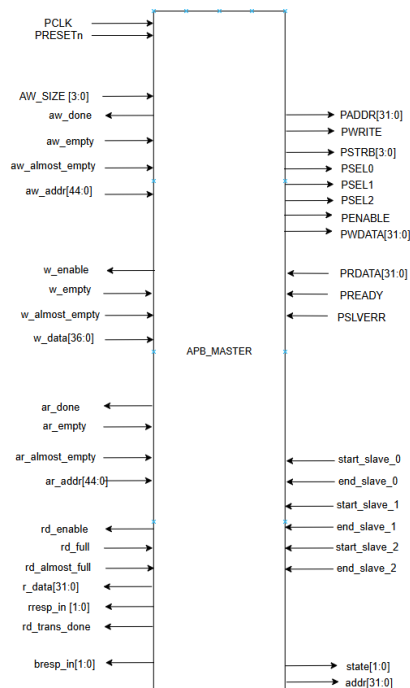
3.2.2.2 APB Clock Domain

APB Clock Domain chịu trách nhiệm nhận các giao dịch từ AXI Clock Domain, chuyển đổi chúng thành các giao dịch APB4 tương ứng và điều khiển quá trình truy cập đến các APB Slave. Đây là miền xử lý chính của bridge, nơi thực hiện phân xử yêu cầu đọc/ghi, giải mã địa chỉ, phát sinh tín hiệu điều khiển APB và quản lý các thanh ghi cấu hình nội bộ.



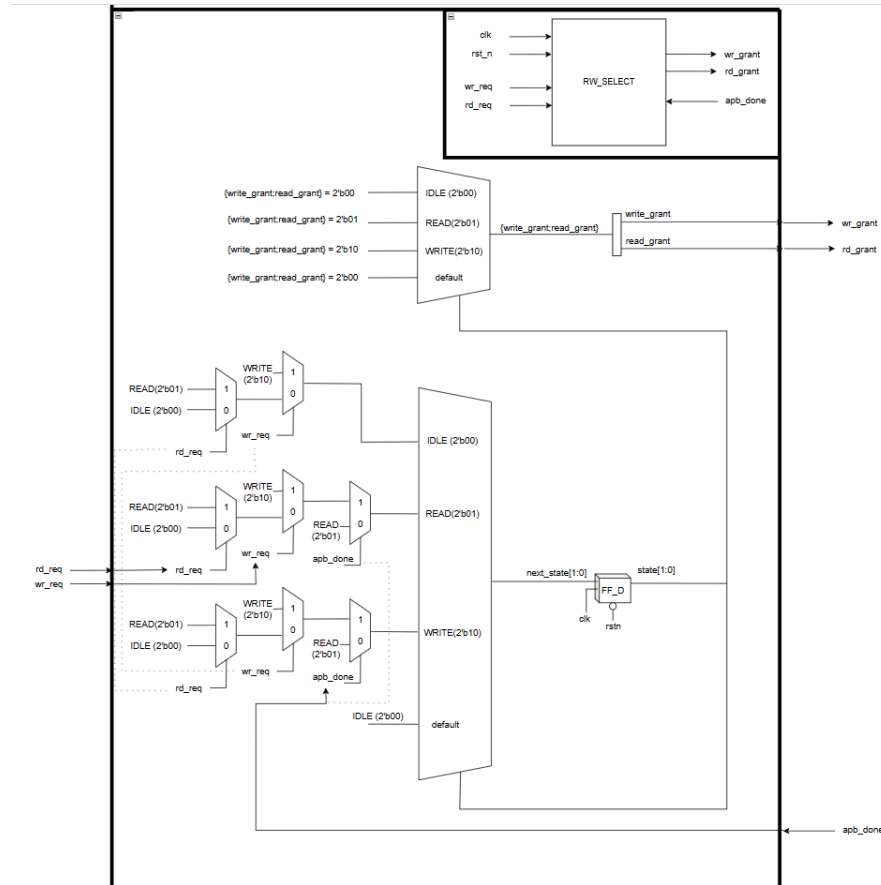
Hình 3.8: Sơ đồ khối APB Clock Domain

APB_MASTER là khối trung tâm của APB Clock Domain, chịu trách nhiệm phát sinh các giao dịch APB4, điều khiển các tín hiệu PADDR, PWRITE, PWDATA, PENABLE và thu thập phản hồi từ phía APB Slave.



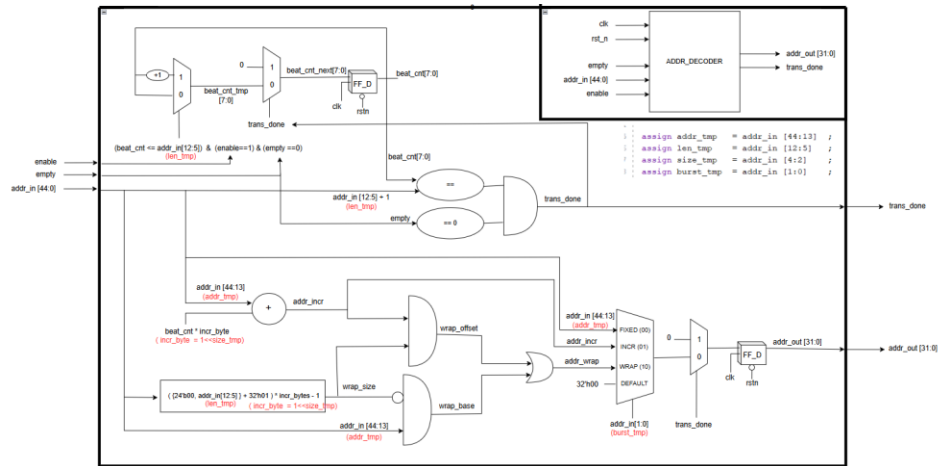
Hình 3.9: Sơ đồ khối APB_MASTER

RW_SELECT là khối phân xử giữa các yêu cầu đọc và ghi đến từ AXI Clock Domain. Khối này quyết định loại giao dịch được xử lý tại từng thời điểm nhằm bảo đảm việc sử dụng bus APB một cách tuần tự và nhất quán.



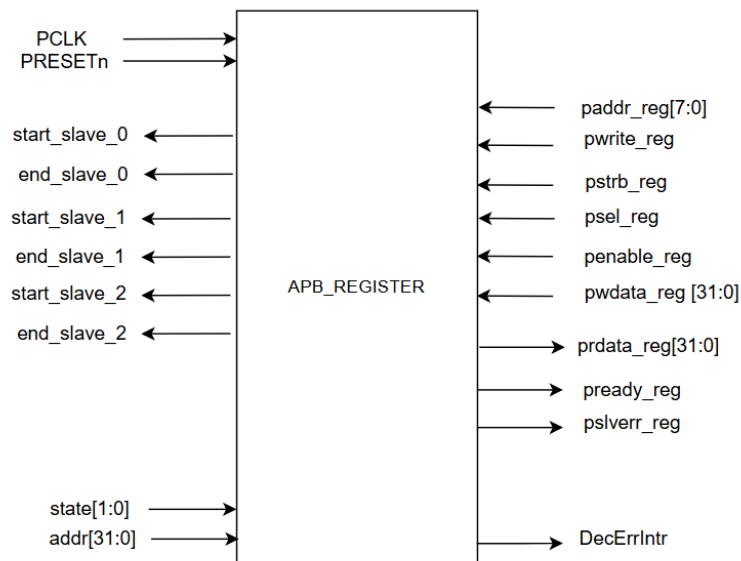
Hình 3.10: Sơ đồ trạng thái FSM của khối RW_SELECT

ADDR_DECODER thực hiện tính toán địa chỉ cho từng beat trong burst AXI và xác định APB Slave tương ứng dựa trên cấu hình địa chỉ hiện tại. Khối này đồng thời phát hiện các trường hợp địa chỉ không hợp lệ hoặc chồng lấn vùng địa chỉ.



Hình 3.11: Nguyên lý tính địa chỉ của ADDR_DECODER theo loại burst

APB_REGISTER quản lý tập thanh ghi cấu hình của bridge, bao gồm các thanh ghi BAMS dùng cho Runtime Address Remapping và thanh ghi BIR dùng cho cơ chế xử lý lỗi và ngắt. Thông qua các thanh ghi này, phần mềm có thể thay đổi bản đồ địa chỉ của các APB Slave trong thời gian chạy mà không cần sửa đổi phần cứng.



Hình 3.12: Sơ đồ khối APB_REGISTER

3.2.3 Mô tả các thanh ghi cấu hình

Bảng 3.1: Bản đồ thanh ghi nội bộ của APB_REGISTER

Địa chỉ	Tên thanh ghi	Viết tắt	Mô tả
0x00	Base Address & Memory Size 0	BAMS0	Cấu hình địa chỉ Slave 0
0x04	Base Address & Memory Size 1	BAMS1	Cấu hình địa chỉ Slave 1
0x08	Base Address & Memory Size 2	BAMS2	Cấu hình địa chỉ Slave 2
0x0C	Bridge Interrupt Register	BIR	Interrupt control

Để hỗ trợ chức năng Runtime Address Remapping, bridge sử dụng các thanh ghi BAMS để xác định vùng địa chỉ của từng APB Slave. Mỗi thanh ghi BAMS lưu hai thông tin chính gồm địa chỉ bắt đầu (Base Address) và kích thước vùng nhớ (Memory Size). Bộ giải mã địa chỉ (Address Decoder) sử dụng các thông tin này để xác định APB Slave tương ứng cho mỗi giao dịch. Trong thiết kế này, trường SIZE được mã hóa bằng 2 bit như trình bày trong Bảng 3.1.

Bảng 3.2: Mã hóa kích thước vùng địa chỉ APB Slave

Size	Kích thước
2'b00	1KB
2'b01	2 KB
2'b10	4 KB
2'b11	8 KB

Ví dụ, giả sử APB Slave 0 cần được ánh xạ vào vùng địa chỉ từ 0x9800_4C00 đến 0x9800_4FFF. Khi đó, kích thước vùng địa chỉ tương ứng là:

$$0x9800_4FFF - 0x9800_4C00 + 1 = 0x400 \text{ Byte} = 1 \text{ KB.}$$

Tra theo Bảng 3.1, kích thước 1 KB tương ứng với giá trị SIZE = 2'b00. Do kích thước nhỏ nhất được hỗ trợ là 1 KB (2^{10} Byte), 10 bit địa chỉ thấp của Base Address không cần lưu trong thanh ghi BAMS. Vì vậy, thanh ghi BAMS chỉ lưu 22 bit địa chỉ cao nhất, tương ứng với các bit [31:10] của Base Address 0x9800_4C00.

Đối với địa chỉ cơ sở 0x9800_4C00, 10 bit địa chỉ thấp được loại bỏ và 22 bit địa chỉ cao còn lại được lưu vào trường Base[31:10] của thanh ghi BAMS0. Theo Hình 3.13, giá trị này được biểu diễn dưới dạng:

$$\text{Base}[31:10] = \{12'h980, 10'h013\}$$

	[31:28]	27:24	[23:20]	[19:16]	[15:12]	[11:8]	[7:4]	[3:0]
S0 address[31:0]	9	8	0	0	4	c	0	0
	[31:28]	[27:24]	[23:20]		[19:18]	[17:14]	[13:10]	
Base[31:10]	9	8	0		0	1	3	

Hình 3.13: Ví dụ cấu hình Base Address cho APB Slave 0

Khi bộ giải mã địa chỉ sử dụng giá trị Base Address và Size đã được cấu hình trong BAMS0, vùng địa chỉ của APB Slave 0 sẽ được xác định từ 0x9800_4C00 đến 0x9800_4FFF. Do đó, mọi giao dịch có địa chỉ nằm trong khoảng này sẽ được định tuyến tới APB Slave 0, trong khi các địa chỉ nằm ngoài vùng trên sẽ không kích hoạt tín hiệu chọn của Slave 0.

Thông tin Base Address và Memory Size của các APB Slave được lưu trữ trong các thanh ghi BAMS0, BAMS1 và BAMS2, trong khi các thông tin điều khiển và trạng thái ngắt, lỗi được quản lý bởi thanh ghi BIR (Bridge Interrupt Register).

Bảng 3.3: Cấu trúc thanh ghi BAMS0

Bit	Tên	Kiểu truy cập	Giá trị mặc định	Mô tả
[31:10]	Base	RW	{12'h000, 10'h000}	Địa chỉ cơ sở
[9:2]	-	Reserved	0	Các bit dự trữ, không sử dụng
[1:0]	Size	RW	2'b10	Kích thước của không gian địa chỉ.

Bảng 3.4: Cấu trúc thanh ghi BAMS1

Bit	Tên	Kiểu truy cập	Giá trị mặc định	Mô tả
[31:10]	Base	RW	{12'h000, 10'h004}	Địa chỉ cơ sở
[9:2]	-	Reserved	0	Các bit dự trữ, không sử dụng
[1:0]	Size	RW	2'b10	Kích thước của không gian địa chỉ.

Bảng 3.5: Cấu trúc thanh ghi BAMS2

Bit	Tên	Kiểu truy cập	Giá trị mặc định	Mô tả
[31:10]	Base	RW	{12'h000,10'h008}	Địa chỉ cơ sở
[9:2]	-	Reserved	0	Các bit dự trữ, không sử dụng
[1:0]	Size	RW	2'b10	Kích thước của không gian địa chỉ.

Bảng 3.6: Cấu trúc thanh ghi BIR

Bit	Tên	Kiểu truy cập	Giá trị mặc định	Mô tả
[31:2]	-	Rsvd	0	Các bit dự trữ, không sử dụng
1	DecErrSt	RW1C	0	Trạng thái ngắt (Interrupt Status). Bit này được set khi bridge nhận địa chỉ không tồn tại (non-existing address) 0: Không xảy ra lỗi decode address 1: Ghi 1 để xóa status (Write-1-to-Clear).
0	DecErrEn	RW	1	Cho phép hoặc vô hiệu hóa tín hiệu ngắt DecErrIntr. 0: Vô hiệu hóa interrupt. 1: Cho phép interrupt.

3.2.4 Phân tích các nội dung cần kiểm chứng

Từ đặc tả và nguyên lý hoạt động của Runtime-Configurable AXI-to-APB Bridge, có thể xác định các nhóm chức năng trọng tâm cần được kiểm chứng như sau:

- Cấu hình và truy cập các thanh ghi nội bộ.
- Chuyển đổi giao dịch AXI Write sang APB.
- Chuyển đổi giao dịch AXI Read sang APB.
- Giải mã địa chỉ và lựa chọn APB Slave tương ứng.
- Khả năng hoạt động dưới các kích bản ngẫu nhiên và điều kiện biên.

- Xử lý lỗi PSLVERR và Decode Error.
- Cơ chế phát sinh và xóa ngắt.
- Cơ chế Runtime Address Remapping.

3.3 THIẾT KẾ MÔI TRƯỜNG KIỂM TRA UVM

3.3.1 Luồng kiểm chứng tổng thể

Dựa trên quy trình Design Verification trong công nghiệp bán dẫn [14], quá trình kiểm chứng thường được tổ chức thành sáu giai đoạn chính như trình bày trong Bảng 3.7.

Bảng 3.7: Các giai đoạn trong quy trình Design Verification

Giai đoạn	Tên giai đoạn	Nội dung kiểm chứng chính
1	Phân tích yêu cầu thiết kế	Nghiên cứu các tài liệu đặc tả và yêu cầu chức năng nhằm hiểu rõ hành vi mong muốn của thiết kế và xác định các nội dung cần kiểm chứng.
2	Xây dựng kế hoạch kiểm chứng Verification Plan	Xác định phạm vi kiểm chứng, các hạng mục cần kiểm tra, phương pháp kiểm chứng, tiêu chí đánh giá
3	Thiết kế cấu trúc dự án và kiến trúc Testbench	Thiết kế cấu trúc thư mục dự án, xác định kiến trúc tổng thể của UVM Testbench và mối quan hệ giữa các thành phần trong môi trường kiểm chứng.
4	Xây dựng môi trường kiểm chứng	Hiện thực các thành phần kiểm chứng dựa trên kiến trúc đã được xác định ở giai đoạn trước.
5	Phát triển Testcase	Xây dựng các testcase nhằm tạo stimulus, kiểm tra hành vi của thiết kế và đánh giá kết quả thông qua các cơ chế kiểm tra phù hợp.
6	Chạy mô phỏng, Debug và sửa lỗi	Thực hiện mô phỏng, phân tích kết quả kiểm chứng, xác định nguyên nhân gây lỗi và tiến hành khắc phục các vấn đề phát hiện được.
7	Xây dựng và đánh giá Coverage	Thu thập số liệu coverage, xác định các nội dung chưa được kiểm tra đầy đủ và bổ sung hoạt động kiểm chứng tương ứng.

3.3.2 Xây dựng kế hoạch kiểm chứng Verification Plan

Dựa trên các nội dung cần kiểm chứng đã được xác định trong Mục 3.2.4, các verification group được xác định và tổ chức thành Verification Plan. Mỗi verification group đại diện cho một nhóm chức năng của DUT và được gắn với phương pháp kiểm chứng cũng như tiêu chí đánh giá tương ứng. Verification Plan của Runtime-Configurable AXI-to-APB Bridge được trình bày trong Bảng 3.8.

Bảng 3.8: Verification Plan

STT	Nhóm tính năng	Nội dung kiểm chứng chính	Phương pháp	Tiêu chí đánh giá
1	APB Register Configuration	Cấu hình và truy cập các thanh ghi nội bộ: • Đọc/ghi BAMS và BIR • Kiểm tra vùng reserved, byte access và giá trị mặc định	RAL + Directed testcase	Hành vi thanh ghi đúng specification
2	AXI Write Transaction	• Chuyển đổi giao dịch AXI Write sang APB • Giải mã địa chỉ và lựa chọn APB Slave • Hỗ trợ burst FIXED/INCR/WRAP • Xử lý lỗi PSLVERR	Directed + Constrained-random	Write path hoạt động đúng
3	AXI Read Transaction	• Chuyển đổi giao dịch AXI Read sang APB • Giải mã địa chỉ và lựa chọn APB Slave • Hỗ trợ burst FIXED/INCR/WRAP • Xử lý lỗi PSLVERR	Directed + Constrained-random	Read path hoạt động đúng
4	AXI Random R/W Transaction	• Khả năng hoạt động dưới các kích bản ngẫu nhiên và điều kiện biên	Constrained-random	DUT ổn định dưới random traffic
5	Interrupt Mechanism	• Cơ chế phát sinh và xóa ngắt • Kích hoạt/vô hiệu hóa interrupt • Xử lý trạng thái ngắt dạng sticky và cơ chế WIC	RAL + Directed + Checker	Logic interrupt đúng specification

		• Kiểm tra interrupt liên quan đến Decode Error		
6	Dynamic Address Remap	• Cơ chế Runtime Address Remapping • Thay đổi ánh xạ địa chỉ trong thời gian chạy • Kiểm tra vùng địa chỉ cũ không còn hợp lệ sau remap • Kiểm tra truy cập sau remap	RAL + Directed + Stress testcase	Address decode và remap đúng

3.3.3 Thiết kế cấu trúc dự án và kiến trúc Testbench

Dựa trên Verification Plan đã xây dựng, môi trường kiểm chứng được tổ chức theo cấu trúc phân cấp thư mục nhằm hỗ trợ việc quản lý mã nguồn, phát triển testbench và tái sử dụng các thành phần verification được minh họa trong Hình3.14.

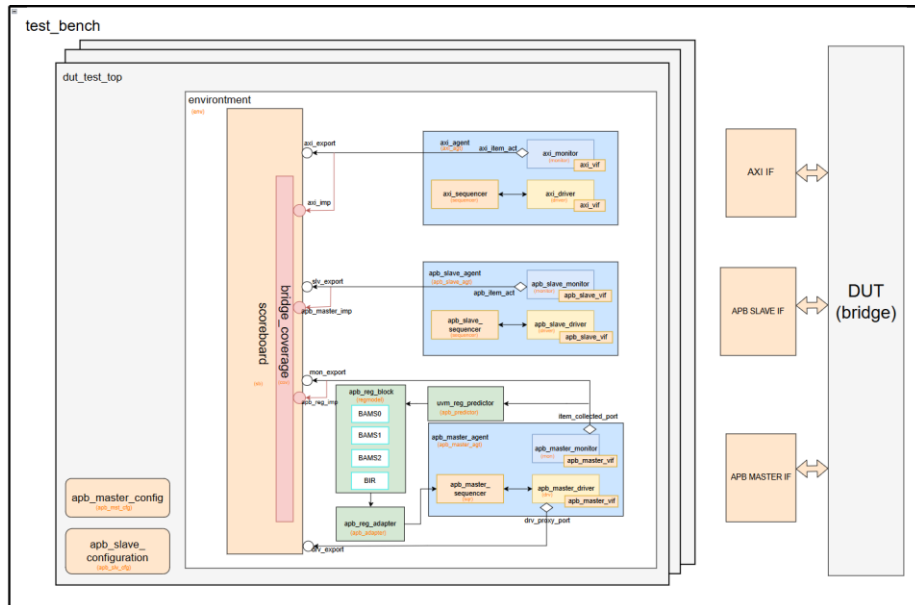


Hình 3.14: Cấu trúc phân cấp thư mục và các thành phần chính của môi trường verification

Thư mục RTL chứa mã nguồn của DUT và các module liên quan. Thư mục VIP chứa các Verification IP phục vụ việc mô phỏng và giám sát các giao diện của DUT. Thư mục Testcase chứa các lớp kiểm thử kế thừa từ base test, trong khi thư mục Script chứa các tệp hỗ trợ biên dịch, mô phỏng và thu thập coverage.

Sau khi xác định cấu trúc dự án, kiến trúc tổng thể của UVM Testbench được xây dựng nhằm mô tả các thành phần chính của môi trường kiểm chứng, mỗi

quan hệ giữa các thành phần và luồng trao đổi dữ liệu trong hệ thống. Kiến trúc tổng thể của UVM Testbench được trình bày trong Hình 3.15.



Hình 3.15: Kiến trúc tổng thể UVM Testbench của AXI-to-APB Bridge

3.3.4 Xây dựng các thành phần của UVM Testbench

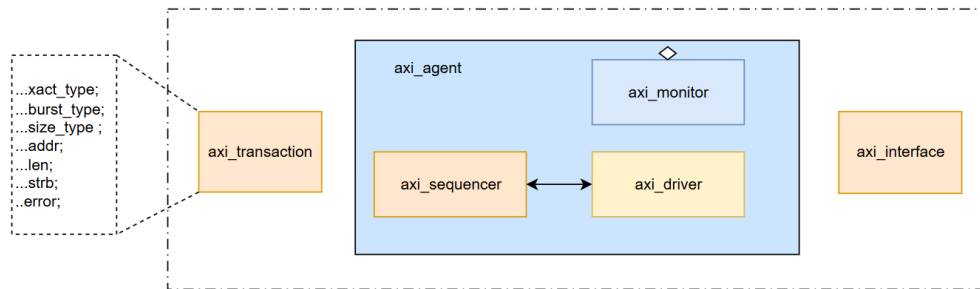
3.3.4.1 Xây dựng Verification IP (VIP)

Verification IP (VIP) là các thành phần kiểm chứng được xây dựng để mô phỏng, giám sát và phân tích hoạt động của một giao thức cụ thể, chẳng hạn như UART, SPI, I2C, AXI, APB, ... Trong đề tài này, ba VIP được phát triển tương ứng với ba giao diện của DUT gồm AXI VIP, APB Slave VIP và APB Master VIP. Mỗi VIP được xây dựng độc lập theo kiến trúc chuẩn của UVM với các thành phần transaction, sequence, sequencer, driver, monitor và agent. Việc tách biệt các VIP giúp môi trường kiểm chứng có tính module cao, dễ bảo trì và thuận lợi cho việc tái sử dụng trong các dự án khác.

AXI VIP — tạo và giám sát giao dịch AXI

AXI VIP mô phỏng AXI Master kết nối với giao diện AXI của DUT. Điểm quan trọng trong thiết kế AXI VIP là Monitor sử dụng kiến trúc đa luồng với năm task song song tương ứng với năm kênh AXI (AW, W, B, AR, R), phản ánh đúng tính bất đồng bộ của giao thức AXI. Các kênh được kết nối với nhau thông

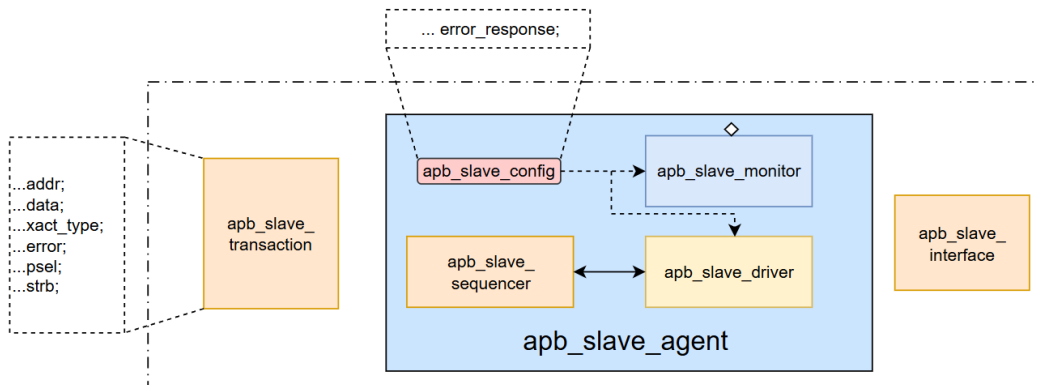
qua mailbox nội bộ để đảm bảo transaction được lắp ráp đúng thứ tự. Kiến trúc AXI VIP được minh họa tại Hình 3.16.



Hình 3.16: Kiến trúc AXI VIP

APB Slave VIP — mô phỏng phản hồi ngoại vi

APB Slave VIP mô phỏng các APB Slave phản hồi giao dịch do bridge phát sinh. VIP này có vai trò quan trọng trong việc kiểm tra khả năng xử lý lỗi: Driver có thể được cấu hình để luôn trả về PSLVERR=1, cho phép testcase xác minh rằng bridge chuyển đổi đúng lỗi APB thành BRESP/RRESP tương ứng trên kênh AXI. Ngoài ra, Driver còn ngẫu nhiên hóa số wait state từ 1 đến 3 chu kỳ để kiểm tra tính ổn định của bridge dưới điều kiện slave có độ trễ biến đổi. Kiến trúc APB Slave VIP được trình bày tại Hình 3.17.

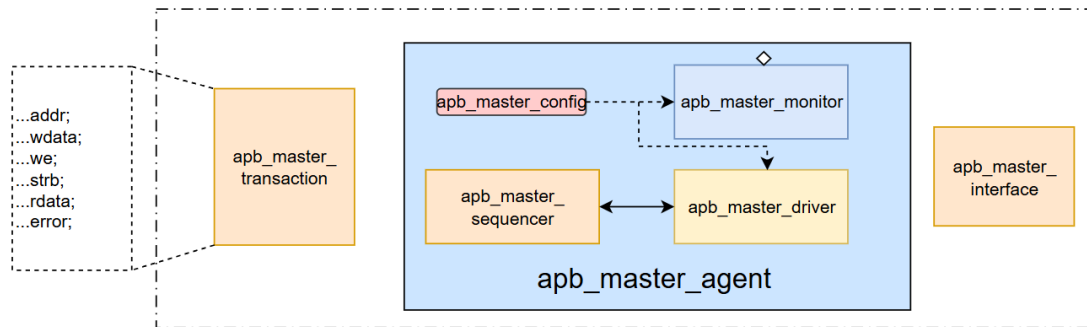


Hình 3.17: Kiến trúc APB Slave VIP

APB Slave VIP — mô phỏng phản hồi ngoại vi

APB Master VIP phục vụ luồng cấu hình thanh ghi nội bộ của bridge. Trong môi trường kiểm chứng, APB Master VIP được kết nối với giao diện APB Register tách biệt hoàn toàn với các APB Slave. VIP này tích hợp với RAL

(Register Abstraction Layer) của UVM thông qua adapter và predictor, cho phép testcase thao tác thanh ghi ở mức tên logic thay vì địa chỉ vật lý, và tự động xác minh giá trị sau mỗi lần ghi. Kiến trúc APB Master VIP được trình bày tại Hình 3.18.



Hình 3.18: Kiến trúc APB Master VIP

3.3.4.2 Tích hợp và kết nối các thành phần UVM Testbench

Sau khi hoàn thành việc xây dựng các Verification IP (VIP), bước tiếp theo là xây dựng cấu trúc phân cấp của UVM Testbench. Môi trường kiểm chứng được tổ chức theo ba tầng chính gồm Testbench Top, Testcase và Environment. Cấu trúc này đóng vai trò khung xương của toàn bộ testbench, tạo nền tảng để tích hợp các VIP và các thành phần kiểm chứng ở các bước tiếp theo.

SystemVerilog Interface gom nhóm các tín hiệu rời rạc thành một thực thể duy nhất để kết nối trực tiếp môi trường kiểm chứng UVM với thiết kế phần cứng. Việc xây dựng interface bao gồm ba bước cốt lõi. Thứ nhất, khai báo toàn bộ các chân tín hiệu đặc thù của giao thức dưới dạng biến logic. Thứ hai, thiết lập khối đồng bộ thời gian clocking block nhằm quy định chính xác thời điểm lấy mẫu và xuất tín hiệu so với sườn xung nhịp, giúp triệt tiêu hoàn toàn lỗi chạy đua tín hiệu. Cuối cùng, định nghĩa cơ chế phân quyền modport để bảo vệ đường truyền: cấp quyền xuất tín hiệu cho Driver để điều khiển bus, và chỉ cấp quyền đọc cho Monitor để giám sát thụ động, tránh làm sai lệch dữ liệu, thể hiện qua hình 3.19.

```

interface apb_slave_if(input bit PCLK, input bit PRESETn);
    logic [`APB_ADDR_WIDTH-1:0] PADDR;
    logic                                PWRITE;
    logic [3:0]                        PSTRB;
    logic                                PSEL0;
    logic                                PSEL1;
    logic                                PSEL2;
    logic                                PENABLE;
    logic [`APB_DATA_WIDTH-1:0] PWDATA;
    logic [`APB_DATA_WIDTH-1:0] PRDATA = 0;
    logic                                PREADY = 0;
    logic                                PSLVERR = 0;
    logic                                DecErrIntr;

    // --- Driver Clocking Block ---
    clocking drv_cb @(posedge PCLK);
        input PADDR, PWRITE, PSTRB, PSEL0, PSEL1, PSEL2, PENABLE, PWDATA, DecErrIntr;
        output PRDATA, PREADY, PSLVERR;
    endclocking

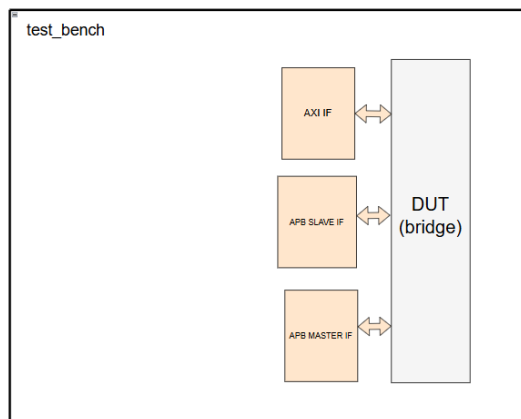
    // --- Monitor Clocking Block ---
    clocking mon_cb @(posedge PCLK);
        input PADDR, PWRITE, PSTRB, PSEL0, PSEL1, PSEL2, PENABLE, PWDATA;
        input PRDATA, PREADY, PSLVERR, DecErrIntr;
    endclocking

    modport DRV (clocking drv_cb, input PCLK, input PRESETn);
    modport MON (clocking mon_cb, input PCLK, input PRESETn);

```

Hình 3.19: Cấu trúc interface cơ bản

Quá trình bắt đầu từ tầng Testbench Top — lớp tính của môi trường nơi DUT được khởi tạo và kết nối với các interface. Các interface đóng vai trò cầu nối giữa thế giới tín hiệu của DUT và thế giới transaction của UVM. Hình 3.20 minh họa kiến trúc UVM Testbench sau khi hoàn tất quá trình build hierarchy.



Hình 3.20: Kiến trúc Testbench Top sau khởi tạo DUT và các Interface tĩnh

Testbench top đóng gói các cụm tín hiệu rời rạc thành các interface (axi_vif, apb_master_vif, apb_slave_vif) để dễ dàng quản lý và cắm trực tiếp vào các port của DUT, được thể hiện qua hình 3.21.

```

module testbench;
    import uvm_pkg::*;
    import axi_pkg::*;
    import apb_master_pkg::*;
    import apb_slave_pkg::*;
    import test_pkg::*;

    // Clock & Reset signals
    bit ACLK;
    bit ARESETn;
    bit PCLK;
    bit PRESETn;

    // Interfaces
    axi_if      axi_vif(ACLK, ARESETn);
    apb_master_if apb_master_vif(PCLK, PRESETn);
    apb_slave_if apb_slave_vif(PCLK, PRESETn);

    // DUT Instance
    AXI_to_APB_bridge dut (
        .ACLK      (ACLK),
        .ARESETn    (ARESETn),
        .PCLK       (PCLK),
        .PRESETn     (PRESETn),
        // --- AXI Write Address Channel ---
        .AWADDR     (axi_vif.AWADDR),
        .AWLEN       (axi_vif.AWLEN),
        .AWSIZE      (axi_vif.AWSIZE),

```

Hình 3.21: Khai báo interface và kết nối DUT

Testbench top còn giả lập dao động của xung nhịp và tín hiệu reset cứng. Đây là nhịp đập cơ sở để toàn bộ mạch RTL hoạt động, thể hiện qua hình 3.22.

```

// Reset Generation
initial begin
    ARESETn = 0;
    PRESETn = 0;
    #20ns;
    ARESETn = 1;
    PRESETn = 1;
end

// ACLK Generation (100MHz)
initial begin
    ACLK = 0;
    forever #5ns ACLK = ~ACLK;
end

// PCLK Generation (50MHz)
initial begin
    PCLK = 0;
    forever #10ns PCLK = ~PCLK;
end

```

Hình 3.22: Testbench tạo Clock và Reset

Testbench còn là nơi “gửi” các interface vật lý vào một không gian lưu trữ chung của UVM. Các thành phần UVM bên dưới có thể “lấy” interface này ra dùng mà không cần nối dây vật lý phức tạp. Lệnh `run_test()` kích hoạt UVM, bắt đầu chạy các phase (build, connect, run...) theo đúng trình tự, thể hiện qua hình 3.23.

```

// UVM Config DB & Test Run
initial begin
    uvm_config_db#(virtual axi_if)::set(null, "uvm_test_top", "axi_vif", axi_vif);
    uvm_config_db#(virtual apb_master_if)::set(null, "uvm_test_top", "apb_master_vif", apb_master_vif);
    uvm_config_db#(virtual apb_slave_if)::set(null, "uvm_test_top", "apb_slave_vif", apb_slave_vif);

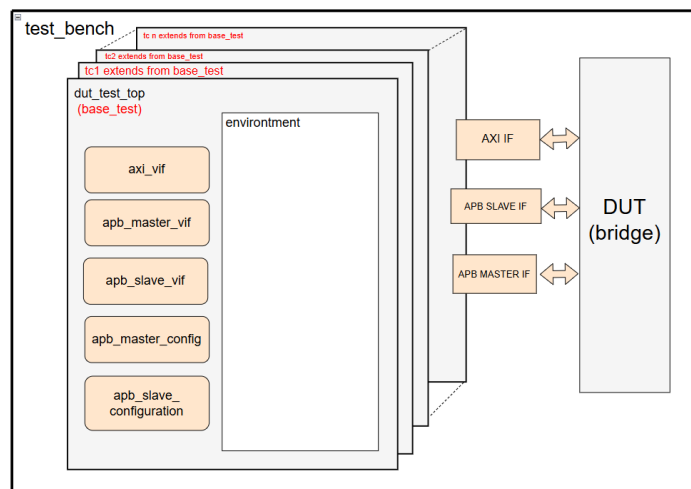
    run_test();
end

```

Hình 3.23: Testbench đăng ký UVM Config DB và khởi động UVM

Tiếp theo, quá trình build đi xuống tầng Testcase. Trong môi trường kiểm chứng, một lớp Base Test được xây dựng làm nền tảng cho toàn bộ các testcase. Base Test chịu trách nhiệm khởi tạo Environment và thiết lập các thông tin cấu

hình cần thiết thông qua UVM Config Database, bao gồm virtual interface và các configuration object được sử dụng bởi các VIP. Nhờ cơ chế này, các thành phần bên trong Environment có thể truy cập tài nguyên và nhận cấu hình một cách tập trung mà không cần phụ thuộc trực tiếp vào cấu trúc phân cấp của hệ thống. Ở giai đoạn này, môi trường chỉ thực hiện build Base Test và các thành phần hạ tầng liên quan; các testcase cụ thể và sequence tương ứng sẽ được trình bày trong Mục 3.3.5. Hình 3.24 trình bày kiến trúc tầng Testcase sau khi tích hợp Environment và Config Database.



Hình 3.24: Kiến trúc Testcase

UVM Test là tầng cao nhất trong hệ thống phân cấp class, đóng vai trò như một người tổng chỉ huy. Tầng này khởi tạo cấu hình chung và quyết định xem hệ thống sẽ chạy kịch bản (Sequence) nào. Thay vì mỗi testcase phải tự setup lại từ đầu, base_test chứa các thao tác lặp đi lặp lại như: lấy giao diện từ DB, khởi tạo cấu hình và tạo ra lớp environment, thể hiện qua hình 3.25

```

class apb_base_test extends uvm_test;
    `uvm_component_utils(apb_base_test)

    environment          env;
    apb_master_config    apb_mst_cfg;
    apb_slave_configuration    apb_slv_cfg;

    virtual axi_if axi_vif;
    virtual apb_master_if apb_master_vif;
    virtual apb_slave_if apb_slave_vif;

    uvm_report_server    svr;
    apb_reg_block        regmodel;

virtual function void build_phase (uvm_phase phase);
    super.build_phase(phase);
    `uvm_info("build_phase", "Entered...", UVM_HIGH)

    if (!uvm_config_db#(virtual axi_if)::get(this,"", "axi_vif", axi_vif))
        `uvm_fatal(get_type_name(), $sformatf("Failed to get axi_vif from uvm_config_db"))
    if (!uvm_config_db#(virtual apb_master_if)::get(this,"", "apb_master_vif", apb_master_vif))
        `uvm_fatal(get_type_name(), $sformatf("Failed to get apb_master_vif from uvm_config_db"))
    if (!uvm_config_db#(virtual apb_slave_if)::get(this,"", "apb_slave_vif", apb_slave_vif))
        `uvm_fatal(get_type_name(), $sformatf("Failed to get apb_slave_vif from uvm_config_db"))

    env          = environment::type_id::create("env", this);
    apb_mst_cfg = apb_master_config::type_id::create("apb_mst_cfg", this);
    apb_slv_cfg = apb_slave_configuration::type_id::create("apb_slv_cfg", this);

    uvm_config_db#(virtual axi_if)::set(this,"env", "axi_vif", axi_vif);
    uvm_config_db#(virtual apb_master_if)::set(this,"env", "apb_master_vif", apb_master_vif);
    uvm_config_db#(virtual apb_slave_if)::set(this,"env", "apb_slave_vif", apb_slave_vif);
    uvm_config_db#(apb_master_config)::set(this,"env", "apb_mst_cfg", apb_mst_cfg);
    uvm_config_db#(apb_slave_configuration)::set(this,"env", "apb_slv_cfg", apb_slv_cfg);

    `uvm_info("build_phase", "Exiting...", UVM_HIGH)
endfunction: build_phase

```

Hình 3.25: Cấu trúc Base Test

Mọi testcase đều extends base_test. Chúng ta chỉ cần viết lại run_phase để gọi đúng Sequence mong muốn. raise_objection và drop_objection là cách kiểm soát thời gian chạy. UVM sẽ chờ đến khi tất cả objection được "drop" thì mới kết thúc quá trình mô phỏng, thể hiện qua hình 3.26.

```

class tc11_read_default extends apb_base_test;
  `uvm_component_utils(tc11_read_default)

  uvm_reg_hw_reset_seq seq;

  function new(string name="tc11_read_default", uvm_component parent);
    super.new(name,parent);
  endfunction

  virtual task run_phase(uvm_phase phase);
    // 1. Tạo sequence
    seq = uvm_reg_hw_reset_seq::type_id::create("seq");

    phase.raise_objection(this);

    // 2. GÁN MODE RAL
    seq.model = env.regmodel;

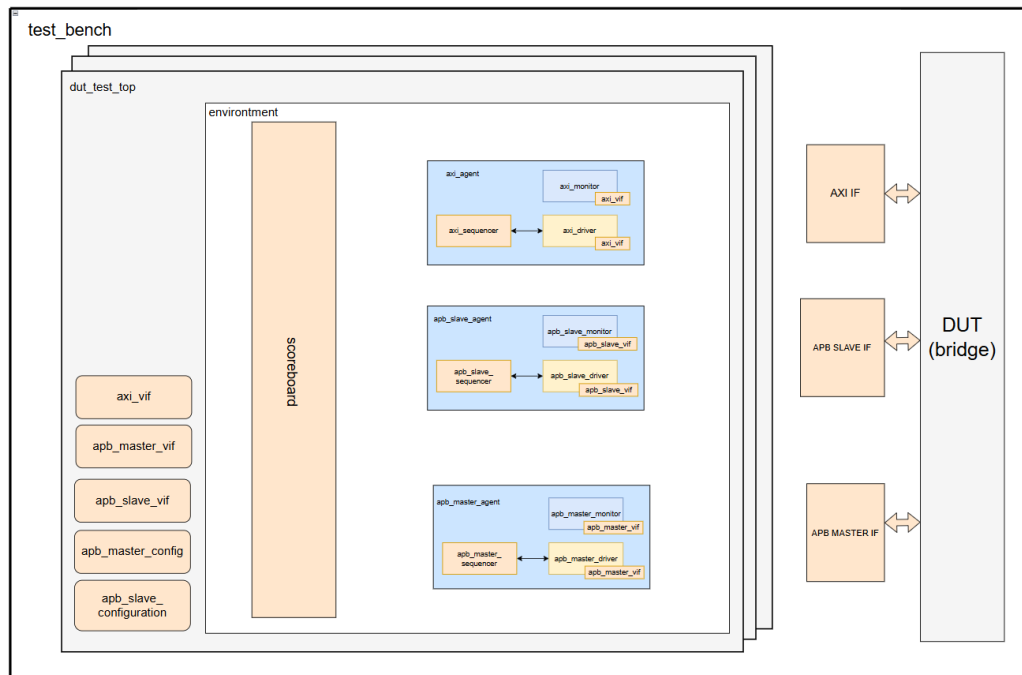
    // 3. START SEQUENCE:
    seq.start(env.apb_master_agt.sqr);

    phase.drop_objection(this);
  endtask
endclass

```

Hình 3.26: Cấu trúc kịch bản test cụ thể

Tại tầng Environment, toàn bộ các thành phần verification được khởi tạo và tích hợp: VIP (AXI Agent, APB Slave Agent, APB Master Agent), Scoreboard. Quan trọng hơn, tầng này cũng thiết lập RAL Predictor — thành phần tự động cập nhật mirror của RAL Model mỗi khi một giao dịch register được quan sát bởi APB Master Monitor. Nhờ đó Scoreboard luôn biết chính xác trạng thái thanh ghi hiện tại của DUT mà không cần đọc thanh ghi chủ động. Hình 3.27 trình bày kiến trúc tầng Environment.



Hình 3.27: Kiến trúc Environment sau khi tích hợp VIP và Scoreboard

Theo quy tắc của UVM build_phase luôn chạy từ trên xuống dưới. Lớp cha (Environment) phải khởi tạo xong các khối con (Agent, Scoreboard, Coverage) trước khi chuyển sang bước tiếp theo, thể hiện qua hình 3.28.

```

class environment extends uvm_env;

  `uvm_component_utils(environment)

  // Components
  scoreboard      sb;
  apb_master_agent apb_master_agt;
  apb_slave_agent  apb_slave_agt;
  axi_agent        axi_agt;
  bridge_coverage  cov;

  // Configs
  apb_master_config      apb_mst_cfg;
  apb_slave_configuration apb_slv_cfg;

  // Virtual interfaces
  virtual axi_if      axi_vif;
  virtual apb_master_if apb_master_vif;
  virtual apb_slave_if  apb_slave_vif;

  // RAL
  apb_reg_block regmodel;
  apb_reg_adapter apb_adapter;
  uvm_reg_predictor #(apb_master_seq_item) apb_predictor;

  // Create components
  sb          = scoreboard::type_id::create("sb", this);
  apb_master_agt = apb_master_agent::type_id::create("apb_master_agt", this);
  apb_slave_agt  = apb_slave_agent::type_id::create("apb_slave_agt", this);
  axi_agt        = axi_agent::type_id::create("axi_agt", this);
  cov            = bridge_coverage::type_id::create("cov", this);

  // Create RAL
  regmodel = apb_reg_block::type_id::create("regmodel", this);
  regmodel.build();
  regmodel.reset();

```

Hình 3.28: Environment khởi tạo các thành phần trong build_phase

Trong connect_phase, các khối bắt đầu nối cổng (port) lại với nhau. Các kết nối này sử dụng cơ chế TLM Analysis Port – một dạng kết nối một chiều, non-blocking. Monitor cứ việc gửi dữ liệu, Scoreboard và Coverage sẽ "lắng nghe" và thu thập, thể hiện qua hình 3.29.

```

// Connect phase
virtual function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    `uvm_info("ENV_CONNECT", "Entered connect_phase", UVM_HIGH)

    // Scoreboard connect
    apb_master_agt.mon.item_collected_port.connect(sb.mon_export);
    apb_master_agt.drv.drv_proxy_port.connect(sb.drv_export);
    apb_slave_agt.monitor.apb_item_act.connect(sb.slv_export);
    axi_agt.monitor.axi_item_act.connect(sb.axi_export);
    // Coverage connect
    axi_agt.monitor.axi_item_act.connect(cov.axi_imp);
    apb_master_agt.mon.item_collected_port.connect(cov.apb_master_imp);
    apb_slave_agt.monitor.apb_item_act.connect(cov.apb_slave_imp);
    // RAL frontdoor
    regmodel.apb_map.set_sequencer(apb_master_agt.sqr, apb_adapter);
    // Predictor connect
    apb_master_agt.mon.item_collected_port.connect(apb_predictor.bus_in);

    `uvm_info("ENV_CONNECT", "Exiting connect_phase", UVM_HIGH)
endfunction : connect_phase

```

Hình 3.29: Environment nối dây luồng dữ liệu (connect_phase)

Mỗi Agent chịu trách nhiệm cho một giao thức duy nhất (ở đồ án này là AXI, APB Slave, APB Master). Nó gom nhóm 3 thành phần cốt lõi: Sequencer (điều phối), Driver (phát tín hiệu), và Monitor (theo dõi). Nếu Agent ở chế độ UVM_ACTIVE, nó tạo ra cả Driver và Sequencer để tương tác với DUT. Nếu ở chế độ UVM_PASSIVE, nó chỉ tạo Monitor để quan sát (thường dùng khi có thiết bị khác đang làm Master trên bus), thể hiện qua hình 3.30.

```

class apb_slave_agent extends uvm_agent;
    `uvm_component_utils(apb_slave_agent)

    // --- Sub-components ---
    apb_slave_monitor      monitor;
    apb_slave_driver       driver;
    apb_slave_sequencer     sequencer;
    apb_slave_configuration apb_slv_cfg;

    // --- Virtual Interface ---
    virtual apb_slave_if apb_slave_vif;

// 3. Create Monitor (Always present)
monitor = apb_slave_monitor::type_id::create("monitor", this);

// Pass config and interface to Monitor
uvm_config_db#(apb_slave_configuration)::set(this, "monitor", "apb_slv_cfg", apb_slv_cfg);
uvm_config_db#(virtual apb_slave_if)::set(this, "monitor", "apb_slave_vif", apb_slave_vif);

// 4. Create Driver and Sequencer only if Agent is ACTIVE
if (get_is_active() == UVM_ACTIVE) begin
    sequencer = apb_slave_sequencer::type_id::create("sequencer", this);
    driver     = apb_slave_driver::type_id::create("driver", this);

    // Pass config and interface to Driver
    uvm_config_db#(apb_slave_configuration)::set(this, "driver", "apb_slv_cfg", apb_slv_cfg);
    uvm_config_db#(virtual apb_slave_if)::set(this, "driver", "apb_slave_vif", apb_slave_vif);

```

Hình 3.30: Agent khởi tạo các thành phần

Agent là nơi chứa cả Sequencer và Driver, Sequencer là nơi chứa các kịch bản, Driver là người thực thi. Lệnh connect này đảm bảo khi Driver rảnh, nó có thể gọi `get_next_item()` để kéo transaction tiếp theo từ Sequencer xuống, thể hiện qua hình 3.31.

```

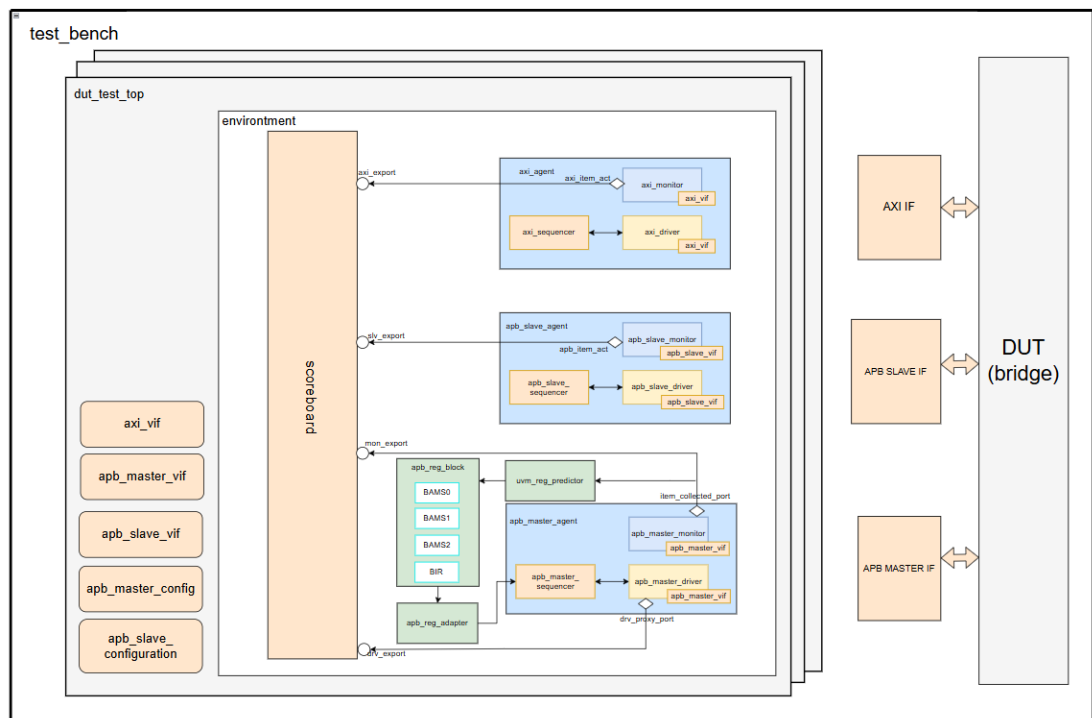
// --- Connect Phase ---
virtual function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);

    // Connect Driver to Sequencer for ACTIVE agents
    if (get_is_active() == UVM_ACTIVE) begin
        driver.seq_item_port.connect(sequencer.seq_item_export);
    end
endfunction: connect_phase

```

Hình 3.31: Agent kết nối Driver với Sequencer

Sau khi hoàn tất quá trình build hierarchy, các thành phần trong môi trường kiểm chứng được kết nối thông qua cơ chế TLM của UVM. Các Monitor trong AXI VIP, APB Slave VIP và APB Master VIP phát transaction qua Analysis Port đến Scoreboard. Đồng thời, APB Master Monitor được kết nối với RAL Predictor để cập nhật trạng thái các thanh ghi cấu hình. Nhờ các kết nối này, Scoreboard có thể thu thập đầy đủ thông tin từ các giao diện quan sát và thực hiện kiểm tra tính đúng đắn của DUT. Hình 3.32 trình bày kiến trúc UVM Testbench sau khi hoàn tất quá trình kết nối.



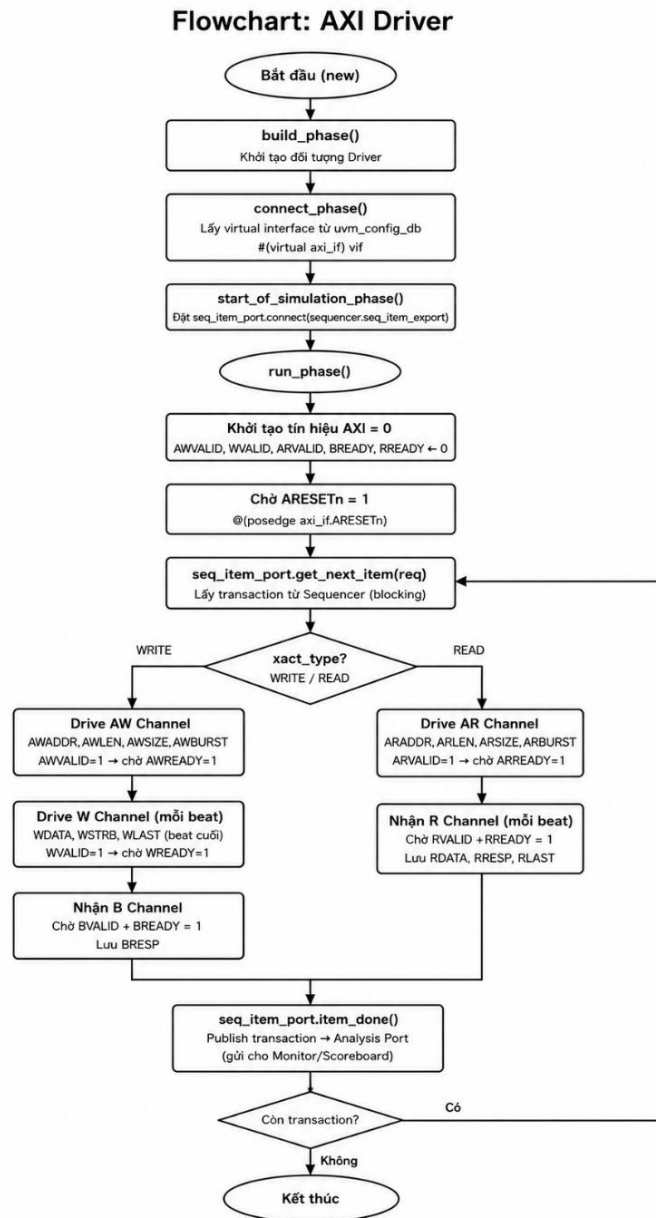
Hình 3.32: Kiến trúc UVM Testbench sau khi hoàn tất kết nối

3.3.4.3 Cơ chế xử lý giao dịch và kiểm tra kết quả trong UVM

Các thành phần Driver, Monitor và Scoreboard đóng vai trò trung tâm trong quá trình xử lý giao dịch và kiểm tra kết quả của DUT. Phần này phân tích cơ chế hoạt động của từng thành phần nhằm làm rõ cách dữ liệu được tạo, thu thập và sử dụng để đánh giá tính đúng đắn của thiết kế.

AXI Driver

AXI Driver đảm nhận vai trò chuyển đổi transaction object trừu tượng từ Sequencer thành chuỗi tín hiệu vật lý trên bus AXI4. Driver không chứa bất kỳ logic sinh stimulus nào (toàn bộ quyết định về nội dung giao dịch đến từ Sequence) mà chỉ tập trung vào việc thực thi đúng giao thức handshake trên từng kênh AXI. Flowchart hoạt động được trình bày tại Hình 3.33.



Hình 3.33: Flowchart hoạt động của AXI Driver

Trong `build_phase()`, Driver được khởi tạo như một UVM đối tượng. Sang `connect_phase()`, Driver lấy giao diện ảo từ `uvm_config_db` và kết nối

seq_item_port với Sequencer - đây là bước thiết lập giao dịch truyền kênh giữa hai thành phần.

Khởi tạo: Tất cả tín hiệu AXI được gán giá trị 0 và Driver chờ ARESETn=1 trước khi bắt đầu xử lý. Việc khởi tạo đúng trạng thái idle là quan trọng vì AXI4 định nghĩa rõ trạng thái hợp lệ của các tín hiệu khi không có giao dịch.

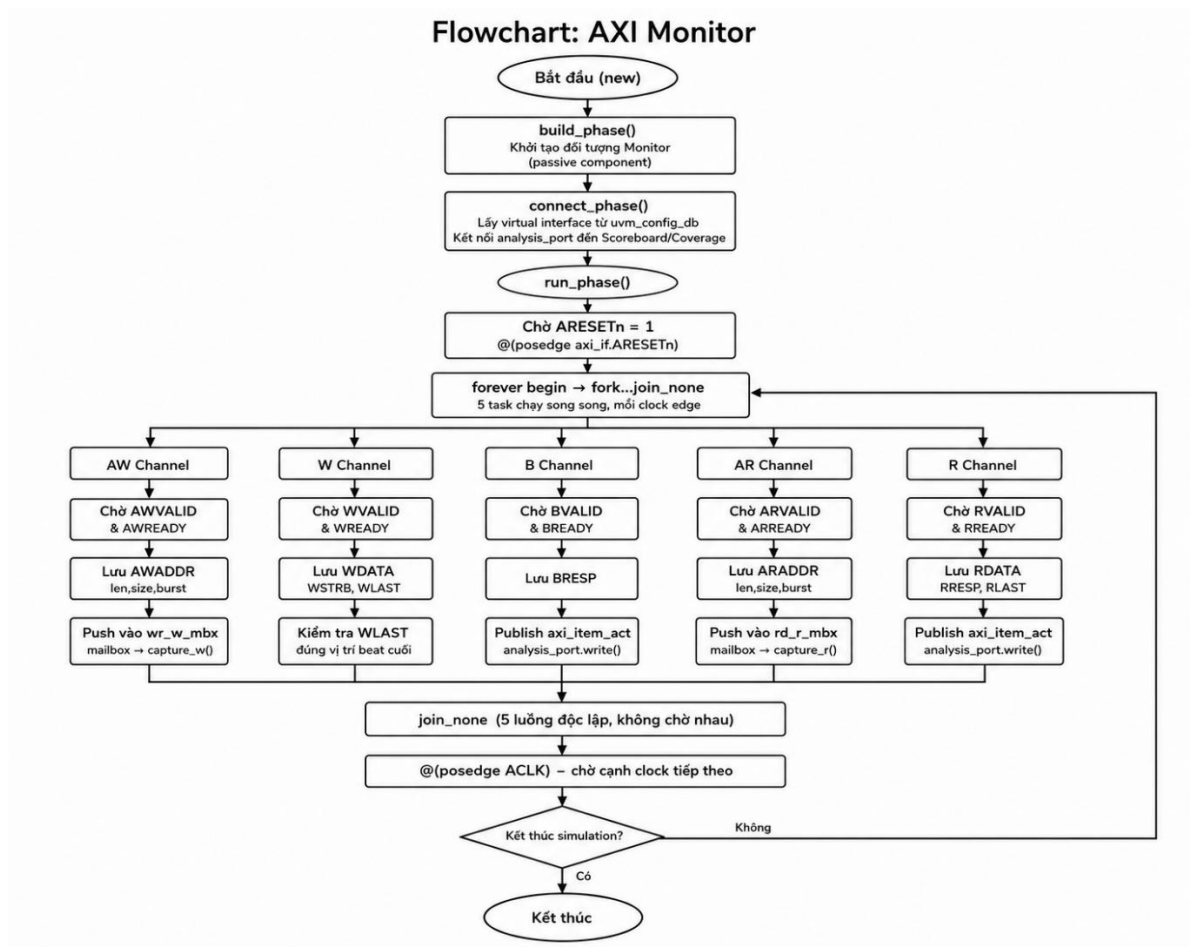
Vòng lặp chính: Driver gọi get_next_item() để lấy transaction từ Sequencer, xử lý, rồi gọi item_done() để thông báo hoàn tất. Chu kỳ này lặp vô hạn trong suốt simulation, đảm bảo Driver luôn sẵn sàng xử lý transaction tiếp theo ngay khi Sequencer cung cấp.

WRITE path: Driver lần lượt thực hiện handshake trên kênh AW (địa chỉ), kênh W (dữ liệu, mỗi beat với WLAST đúng vị trí), và chờ phản hồi trên kênh B. Theo đặc tả AXI4, kênh AW và W có thể hoạt động đồng thời - Driver tận dụng điều này để tối ưu thông lượng.

READ path: Driver thực hiện handshake trên kênh AR (địa chỉ), sau đó thu thập từng beat dữ liệu trên kênh R và xác nhận RLAST tại đúng beat cuối. Driver lưu lại RDATA để Scoreboard có thể so sánh sau.

AXI Monitor

AXI Monitor hoạt động hoàn toàn thụ động, quan sát các tín hiệu AXI qua clocking block mon_cb mà không ảnh hưởng đến trạng thái bus. Điểm thiết kế quan trọng nhất của AXI Monitor là kiến trúc đa luồng: năm task song song capture_aw(), capture_w(), capture_b(), capture_ar() và capture_r() chạy trong fork...join, phản ánh đúng bản chất bất đồng bộ của năm kênh AXI. Flowchart được trình bày tại Hình 3.34.



Hình 3.34: Flowchart hoạt động của AXI Monitor

Trong `build_phase()`, Monitor được khởi tạo như thành phần thụ động. Trong `connect_phase()`, Monitor lấy giao diện ảo từ `uvm_config_db` và kết nối `analysis_port` với Scoreboard và Coverage Collector.

Khi vào `run_phase()`, Monitor chờ `ARESETn = 1` rồi chạy vòng lặp vô hạn. Mỗi lần khởi động vòng lặp `fork...join_none` với tác vụ năm được chạy đồng thời, sau đó đồng bộ tại `@(posedge ACLK)` trước khi lặp lại.

`capture_aw()` và `capture_w()`: Hai task phối hợp thông qua mailbox `wr_w_mbx` để lắp ráp đầy đủ thông tin của một giao dịch ghi. `capture_aw()` lưu thông tin burst rồi đẩy context vào mailbox; `capture_w()` nhận context và ghi `WDATA/WSTRB` từng beat vào đúng vị trí trong mảng, đồng thời kiểm tra `WLAST` tại beat cuối.

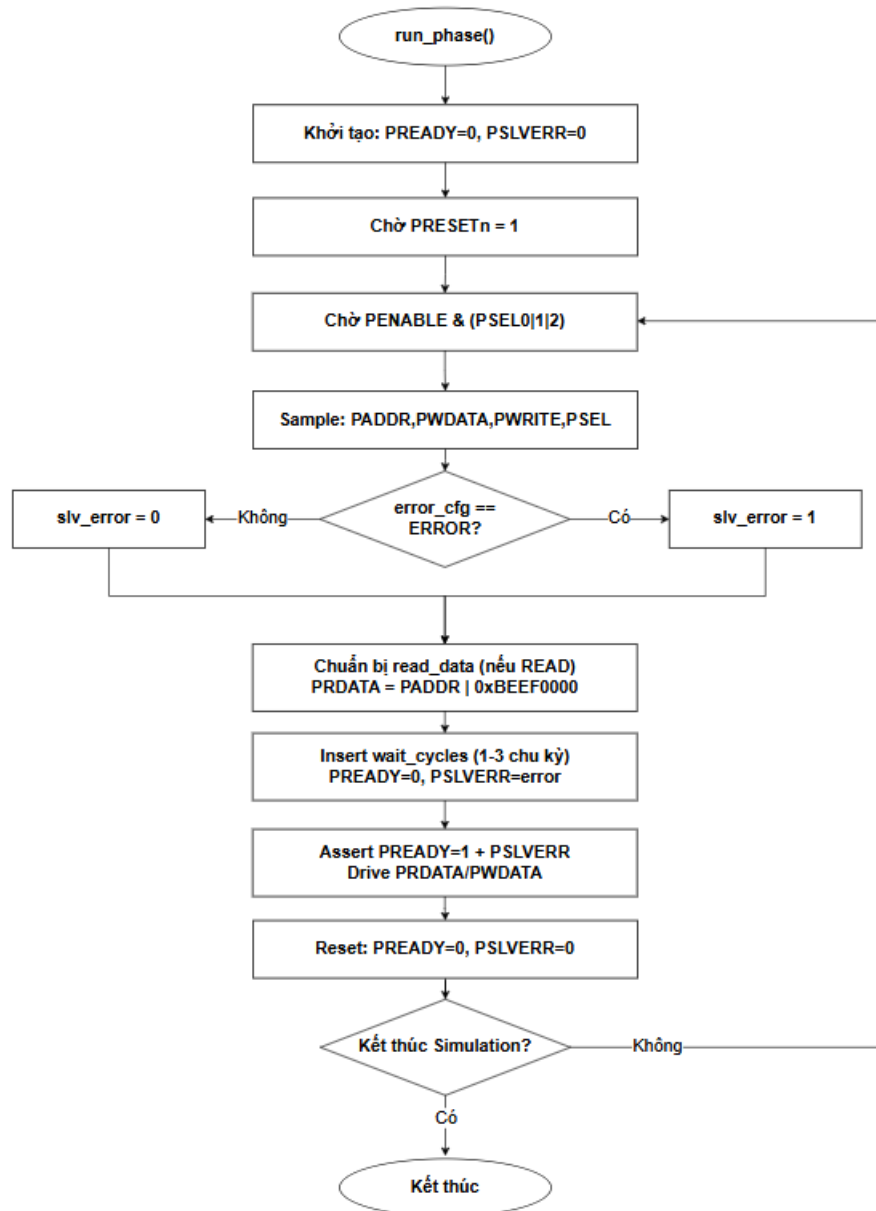
`capture_b()`: Task chờ BVALID/BREADY, đọc BRESP và chờ `w_done=1` (xác nhận W channel đã hoàn tất) trước khi publish transaction hoàn chỉnh. Điều kiện chờ `w_done` là cơ chế đồng bộ hóa quan trọng: đảm bảo Monitor không publish transaction trước khi toàn bộ dữ liệu từ W channel đã được thu thập.

`capture_ar()` và `capture_r()`: Cặp task tương tự cho luồng đọc: `capture_ar()` lưu thông tin burst đọc vào mailbox `rd_r_mbx`; `capture_r()` đọc từng beat RDATA/RRESP và kiểm tra RLAST tại đúng beat cuối theo ARLEN, sau đó publish transaction đọc hoàn chỉnh.

Dữ liệu gửi cho Scoreboard: AXI Monitor gửi `axi_transaction` hoàn chỉnh đến Scoreboard thông qua analysis port. Mỗi transaction chứa địa chỉ, burst type, size, length, toàn bộ WDATA/WSTRB (đối với giao dịch ghi), toàn bộ RDATA (đối với giao dịch đọc), BRESP (đối với giao dịch ghi) và từng RRESP của mỗi beat (đối với giao dịch đọc). Transaction này đóng vai trò dữ liệu đầu vào để Scoreboard xây dựng kết quả mong đợi, từ đó dự đoán hành vi mà DUT phải thực hiện ở phía APB.

APB Slave Driver

APB Slave Driver mô phỏng hành vi phản hồi của APB Slave, bao gồm cả trường hợp thông thường lẫn điều kiện lỗi. Thay vì mô phỏng một slave cụ thể, Driver được thiết kế như một generic slave có thể cấu hình: số wait state được ngẫu nhiên hóa và trạng thái lỗi được kiểm soát qua cấu hình, cho phép testcase kiểm tra bridge dưới nhiều điều kiện slave khác nhau mà không cần nhiều Driver riêng biệt. Flowchart được trình bày tại Hình 3.35.



Hình 3.35: Flowchart hoạt động của APB Slave Driver

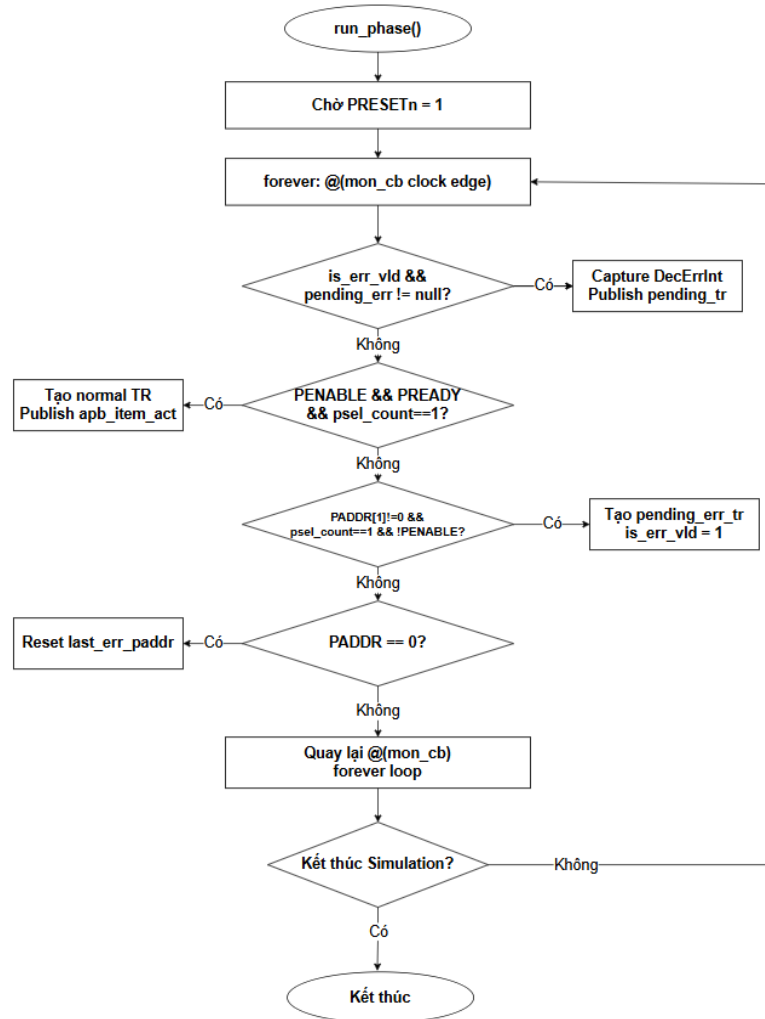
Phát hiện giao dịch: Driver chờ điều kiện $PENABLE=1$ và ít nhất một $PSELx=1$, tức là giai đoạn ACCESS của APB. Quyết định chờ đến ACCESS thay vì SETUP phản ánh thực tế rằng slave cần đọc cả $PENABLE$ lẫn $PSEL$ để biết đây là giao dịch hợp lệ, không phải trạng thái SETUP bình thường.

Wait-state: Số chu kỳ chờ được tạo ngẫu nhiên từ 1 đến 3 (\$urandom_range). Trong suốt thời gian chờ, $PRDATA$ và $PSLVERR$ phải ổn định theo đúng đặc tả APB4, yêu cầu này được kiểm tra bởi SVA trong APB Slave Interface.

Khi cấu hình `error=ERROR`, Driver assert `PSLVERR=1` đồng thời với `PREADY=1`. Read data được tính bằng công thức `PRDATA = PADDR | 0xBEEF_0000` để dễ nhận biết và debug, đồng thời cho phép Scoreboard xác minh dữ liệu ngay cả khi không có slave thực.

APB Slave Monitor

APB Slave Monitor đối mặt với thách thức kỹ thuật đặc thù: tín hiệu `DecErrIntr` xuất hiện một chu kỳ sau khi decode error xảy ra (do logic DUT cần một chu kỳ để xử lý). Vì vậy, Monitor phải duy trì một pipeline lỗi một bước: khi phát hiện decode error ở chu kỳ N, Monitor tạo `pending_err_tr` và đợi đến chu kỳ N+1 để đọc `DecErrIntr` đã ổn định trước khi publish. Flowchart được trình bày tại Hình 3.36.



Hình 3.36: Flowchart hoạt động của APB Slave Monitor

Mỗi chu kỳ clock, Monitor kiểm tra `is_err_vld` trước tiên. Nếu có `pending_err_tr` từ chu kỳ trước, Monitor đọc `DecErrIntr` tại thời điểm hiện tại (đã ổn định sau 1 chu kỳ) và publish transaction lỗi hoàn chỉnh.

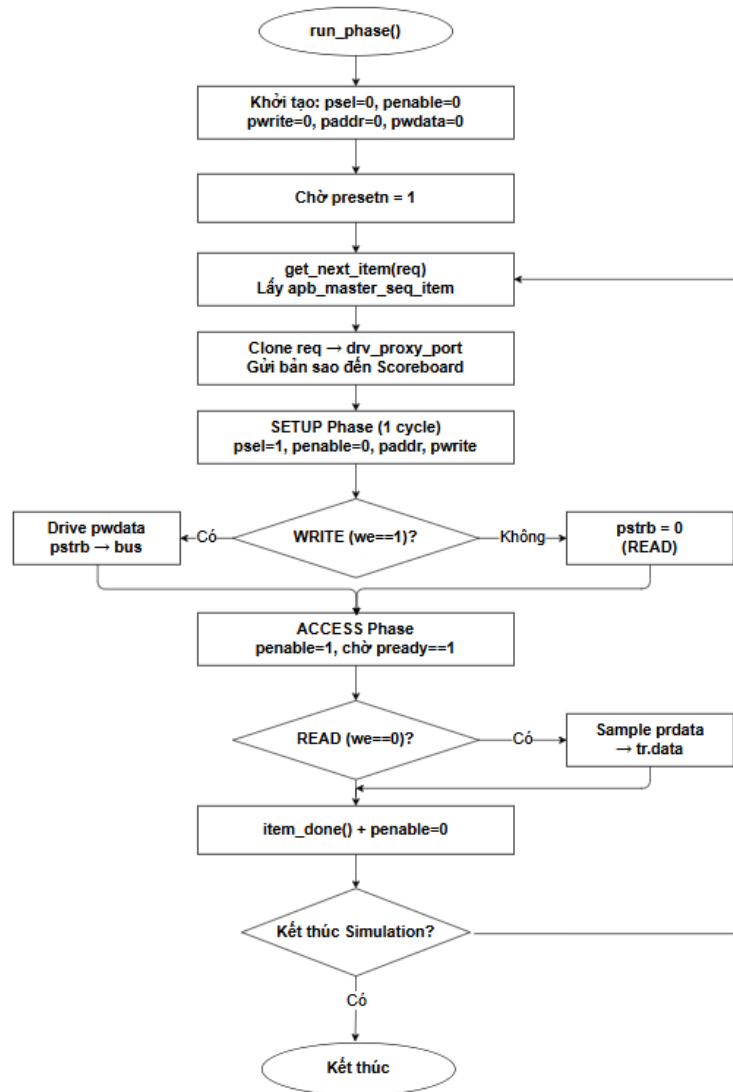
Khi `PENABLE=1`, `PREADY=1` và đúng một `PSELx=1`, Monitor tạo transaction bình thường với đầy đủ thông tin. Điều kiện "đúng một `PSELx`" là quan trọng: nếu không có hoặc nhiều hơn một `PSEL`, đây là trạng thái lỗi, không phải giao dịch thông thường.

Phát hiện decode error: Khi `PADDR` khác 0, `psel_count != 1` và `PENABLE=0`, Monitor nhận ra đây là giao dịch bị lỗi decode (`PSEL_NONE` hoặc `PSEL_MULTI`). Monitor tạo `pending_err_tr` và bật `is_err_vld` để xử lý ở chu kỳ sau.

Transaction APB được gửi đến Scoreboard thông qua analysis port và được xem là kết quả thực tế của DUT. Các thông tin như địa chỉ, dữ liệu, `PSEL`, `PSLVERR` và trạng thái decode error được sử dụng để đối chiếu với hành vi mong đợi được suy ra từ giao dịch AXI đầu vào.

APB Master Driver

APB Master Driver thực thi giao thức APB để ghi và đọc thanh ghi nội bộ của bridge qua giao diện APB Register. Driver này hoạt động theo cơ chế RAL frontdoor: mọi thao tác thanh ghi đều đi qua bus vật lý đảm bảo rằng cả đường dữ liệu lẫn logic decode địa chỉ trong `APB_REGISTER` đều được kiểm tra. Flowchart được trình bày tại Hình 3.37.



Hình 3.37: Flowchart hoạt động của APB Master Driver

SETUP Phase: Driver assert $psel=1$ và drive $paddr$, $pwrite$, $pstrb$ lên bus trong một chu kỳ PCLK với $penable=0$. Nếu là giao dịch ghi, $pwdata$ và $pstrb$ cũng được drive ổn định ngay từ SETUP để đảm bảo slave có đủ thời gian setup trước ACCESS.

ACCESS Phase: Driver assert $penable=1$ và chờ $pready=1$ trong vòng lặp do-while. Trong suốt thời gian chờ, tất cả tín hiệu địa chỉ, dữ liệu và điều khiển phải được giữ ổn định theo đúng ràng buộc APB4.

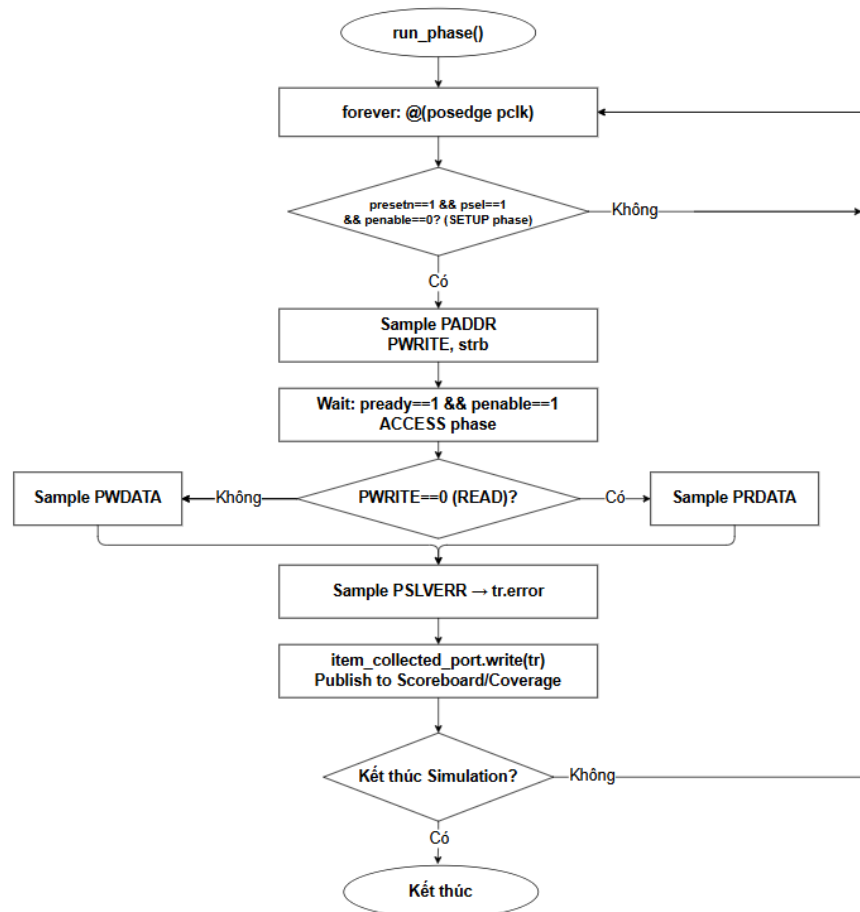
Proxy port: Trước khi thực thi, Driver gửi bản sao transaction qua drv_proxy_port đến Scoreboard. Cơ chế này cho phép Scoreboard biết trước

intent của Driver (address, write/read, data) mà không cần chờ Monitor thu thập — quan trọng trong các kịch bản timing nhạy cảm.

Sample Read Data: Với giao dịch đọc ($we=0$), prdata được sample tại cạnh clock khi pready=1 và lưu vào tr.rdata. RAL Model sẽ cập nhật mirror dựa trên giá trị này thông qua bus2reg() của adapter.

APB Master Monitor

APB Master Monitor giám sát luồng cấu hình thanh ghi qua giao diện APB Register. Điểm khác biệt so với APB Slave Monitor là Monitor này không cần xử lý decode error hay PSEL phức tạp — luồng thanh ghi chỉ có một slave duy nhất là APB_REGISTER. Thay vào đó, Monitor tập trung vào việc phát hiện chính xác các pha SETUP và ACCESS để thu thập đầy đủ thông tin giao dịch. Flowchart được trình bày tại Hình 3.38.



Hình 3.38: Flowchart hoạt động của APB Master Monitor

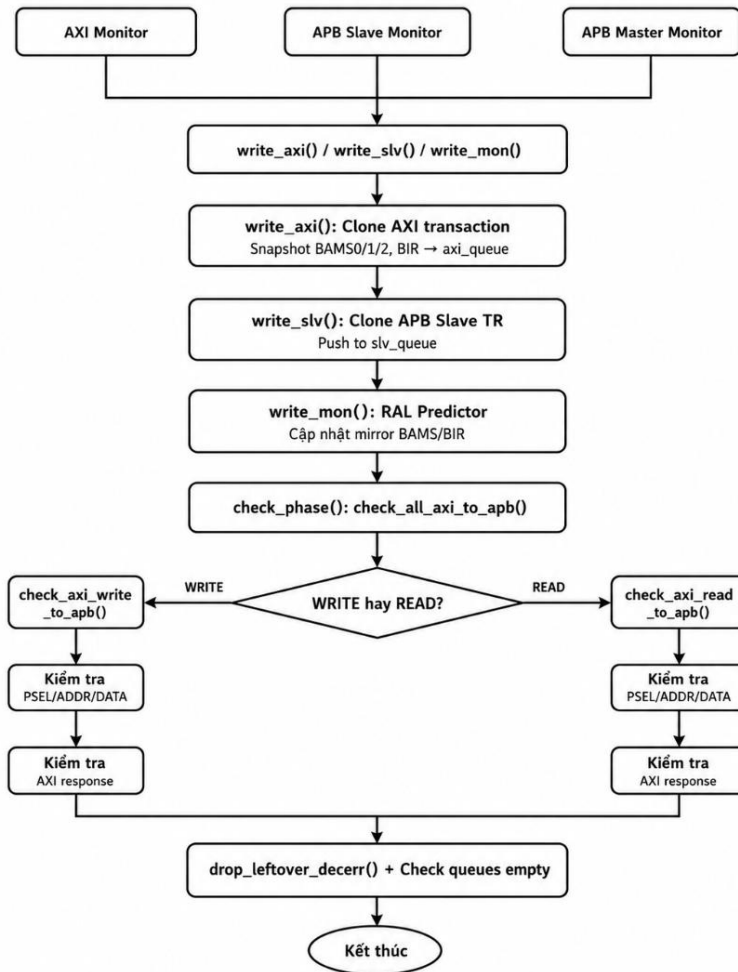
Phát hiện SETUP phase: Monitor quan sát điều kiện $presetn=1$, $psel=1$ và $penable=0$ tại mỗi cạnh clock dương. Khi điều kiện thỏa, PADDR, PWRITE và PSTRB được sample - đây là thời điểm các tín hiệu này ổn định và hợp lệ theo đặc tả APB.

Chờ ACCESS phase: Monitor sử dụng vòng lặp do-while chờ cả $PREADY=1$ lẫn $PENABLE=1$ đồng thời. Điều kiện kép này cần thiết vì $PREADY=0$ trong wait state vẫn có $PENABLE=1$, Monitor phải phân biệt được wait state với giao dịch hoàn tất thực sự.

Sau ACCESS hoàn tất, nếu là giao dịch đọc ($PWRITE=0$), PRDATA được sample và lưu vào tr.rdata để RAL Predictor cập nhật mirror. Transaction được gửi đến cả APB Master Scoreboard export lẫn RAL Predictor bus_in port.

Scoreboard

Scoreboard là thành phần kiểm chứng trung tâm, đặt mình ở vị trí có thể quan sát đồng thời ba luồng thông tin: giao dịch AXI đầu vào, giao dịch APB đầu ra và trạng thái thanh ghi cấu hình. Khả năng kết hợp ba chiều này là điều tạo nên sức mạnh của Scoreboard: không chỉ kiểm tra địa chỉ và dữ liệu đúng, mà còn kiểm tra rằng kết quả nhất quán với cấu hình BAMS tại thời điểm chính xác giao dịch xảy ra. Flowchart được trình bày tại Hình 3.39.



Hình 3.39: Flowchart hoạt động của Scoreboard

Nguồn dữ liệu của Scoreboard đến từ ba thành phần chính: AXI Monitor cung cấp transaction AXI đầu vào, APB Slave Monitor cung cấp transaction APB được DUT phát sinh, và RAL Predictor cung cấp trạng thái hiện tại của các thanh ghi cấu hình. Dựa trên transaction AXI và snapshot cấu hình BAMS/BIR, Scoreboard xây dựng kết quả mong đợi. Các transaction APB được quan sát từ phía DUT được xem là kết quả thực tế. Quá trình kiểm tra được thực hiện bằng cách đối chiếu địa chỉ, dữ liệu, tín hiệu chọn slave, phản hồi lỗi và trạng thái ngắt giữa expected và actual. Một giao dịch được đánh giá PASS khi toàn bộ các trường liên quan khớp với dự đoán; ngược lại, bất kỳ sai lệch nào đều được ghi nhận là FAIL và báo lỗi thông qua Scoreboard.

Thu thập và snapshot: Khi `write_axi()` được gọi, Scoreboard không chỉ lưu transaction AXI mà còn ghi nhận lại (snapshot) giá trị mirror của BAMS0, BAMS1, BAMS2 và BIR tại đúng thời điểm đó. Cơ chế snapshot này là thiết yếu cho tính năng runtime remap: BAMS có thể thay đổi sau khi giao dịch AXI đã bắt đầu, nhưng Scoreboard phải dùng giá trị BAMS tại thời điểm giao dịch bắt đầu để kiểm tra đúng.

Giải mã địa chỉ từ snapshot: Hàm `predict_psel_from_snapshot()` tái tạo logic giải mã địa chỉ của DUT bằng cách dùng BAMS snapshot để tính base address và size của từng slave, rồi so sánh với địa chỉ beat. Kết quả có thể là `DECODE_OK` (khớp đúng một slave), `DECODE_NO_MATCH` (không khớp slave nào) hay `DECODE_MULTI_MATCH` (khớp nhiều slave), tương ứng với `OKAY`, `DECERR` và `DECERR`.

Kiểm tra WRITE: Với mỗi beat của AXI WRITE burst, Scoreboard sử dụng transaction AXI cùng snapshot BAMS để dự đoán địa chỉ APB, slave đích và dữ liệu mong đợi. Các giá trị này được đối chiếu với APB WRITE transaction thu thập từ APB Slave Monitor. Scoreboard kiểm tra tính chính xác của `PADDR`, `PWDATA`, `PSTRB` và tín hiệu chọn slave tương ứng. Cuối burst, `BRESP` được so sánh với kết quả tổng hợp từ tất cả các beat. Giao dịch được đánh giá `PASS` khi toàn bộ beat được ánh xạ chính xác và `BRESP` khớp với dự đoán; ngược lại sẽ được ghi nhận là `FAIL`.

Kiểm tra READ: Tương tự WRITE nhưng bổ sung kiểm tra dữ liệu đọc. Scoreboard sử dụng transaction AXI và thông tin APB READ để dự đoán dữ liệu mà bridge phải trả về trên kênh R. Giá trị `PRDATA` từ APB Slave phải được chuyển chính xác thành `RDATA` tương ứng, đồng thời `RRESP` phải phản ánh đúng trạng thái của slave tại từng beat. Giao dịch READ được đánh giá `PASS` khi toàn bộ dữ liệu và phản hồi đọc khớp với dự đoán.

Kiểm tra Decode Error và Interrupt: Khi phát hiện decode error, Scoreboard kiểm tra `BIR.DecErrEn` trong snapshot để dự đoán xem ngắt có được phát hay không. Nếu `DecErrEn=1`, một `DecErr` event phải tồn tại trong `slv_queue`; nếu

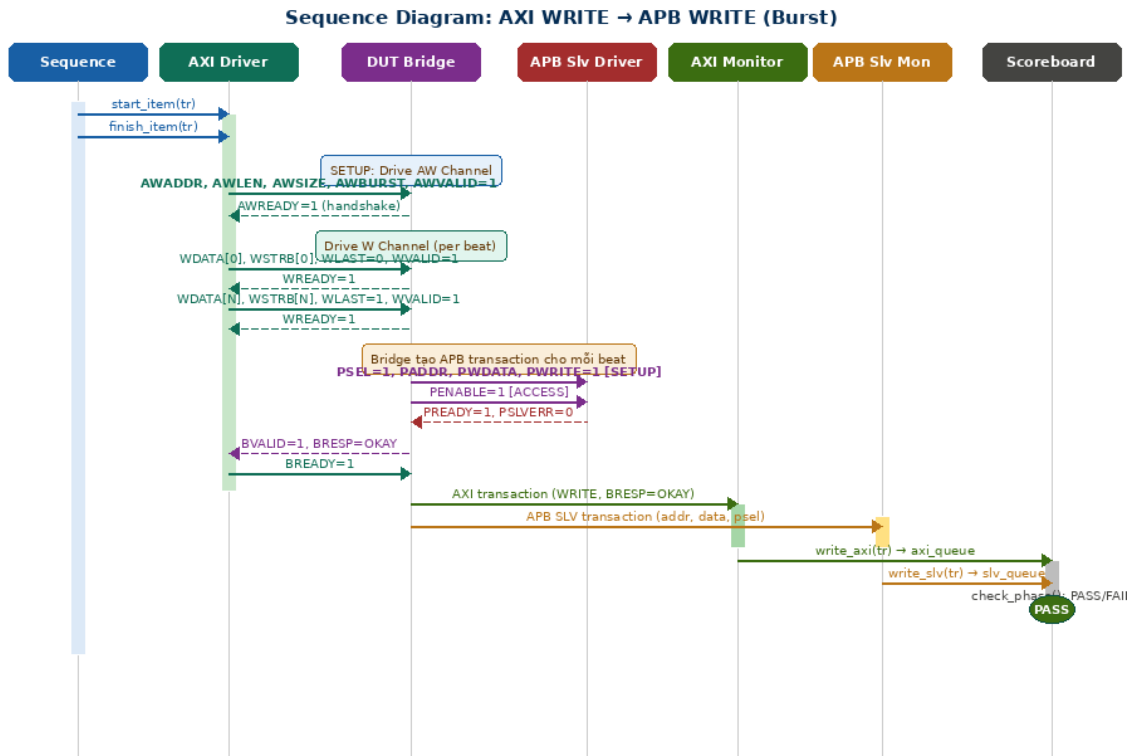
DecErrEn=0, không được có event nào. Đồng thời, BRESP/RRESP phải mang giá trị DECERR tương ứng với kết quả giải mã địa chỉ. Scoreboard cũng gọi BIR.DecErrSt.predict() để cập nhật mirror trạng thái ngất. Bất kỳ sự không nhất quán nào giữa phản hồi AXI, transaction APB và trạng thái thanh ghi cấu hình đều được xem là FAIL.

3.3.4.4 Các luồng giao dịch điển hình trong môi trường kiểm chứng

Sau khi xác định các nhóm testcase trong VPlan, việc xây dựng sequence đóng vai trò thiết yếu trong việc tái sử dụng kích thích và tránh lặp code giữa các testcase. Thay vì tạo một sequence riêng cho từng testcase, môi trường kiểm chứng được thiết kế với vài sequence cốt lõi có tham số hóa linh hoạt, đủ để phủ toàn bộ 70 testcase trong VPlan. Mỗi sequence đại diện cho một luồng tương tác khác nhau trong hệ thống.

Luồng AXI WRITE Burst → APB WRITE

Hình 3.40 mô tả toàn bộ luồng xử lý khi testcase thực hiện một AXI WRITE burst. Sequence kích hoạt AXI Driver, Driver drive tín hiệu AW và W channel, DUT Bridge chuyển đổi từng beat sang APB transaction, APB Slave Driver phản hồi, cuối cùng hai Monitor thu thập và gửi kết quả đến Scoreboard để so sánh.



Hình 3.40: Sequence diagram luồng AXI WRITE burst → APB WRITE

Handshake kênh AW: AXI Driver assert AWVALID cùng AWADDR, AWLEN, AWSIZE, AWBURST. Bridge chấp nhận bằng AWREADY=1. Theo đặc tả AXI4, payload phải ổn định trong suốt thời gian AWVALID=1 và AWREADY=0.

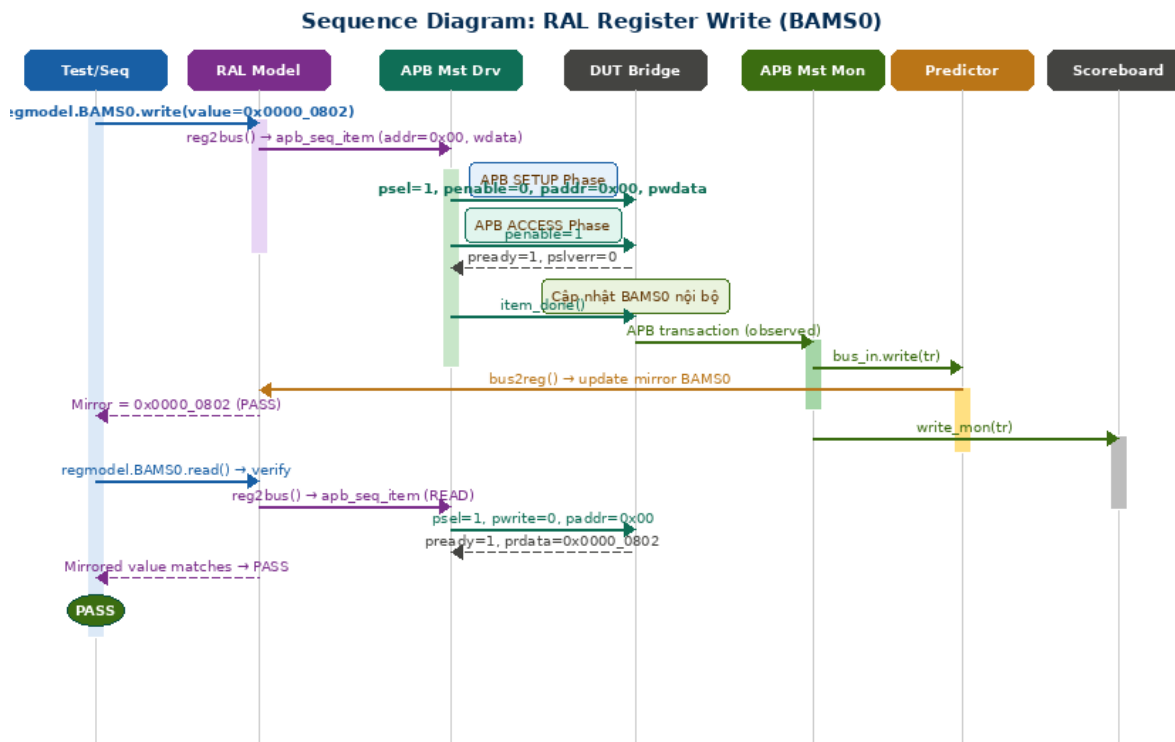
Kênh W: Driver gửi từng beat WDATA/WSTRB, assert WLAST=1 duy nhất tại beat cuối. Bridge xử lý từng beat thành một APB WRITE transaction độc lập, trình tự này được APB Slave Monitor ghi nhận và Scoreboard xác minh.

BRESP và kiểm tra: Sau khi tất cả APB transaction hoàn tất, Bridge tổng hợp BRESP (OKAY, SLVERR hoặc DECERR tùy kết quả các beat) và trả về qua kênh B. AXI Monitor thu thập BRESP và Scoreboard so sánh với giá trị kỳ vọng được tính từ kết quả các APB transaction quan sát được.

Luồng ghi thanh ghi qua RAL (Register Write)

Hình 3.41 mô tả luồng khi testcase sử dụng RAL Model để ghi thanh ghi BAMS0. RAL tự động tạo APB transaction qua reg2bus(), APB Master Driver

thực thi trên bus, APB Master Monitor quan sát và gửi về RAL Predictor để cập nhật mirror value.



Hình 3.41: Sequence diagram luồng ghi thành ghi qua RAL Model

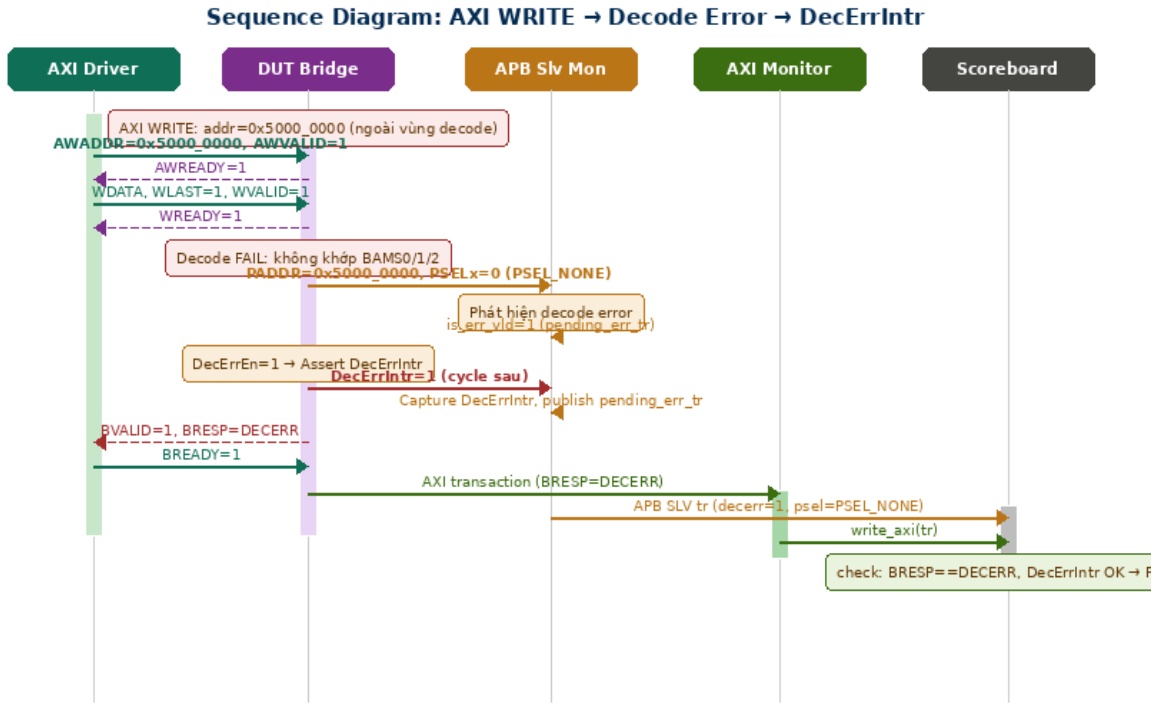
Chuyển đổi `reg2bus()`: Khi testcase gọi `regmodel.BAMS0.write(value)`, RAL tạo `uvm_reg_bus_op` rồi gọi `reg2bus()` của adapter để chuyển thành `apb_master_seq_item` với địa chỉ byte đúng và strobe phù hợp.

APB Master Monitor quan sát giao dịch hoàn tất và gửi đến Predictor. Predictor gọi `bus2reg()` để chuyển ngược lại thành `uvm_reg_bus_op`, rồi cập nhật mirror BAMS0 với giá trị vừa ghi. Từ thời điểm này, mọi thao tác Scoreboard đọc BAMS0 đều nhận được giá trị mới.

Testcase thực hiện `regmodel.BAMS0.read()` với `UVM_CHECK` để xác minh giá trị thực tế trên bus khớp với giá trị mirror. Đây là bước quan trọng để phát hiện các lỗi bit không ghi được.

Luồng Decode Error và DecErrIntr

Hình 3.42 mô tả kịch bản AXI WRITE đến địa chỉ nằm ngoài vùng được cấu hình trong BAMS. Bridge không tìm thấy slave phù hợp, không phát PSELx nào, và trả về BRESP=DECERR. Nếu BIR.DecErrEn=1, tín hiệu DecErrIntr được assert sau 1 chu kỳ để báo interrupt.



Hình 3.42: Sequence diagram luồng Decode Error và DecErrIntr

Slave Monitor phát hiện decode error (PADDR!=0, psel_count!=1, PENABLE=0) và lưu pending_err_tr với is_err_vld=1. Chu kỳ tiếp theo, Monitor đọc DecErrIntr đã ổn định và gán vào pending_err_tr trước khi publish. Độ trễ một chu kỳ này phản ánh đúng timing thực tế của DUT.

Scoreboard xác minh: Scoreboard nhận cả AXI transaction (BRESP=DECERR) lẫn APB Slave transaction (decerr=1 nếu DecErrEn=1) và so sánh chéo: BRESP phải là DECERR, DecErrIntr phải nhất quán với BIR.DecErrEn trong snapshot, và BIR.DecErrSt phải được predict sang 1.

3.3.5 Thiết kế Testcase

Sau khi hoàn tất việc xây dựng và kết nối môi trường kiểm chứng, bước tiếp theo là phát triển các sequence và testcase nhằm tạo stimulus cho DUT. Các sequence được thiết kế để tái sử dụng giữa nhiều testcase khác nhau, trong khi testcase chịu trách nhiệm cấu hình môi trường và lựa chọn các kịch bản kiểm tra phù hợp. Để bảo đảm quá trình kiểm chứng có hệ thống và bám sát Verification Plan, các testcase được phân thành các nhóm chức năng tương ứng với từng đặc tính cần kiểm chứng của thiết kế.

3.3.5.1 Kiểm tra phân hệ thanh ghi cấu hình nội bộ (Register Verification)

Dưới đây là bảng 3.9 đặc tả chi tiết kịch bản kiểm thử phân hệ thanh ghi cấu hình nội bộ. Nhóm kịch bản này tập trung vào việc xác thực tính toàn vẹn dữ liệu, khả năng truy cập linh hoạt (Word, Byte) và tính năng bảo vệ các vùng nhớ dự phòng của khối APB_REGISTER trước khi các luồng dữ liệu chính được phép lưu thông qua cầu nối.

Bảng 3.9: Kịch bản kiểm tra thanh ghi cấu hình nội bộ

Thành phần kiểm thử	Nội dung kiểm thử chi tiết	Tiêu chí đánh giá thành công (PASS)
Xác thực khối APB_REGISTER	Thực hiện chuỗi giao dịch Ghi/Đọc ngẫu nhiên thông qua giao diện cấu hình (APB Master) vào các thanh ghi: BAMS0, BAMS1, BAMS2 và BIR. Quá trình kiểm tra bao gồm: – Truy cập theo Word. – Truy cập từng Byte đơn lẻ (Byte Access). – Cố tình ghi vào các vùng địa chỉ dự phòng	– Giá trị đọc ra từ các trường thanh ghi có thuộc tính Read/Write phải trùng khớp 100% với giá trị đã ghi. – Các vùng bit dự phòng (Reserved) phải giữ nguyên giá trị mặc định bằng 0, không bị ảnh hưởng bởi lệnh ghi trái phép.

3.3.5.2 Kiểm tra logic giải mã và tính toán địa chỉ AXI Burst

Bảng 3.10 dưới đây mô tả kịch bản kiểm tra dòng traffic chính của giao thức AXI4 nhằm xác thực độ chính xác của khối ADDR_DECODER. Kịch bản này ép phần cứng phải tính toán chính xác địa chỉ cho từng beat dữ liệu tương ứng với cả 3 chế độ Burst.

Bảng 3.10: Kịch bản kiểm tra logic giải mã và tính toán địa chỉ AXI Burst

Chế độ Burst (AXI)	Nội dung kiểm thử chi tiết	Tiêu chí đánh giá thành công (PASS)
Kiểu FIXED	Địa chỉ gốc giữ nguyên qua mọi beat. Thực hiện kiểm tra bộ strobe dữ liệu bằng cách phátWSTRB ngẫu nhiên (từ Byte, Half-word đến Word) để xác nhận tín hiệu PSTRB tương ứng trên bus APB dịch mã chính xác.	– Với mỗi beat dữ liệu tại APB Monitor, địa chỉ và tín hiệu chọn Slave (PSELx) phải khớp chính xác tuyệt đối với mô hình toán học giải mã địa chỉ. – Toàn bộ các điểm bao phủ trong Covergroup của Strobe và Burst phải được lấp đầy (Hit 100%).
Kiểu INCR	Kiểm tra mạch cộng tự động tăng địa chỉ tuần tự, đảm bảo địa chỉ tăng theo đúng kích thước (Size) của transaction.	<i>(Đồng thời thỏa mãn các tiêu chí bao phủ địa chỉ như trên)</i>
Kiểu WRAP	Kích hoạt các transaction lệch ranh giới cache để kiểm tra logic tính toán biên trên (wrap_upper) và biên dưới (wrap_lower), ép phần cứng thực hiện cuộn chiếu địa chỉ quay vòng một cách chính xác.	<i>(Đồng thời thỏa mãn các tiêu chí bao phủ địa chỉ như trên)</i>

3.3.5.3 Kiểm tra cơ chế cấu hình lại bản đồ địa chỉ trong thời gian chạy

Bảng 3.11 dưới đây chi tiết hóa kịch bản xác thực tính năng "Runtime Configurable" — một trong những điểm mạnh cốt lõi của thiết kế. Kịch bản được thiết lập theo chuỗi hành vi động để chứng minh khả năng tái lập bản đồ địa chỉ tức thời mà không gây xung đột hệ thống.

Bảng 3.11: Kịch bản kiểm tra cơ chế cấu hình lại bản đồ địa chỉ động

Bước thực hiện	Nội dung kiểm thử chi tiết	Tiêu chí đánh giá thành công (PASS)
1. Trạng thái ổn định	Hệ thống đang truyền nhận dữ liệu ổn định vào vùng địa chỉ mặc định của Slave 0.	– Ngay sau chu kỳ thanh ghi cấu hình cập nhật thành công (khi hệ thống ở trạng thái IDLE), bản đồ địa chỉ mới phải có hiệu lực tức thì. – Giao dịch AXI trở vào địa chỉ mới phải kích hoạt đúng $PSEL0 = 1$. – Nếu cố tình truy cập vào vùng địa chỉ cũ, hệ thống phải từ chối giải mã để bảo vệ an toàn dữ liệu.
2. Cấu hình lại	Thực hiện lệnh ghi Frontdoor thay đổi giá trị cấu hình trong thanh ghi BAMS0, dịch chuyển không gian địa chỉ của Slave 0 sang một vùng nhớ mới.	<i>(Kết quả được xác thực ở Bước 3)</i>
3. Kiểm tra đáp ứng	Ngay lập tức, phát transaction AXI vào vùng địa chỉ mới để kiểm tra đáp ứng của phần cứng cầu nối.	<i>(Đánh giá bằng tiêu chí hệ thống ở Bước 1)</i>

3.3.5.4 Kiểm tra cơ chế quản lý ngắt và lỗi giải mã (DECERR)

Dưới đây là bảng 3.12 kịch bản sử dụng kỹ thuật tiêm lỗi để đánh giá khả năng tự bảo vệ và phản ứng an toàn của thiết kế khi hệ thống xảy ra sự cố cố tình truy cập vào không gian ngoại vi bất hợp pháp.

Bảng 3.12: Kịch bản kiểm tra cơ chế quản lý ngắt và lỗi giải mã

Trường hợp kiểm thử	Nội dung kiểm thử chi tiết	Tiêu chí đánh giá thành công (PASS)
Bật ngắt (BIR.DecErrEn = 1)	Cho AXI Master VIP phát transaction vào vùng địa chỉ không tồn tại trên bản đồ. Kiểm tra hệ thống có trả về mã lỗi BRESP/RRESP = DECERR cho Master, kéo tín hiệu ngắt phần cứng DecErrIntr lên mức cao và set bit sticky BIR.DecErrSt hay không. Tiếp tục thử nghiệm xóa ngắt bằng cơ chế Write-1-to-Clear (W1C).	- Mạch logic phải phân tách rõ ràng giữa việc báo lỗi trên bus hệ thống và phát tín hiệu ngắt phần cứng độc lập. - Cơ chế xóa ngắt W1C hoạt động chuẩn xác: chỉ xóa trạng thái ngắt khi ghi bit 1, ghi bit 0 giữ nguyên trạng thái hiện tại.
Tắt ngắt (BIR.DecErrEn = 0)	Lặp lại việc truy cập sai địa chỉ vào vùng địa chỉ không tồn tại Xác thực xem tín hiệu ngắt phần cứng DecErrIntr giữ ở mức thấp trong khi phản hồi lỗi DECERR trên bus vẫn hoạt động bình thường hay không.	<i>(Áp dụng tiêu chí phân tách logic báo lỗi và ngắt cứng như trên)</i>

3.3.5.5 Kiểm tra truyền dẫn phản hồi lỗi từ Slave ngoại vi

Bảng 3.13 dưới đây thể hiện kịch bản kiểm tra khi các Slave ngoại vi phía APB gặp lỗi vận hành và chuyển tín hiệu báo lỗi (SLVERR) đồng bộ ngược dòng về phía Master hệ thống.

Bảng 3.13: Kịch bản kiểm tra khả năng truyền dẫn phản hồi lỗi ngoại vi

Thành phần mô phỏng	Nội dung kiểm thử chi tiết	Tiêu chí đánh giá thành công (PASS)
APB Slave lỗi & Cầu nối chuyển đổi	Cấu hình APB Slave VIP chủ động kéo tích cực tín hiệu PSLVERR = 1 trong chu kỳ phản hồi dữ liệu nhằm giả lập một lỗi vận hành của ngoại vi (ví dụ: ngoại vi bị timeout hoặc lỗi chặn lẻ dữ liệu).	- Cầu nối AXI-to-APB phải bắt được trạng thái lỗi bất đồng bộ này. - Thực hiện đóng gói transaction và chuyển đổi đồng bộ ngược về kênh phản hồi của AXI Master dưới dạng: BRESP = SLVERR (luồng ghi) hoặc RRESP = SLVERR (luồng đọc). - Tuyệt đối không làm treo bus hệ thống trong suốt quá trình báo lỗi.

CHƯƠNG 4: KẾT QUẢ MÔ PHỎNG VÀ KIỂM CHỨNG

4.1 KẾT QUẢ KIỂM CHỨNG MỨC GIAO THỨC

Trước khi phân tích kết quả kiểm chứng theo Vplan, phần này trình bày các dạng sóng mô phỏng điển hình thu thập trong quá trình chạy testcase, nhằm cung cấp bằng chứng trực quan rằng thiết kế tuân thủ đúng đặc tả giao thức AXI4 và APB4. Việc đối chiếu dạng sóng thực tế với dạng sóng chuẩn là một trong những phương pháp kiểm chứng quan trọng nhất trong thiết kế phần cứng số, vì nó cho phép phát hiện các vi phạm về thời gian và trình tự tín hiệu mà kiểm tra chức năng thuần túy có thể bỏ sót. Các dạng sóng sau được chọn lọc từ các kịch bản đại diện cho từng giao thức, đồng thời liên kết trực tiếp với các lỗi được phân tích tại mục 4.4. Các yêu cầu tuân thủ giao thức không được đưa thành các hạng mục riêng trong Verification Plan do đã được kiểm tra tự động bằng hệ thống SVA protocol checker tích hợp trong môi trường kiểm chứng; vì vậy, các dạng sóng dưới đây chủ yếu đóng vai trò minh họa và xác nhận trực quan cho kết quả kiểm tra giao thức.

Bảng 4.1: Cấu hình môi trường mô phỏng

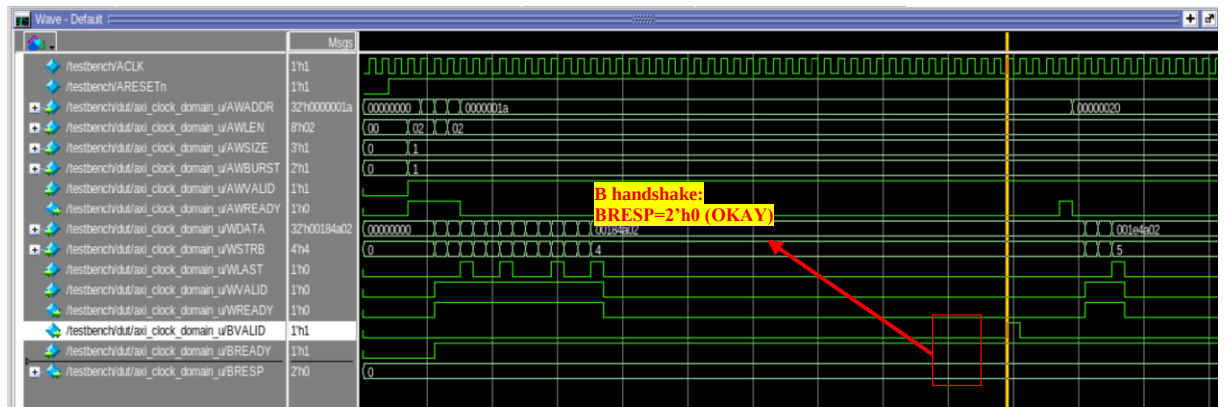
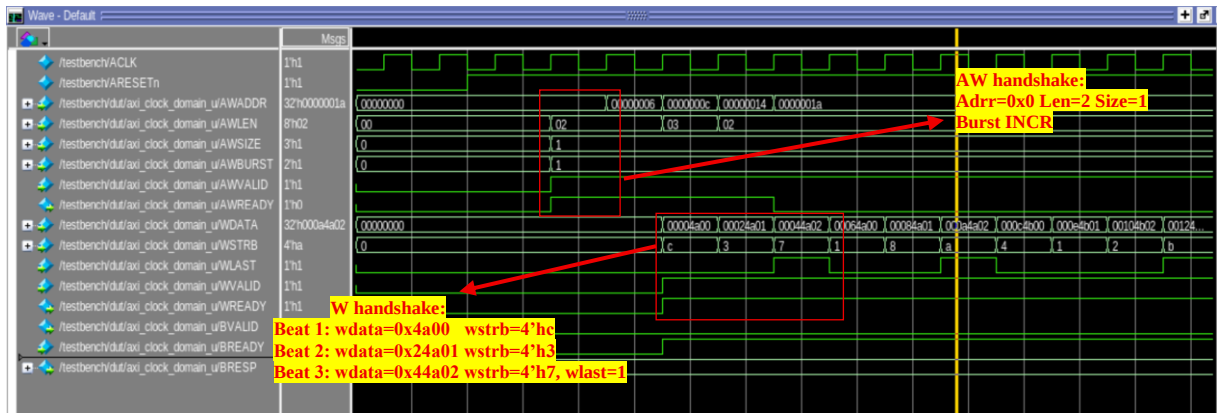
Thành phần	Thông tin
Ngôn ngữ mô phỏng	SystemVerilog (IEEE 1800-2012)
Phương pháp kiểm chứng	UVM 1.2 (Universal Verification Methodology)
Tần số ACLK (miền AXI)	100 MHz – chu kỳ 10 ns
Tần số PCLK (miền APB)	50 MHz – chu kỳ 20 ns
Tỷ lệ tần số ACLK : PCLK	2 : 1
Thời gian reset	20 ns (ARESETn và PRESETn deassert đồng thời)
Tổng số testcase	70 testcase (6 nhóm theo VPlan)
Loại testcase	Directed testcase + Constrained-random testcase

Công cụ Coverage	Functional Coverage (UVM covergroup) + Code Coverage
------------------	--

4.1.1 AXI Write Protocol

Hình 4.1 trình bày dạng sóng của một giao dịch ghi burst kiểu INCR theo đặc tả AXI4, với độ dài burst ba beat (AWLEN=2) và kích cỡ truyền hai byte mỗi beat (AWSIZE=1) với địa chỉ bắt đầu là 0x0, 0x6. Cần lưu ý rằng giao dịch ghi AXI4 sử dụng ba kênh riêng biệt hoạt động bất đồng bộ với nhau, phản ánh triết lý thiết kế đặc trưng của kiến trúc AXI so với APB.

Kênh địa chỉ ghi (AW) thực hiện handshake một chu kỳ: Master assert AWVALID=1 cùng với địa chỉ AWADDR và thông tin burst (như với địa chỉ bắt đầu là 0x0, kiểu burst là INCR (AWBURST=1), độ dài 3 beat (AWLEN=2), kích cỡ mỗi beat là 2 byte (AWSIZE=1)), sau đó duy trì ổn định cho đến khi Slave xác nhận bằng AWREADY=1. Kênh dữ liệu ghi (W) độc lập với kênh AW - Master có thể bắt đầu gửi dữ liệu sau hoặc đồng thời với địa chỉ. Mỗi beat dữ liệu được đánh dấu bằng tổ hợp WVALID/WREADY, trong đó tín hiệu WLAST=1 chỉ được assert duy nhất tại beat cuối cùng. Nhìn hình 4.1, ta thấy giao dịch đầu tiên ở kênh W có 3 beat dữ liệu là: WDATA=0x4a00 với WSTRB=0xc; WDATA=0x24a01 với WSTRB=3; WDATA=0x44a02 với WSTRB=7 và WLAST lên 1 tại beat cuối cùng này. Phù hợp với thông tin kênh AW đã gửi đi với giao dịch đầu tiên (AWLEN=2). Sau một khoảng thời gian, kênh phản hồi ghi (B) trả về mã BRESP=OKAY (BRESP=0) sau khi toàn bộ dữ liệu đã được chuyển thành công sang phía APB, hoàn tất vòng đời của giao dịch.

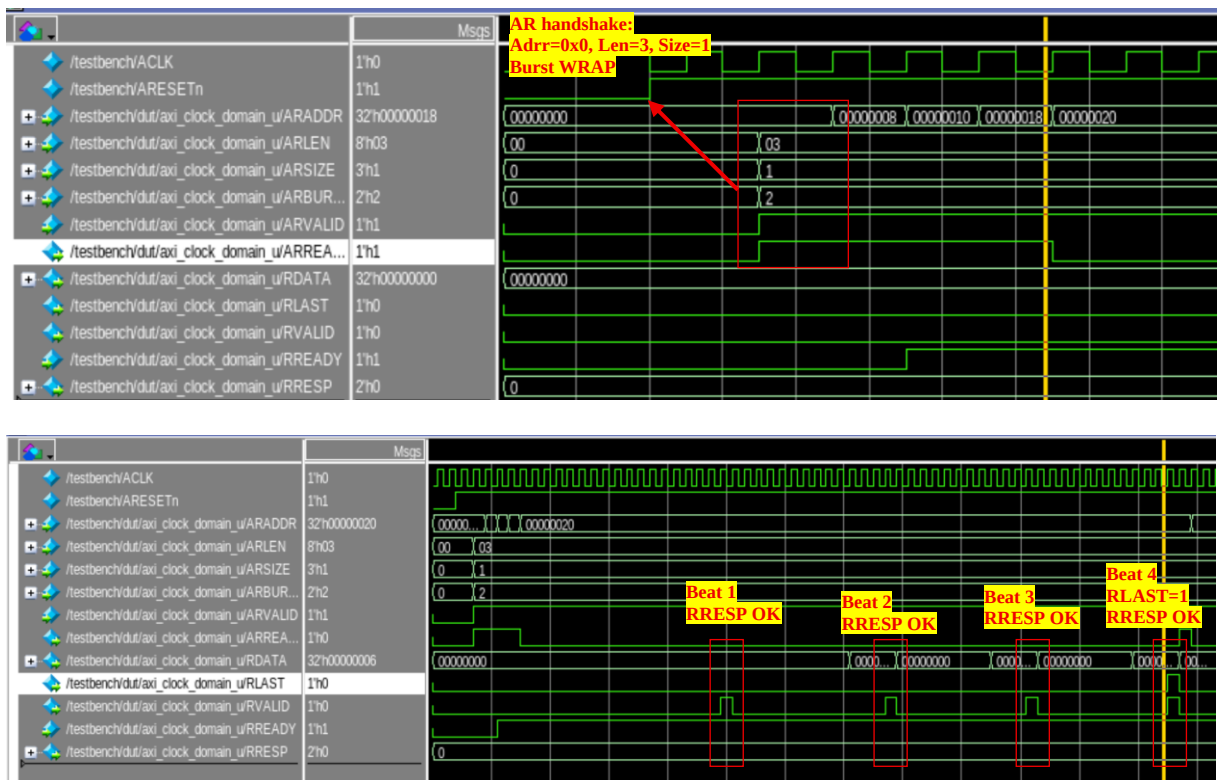


Hình 4.1: Dạng sóng AXI4 Write Burst (INCR, len=3, size=2Byte) – giao thức đúng

4.1.2 AXI Read Protocol

Hình 4.2 trình bày dạng sóng của giao dịch đọc AXI4 burst WRAP, khác với giao dịch ghi, đọc AXI4 chỉ có hai kênh hoạt động: kênh địa chỉ đọc (AR) và kênh dữ liệu đọc (R). Sau khi handshake AR hoàn tất bằng cặp ARVALID/ARREADY, bridge lần lượt trả về từng beat dữ liệu RDATA kèm mã phản hồi RRESP tương ứng trên kênh R.

Tín hiệu RLAST=1 có vai trò đặc biệt quan trọng trong giao thức đọc: nó báo hiệu cho Master AXI rằng beat hiện tại là beat cuối cùng của burst và giao dịch đọc sắp kết thúc. Theo đặc tả AXI4, RLAST phải được assert chính xác tại và chỉ tại beat cuối, không sớm hơn và không muộn hơn.

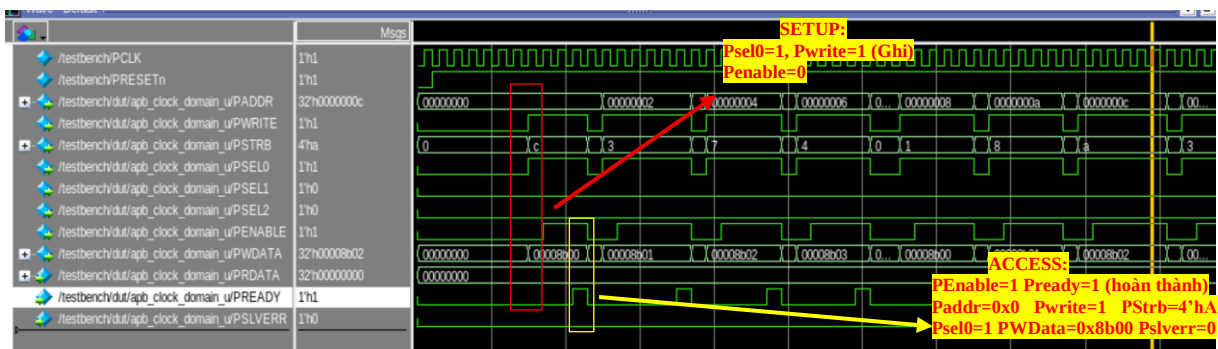


Hình 4.2: Dạng sóng AXI4 Read Burst (WRAP, len=3, size=2Byte) – giao thức đúng

4.1.3 APB Write Protocol

Hình 4.3 trình bày dạng sóng của một giao dịch ghi APB4 với wait state, minh họa cấu trúc hai pha đặc trưng của giao thức APB. Khác biệt cơ bản so với AXI là APB sử dụng bus chia sẻ tuần tự: toàn bộ địa chỉ, dữ liệu và tín hiệu điều khiển được drive đồng thời trên cùng một tập tín hiệu, và chỉ có thể thực hiện một giao dịch tại một thời điểm.

Pha SETUP kéo dài đúng một chu kỳ PCLK, trong đó bridge assert PSELx để chọn slave đích, đồng thời drive PADDR, PWDATA, PWRITE và PSTRB ổn định lên bus nhưng giữ PENABLE=0. Sang pha ACCESS, bridge assert PENABLE=1 và duy trì nguyên xi toàn bộ tín hiệu địa chỉ và dữ liệu. APB Slave có quyền kéo dài giao dịch bằng cách giữ PREADY=0 (wait state). Giao dịch chỉ hoàn tất khi slave assert PREADY=1. Trong thiết kế này, APB Slave Driver tạo ngẫu nhiên từ một đến ba chu kỳ wait state để kiểm tra tính ổn định của bridge dưới điều kiện slave có độ trễ biến đổi.

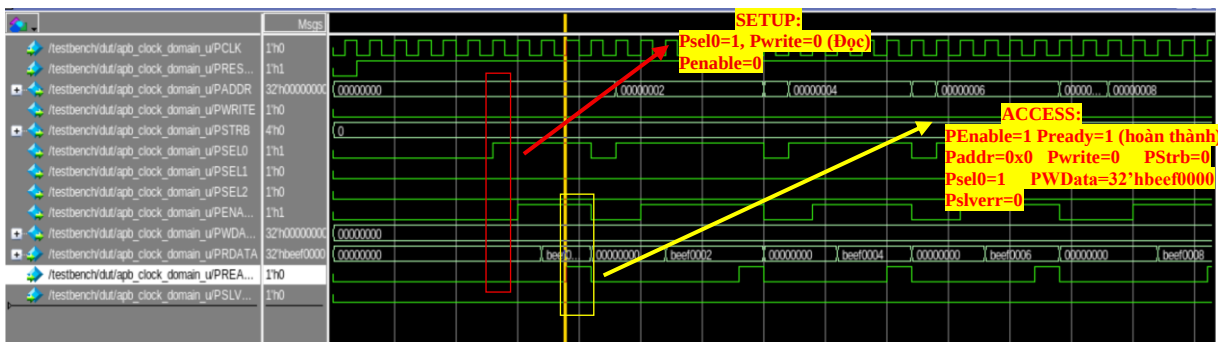


Hình 4.3. Dạng sóng APB4 Write Transaction với Wait States – giao thức đúng

4.1.4 APB Read Protocol

Hình 4.4 trình bày giao dịch đọc APB4 với cơ chế tương tự nhưng có sự khác biệt về hướng dữ liệu. Trong pha ACCESS của giao dịch đọc, APB Slave có trách nhiệm drive dữ liệu lên tín hiệu PRDATA và duy trì giá trị này ổn định trong suốt toàn bộ giai đoạn chờ cho đến khi assert PREADY=1. Bridge capture PRDATA tại đúng cạnh clock khi cả PENABLE và PREADY đồng thời bằng 1.

Yêu cầu về sự ổn định của PRDATA trong toàn bộ pha ACCESS là một trong những điều kiện nghiêm ngặt nhất của giao thức APB4. Bất kỳ sự thay đổi nào của PRDATA trong khi PENABLE=1 và PREADY=0 đều là vi phạm giao thức và có thể dẫn đến bridge capture sai dữ liệu. Yêu cầu này được kiểm tra thông qua các SVA được định nghĩa trong interface APB Slave của môi trường kiểm chứng, và kết quả xác nhận rằng thiết kế tuân thủ đúng yêu cầu này trong tất cả các kịch bản đã kiểm tra.

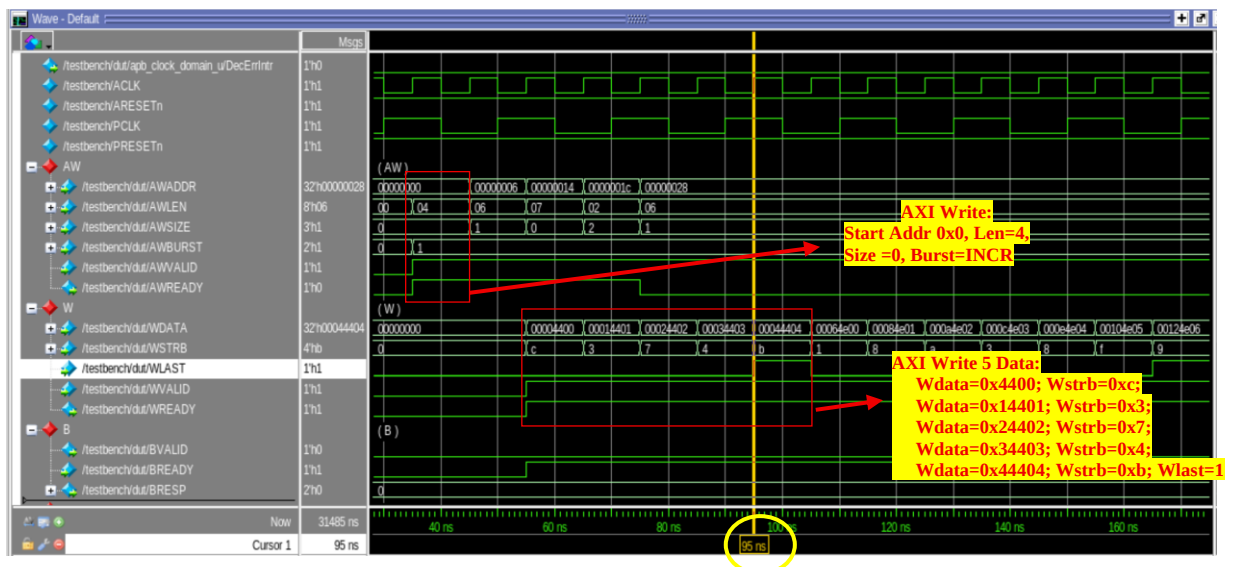


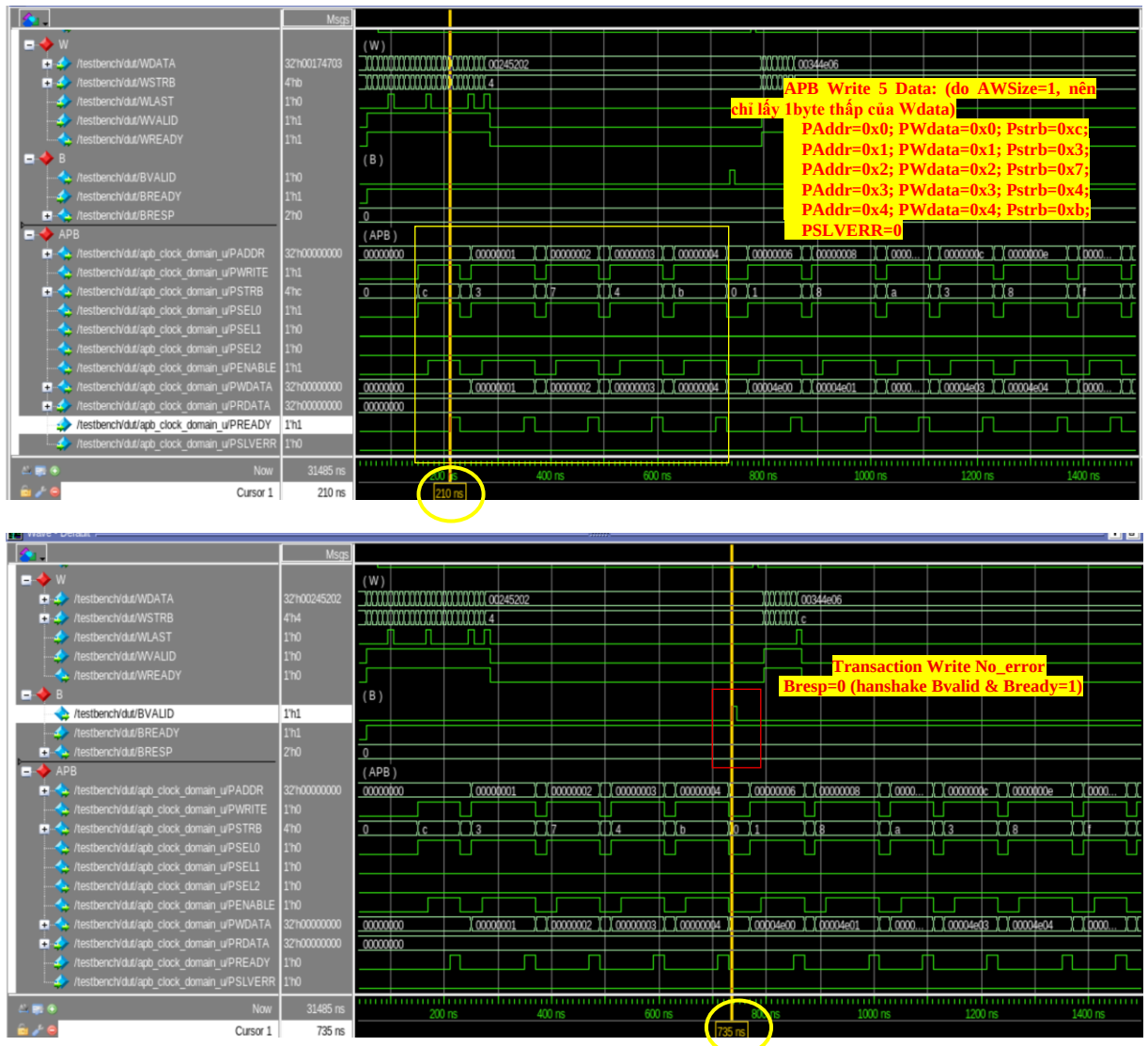
Hình 4.4. Dạng sóng APB4 Read Transaction với Wait States – giao thức đúng

4.1.5 AXI-to-APB Transaction Flow

Hình 4.5 trình bày dạng sóng tổng hợp đặt song song cả hai miền tín hiệu AXI (tần số 100 MHz) và APB (tần số 50 MHz), minh họa toàn bộ luồng xử lý một giao dịch ghi nhiều beat từ khi Master AXI phát lệnh cho đến khi nhận được BRESP. Dạng sóng này phơi bày một trong những thách thức thiết kế phức tạp nhất của cầu nối: vấn đề đồng bộ hóa tín hiệu giữa hai miền xung nhịp không đồng bộ.

Có thể quan sát rõ sự lệch pha về thời gian: giao dịch AXI ở phía trên hoàn tất handshake và gửi dữ liệu trong khoảng thời gian 95ns, thì phía APB nhận được dữ liệu và vào pha SETUP khoảng 160ns, và hoàn thành giao dịch ở 210ns. BRESP được trả về khá lâu ở khoảng 735ns, BRESP chỉ được trả về sau khi PREADY=1 từ phía APB, nghĩa là thông tin về kết quả giao dịch phải truyền ngược từ miền chậm sang miền nhanh.





Hình 4.5. Dạng sóng chuyển đổi giao dịch AXI4–APB4 qua Bridge

4.2 KẾT QUẢ KIỂM CHỨNG THEO VERIFICATION PLAN

4.2.1 Nhóm A – Kiểm tra thanh ghi cấu hình APB

Nhóm này tập trung kiểm chứng chức năng của các thanh ghi cấu hình nội bộ thông qua mô hình RAL. Các testcase bao phủ giá trị mặc định sau reset, quyền truy cập đọc/ghi, vùng địa chỉ reserved và cơ chế byte access. Kết quả kiểm chứng được trình bày trong Bảng 4.2.

Bảng 4.2: Kết quả kiểm chứng nhóm A – Thanh ghi cấu hình APB

STT	Tên Testcase	Mô tả kiểm chứng	Kết quả
1	tc11_read_default	– Đọc giá trị mặc định của BAMS0, BAMS1, BAMS2 và BIR sau reset – So sánh với giá trị reset trong RAL model	PASS
2	tc12_read_write_reserved	– Ghi dữ liệu vào các địa chỉ reserved (0x14-0xFF) – Đọc lại dữ liệu sau ghi – Kỳ vọng luôn nhận giá trị 0x0000_0000	PASS
3	tc13_bams_read_write_access	– Thực hiện 20 vòng ghi/đọc ngẫu nhiên cho các thanh ghi BAM – So sánh dữ liệu bằng UVM_CHECK và RAL mirror	PASS
4	tc14_bir_read_write_access	– Kiểm tra cơ chế W1C của bit DecErrSt – Kiểm tra quyền truy cập RW của bit DecErrEn – Kiểm tra các bit reserved [31:2] luôn đọc về 0	PASS
5	tc15_bams_byte_access	– Ghi từng byte thông qua tín hiệu pstrb_reg – Kiểm tra chỉ các byte được chọn mới thay đổi và xác nhận các byte còn lại giữ nguyên giá trị	PASS

4.2.2 Nhóm B – Kiểm tra giao dịch ghi AXI

Nhóm B kiểm chứng đường dữ liệu ghi - luồng truyền thông tin từ AXI Master xuyên qua bridge đến APB Slave. Nhóm có không gian kịch bản giao dịch ghi AXI rất rộng: ba kiểu burst, ba kích cỡ dữ liệu, ba slave đích, điều kiện có và không có lỗi slave, và các trường hợp đặc biệt khi giao dịch vượt ranh giới giữa hai slave. Tất cả các chiều này được tổ hợp một cách có hệ thống thay vì ngẫu nhiên, đảm bảo mọi tổ hợp quan trọng đều được kích hoạt ít nhất một lần.

Ngoài ba kiểu burst, nhóm B còn kiểm tra đồng thời ba kích cỡ truyền dữ liệu (1 byte, 2 byte, 4 byte), ba slave đích, và điều kiện lỗi PSLVERR. Các testcase crossing kiểm tra trường hợp burst vắt qua ranh giới giữa hai slave, buộc bridge phải tự động chuyển đổi tín hiệu PSEL giữa chừng. Scoreboard xác minh PSEL, PADDR, PWDATA, PSTRB và BRESP cho từng beat. Kết quả được trình bày tại Bảng 4.3.

Bảng 4.3: Kết quả kiểm chứng nhóm B – Giao dịch ghi AXI

STT	Tên Testcase	Mô tả kiểm chứng	Kết quả
1	tc21_psel0_fixed_wr	FIXED burst ghi Slave 0, địa chỉ 0x0000–0x0FFF, kích cỡ 1–4 byte, 1–2 beat. Xác minh PSEL0, PADDR, PWDATA và BRESP=OKAY.	PASS
2	tc22_psel1_fixed_wr	FIXED burst ghi Slave 1, địa chỉ 0x1000–0x1FFF. Xác minh PSEL1 được chọn đúng.	PASS
3	tc23_psel2_fixed_wr	FIXED burst ghi Slave 2, địa chỉ 0x2000–0x2FFF. Xác minh PSEL2 được chọn đúng.	PASS
4	tc24_psel0_fixed_wr_err	FIXED burst ghi Slave 0 với PSLVERR=1. Xác minh BRESP=SLVERR. Testcase phát hiện lỗi đồng bộ hóa BRESP (Bug #3).	PASS
5	tc25_psel1_fixed_wr_err	FIXED burst ghi Slave 1 với PSLVERR=1. Xác minh BRESP=SLVERR.	PASS
6	tc26_psel2_fixed_wr_err	FIXED burst ghi Slave 2 với PSLVERR=1. Xác minh BRESP=SLVERR.	PASS
7	tc27_pselx_fixed_wr_random	FIXED burst ghi với địa chỉ, kích cỡ, độ dài ngẫu nhiên. Scoreboard tự tính slave đích từ BAMS snapshot.	PASS
8	tc31_psel0_incr_wr	INCR burst ghi Slave 0, tới 8 beat.	PASS

		Xác minh địa chỉ APB tăng đúng theo kích cỡ mỗi beat.	
9	tc32_psel1_incr_wr	INCR burst ghi Slave 1, địa chỉ 0x1000–0x1FFF.	PASS
10	tc33_psel2_incr_wr	INCR burst ghi Slave 2, địa chỉ 0x2000–0x2FFF.	PASS
11	tc34_psel0_incr_wr_err	INCR 8 beat ghi Slave 0, kích cỡ 4 byte, PSLVERR=1. Xác minh BRESP=SLVERR.	PASS
12	tc35_psel1_incr_wr_err	INCR 8 beat ghi Slave 1 với PSLVERR=1.	PASS
13	tc36_psel2_incr_wr_err	INCR 8 beat ghi Slave 2 với PSLVERR=1.	PASS
14	tc37_psel01_incr_wr_cross	INCR crossing Slave 0 sang Slave 1. Xác minh PSEL chuyển đúng tại ranh giới 0x1000.	PASS
15	tc38_psel12_incr_wr_cross	INCR crossing Slave 1 sang Slave 2 tại ranh giới 0x2000.	PASS
16	tc39_pselx_incr_wr_random	INCR burst ghi ngẫu nhiên. Scoreboard tự tính PSEL từng beat kể cả crossing và decode error.	PASS
17	tc41_psel0_wrap_wr	WRAP burst ghi Slave 0. Xác minh địa chỉ wrap-around đúng theo ranh giới căn chỉnh.	PASS
18	tc42_psel1_wrap_wr	WRAP burst ghi Slave 1. Xác minh wrap-around.	PASS
19	tc43_psel2_wrap_wr	WRAP burst ghi Slave 2. Xác minh wrap-around.	PASS
20	tc44_psel0_wrap_wr_err	WRAP 8 beat ghi Slave 0 với PSLVERR=1. Xác minh BRESP=SLVERR.	PASS
21	tc45_psel1_wrap_wr_err	WRAP ghi Slave 1 với PSLVERR=1.	PASS
22	tc46_psel2_wrap_wr_err	WRAP ghi Slave 2 với PSLVERR=1.	PASS

23	tc47_psel01_wrap_wr_cross	WRAP crossing Slave 0 sang Slave 1. Xác minh đồng thời wrap-around và chuyển PSEL.	PASS
24	tc48_psel12_wrap_wr_cross	WRAP crossing Slave 1 sang Slave 2.	PASS
25	tc49_pselx_wrap_wr_random	WRAP burst ghi ngẫu nhiên. Scoreboard tự tính wrap address và PSEL.	PASS
26	tc27_decerr_wr	Ghi đến địa chỉ ngoài tất cả vùng slave hợp lệ. Xác minh BRESP=DECERR.	PASS
27	tc_random_wr_stress	Stress test với tất cả tham số ngẫu nhiên, bao gồm cả vùng hợp lệ lẫn không hợp lệ.	PASS

4.2.3 Nhóm C – Kiểm tra giao dịch đọc AXI

Nhóm C là đối xứng với nhóm B về cấu trúc testcase nhưng kiểm chứng đường dữ liệu đọc — chiều truyền từ APB Slave ngược về AXI Master. Mặc dù đối xứng về số lượng và phạm vi, nhóm C đặt ra những thách thức kiểm chứng khác biệt vì giao dịch đọc AXI có những yêu cầu chặt chẽ hơn so với giao dịch ghi.

Việc RLAST sai vị trí là một lỗi giao thức nghiêm trọng vì phía Master AXI dùng tín hiệu này để đếm số beat trong burst đọc. Trong quá trình kiểm chứng nhóm C, testcase phát hiện Bug #4 liên quan đến logic xử lý RLAST khi handshake kéo dài nhiều chu kỳ. Kết quả được trình bày tại Bảng 4.4.

Bảng 4.4: Kết quả kiểm chứng nhóm C – Giao dịch đọc AXI

STT	Tên Testcase	Mô tả kiểm chứng	Kết quả
1	tc51_psel0_fixed_rd	FIXED burst đọc Slave 0. Xác minh PSEL0, PADDR, RDATA khớp PRDATA, RRESP=OKAY và RLAST đúng tại beat cuối.	PASS
2	tc52_psel1_fixed_rd	FIXED burst đọc Slave 1. Xác	PASS

		minh PSEL1 và RDATA.	
3	tc53_psel2_fixed_rd	FIXED burst đọc Slave 2. Xác minh PSEL2 và RDATA.	PASS
4	tc54_psel0_fixed_rd_err	FIXED burst đọc Slave 0 với PSLVERR=1. Xác minh RRESP=SLVERR, RDATA không kiểm tra khi slave báo lỗi.	PASS
5	tc55_psel1_fixed_rd_err	FIXED burst đọc Slave 1 với PSLVERR=1. Xác minh RRESP=SLVERR.	PASS
6	tc56_psel2_fixed_rd_err	FIXED burst đọc Slave 2 với PSLVERR=1. Xác minh RRESP=SLVERR.	PASS
7	tc57_pselx_fixed_rd_random	FIXED burst đọc ngẫu nhiên. Scoreboard tự xác định slave và xác minh RDATA, RRESP.	PASS
8	tc61_psel0_incr_rd	INCR burst đọc Slave 0 tới 8 beat. Xác minh địa chỉ tuyến tính, RDATA từng beat, RLAST đúng vị trí.	PASS
9	tc62_psel1_incr_rd	INCR burst đọc Slave 1.	PASS
10	tc63_psel2_incr_rd	INCR burst đọc Slave 2.	PASS
11	tc64_psel0_incr_rd_err	INCR 8 beat đọc Slave 0 với PSLVERR=1. Xác minh toàn bộ 8 RRESP=SLVERR.	PASS
12	tc65_psel1_incr_rd_err	INCR burst đọc Slave 1 với PSLVERR=1.	PASS
13	tc66_psel2_incr_rd_err	INCR burst đọc Slave 2 với PSLVERR=1.	PASS
14	tc67_psel01_incr_rd_cross	INCR crossing Slave 0 sang Slave 1. Xác minh PSEL chuyển đúng và RDATA từ đúng slave.	PASS
15	tc68_psel12_incr_rd_cross	INCR crossing Slave 1 sang Slave 2.	PASS
16	tc69_pselx_incr_rd_random	INCR burst đọc ngẫu nhiên. Scoreboard tự xác minh RDATA,	PASS

		RRESP từng beat.	
17	tc71_psel0_wrap_rd	WRAP burst đọc Slave 0. Xác minh wrap-around, RDATA và RLAST.	PASS
18	tc72_psel1_wrap_rd	WRAP burst đọc Slave 1.	PASS
19	tc73_psel2_wrap_rd	WRAP burst đọc Slave 2.	PASS
20	tc74_psel0_wrap_rd_err	WRAP 8 beat đọc Slave 0 với PSLVERR=1. Xác minh RRESP=SLVERR.	PASS
21	tc75_psel1_wrap_rd_err	WRAP burst đọc Slave 1 với PSLVERR=1.	PASS
22	tc76_psel2_wrap_rd_err	WRAP burst đọc Slave 2 với PSLVERR=1.	PASS
23	tc77_psel01_wrap_rd_cross	WRAP crossing Slave 0 sang Slave 1.	PASS
24	tc78_psel12_wrap_rd_cross	WRAP crossing Slave 1 sang Slave 2.	PASS
25	tc79_pselx_wrap_rd_random	WRAP burst đọc ngẫu nhiên. Scoreboard tự tính wrap address, xác minh RDATA và RRESP.	PASS
26	tc_decerr_rd	Đọc từ địa chỉ ngoài tất cả vùng slave. Xác minh RRESP=DECERR toàn bộ beat.	PASS
27	tc_random_rd_stress	Stress test đọc với địa chỉ và tham số hoàn toàn ngẫu nhiên.	PASS

4.2.4 Nhóm D – Kiểm tra stress ngẫu nhiên kết hợp đọc và ghi

Nhóm D đại diện cho phương pháp kiểm chứng bổ sung không thể thay thế bằng directed testing: kiểm tra ngẫu nhiên có ràng buộc. Trong khi các testcase của nhóm B và C được thiết kế với kịch bản xác định nhằm kiểm tra từng hành vi cụ thể, testcase của nhóm D ngẫu nhiên hóa đồng thời tất cả tham số có thể biến đổi: địa chỉ, kiểu burst, kích cỡ dữ liệu, độ dài burst, hướng giao dịch và trạng thái lỗi của slave.

Bảng 4.5: Kết quả kiểm chứng nhóm D – Stress ngẫu nhiên đọc/ghi

STT	Tên Testcase	Mô tả kiểm chứng	Kết quả
1	tc81_wr_rd_random_test	Ngẫu nhiên hóa hoàn toàn cả giao dịch ghi lẫn đọc trong cùng phiên chạy, bao gồm địa chỉ, kiểu burst, kích cỡ, độ dài và điều kiện lỗi PSLVERR. Scoreboard kiểm tra đồng thời tất cả giao dịch.	PASS

4.2.5 Nhóm E – Kiểm tra cơ chế ngắt lỗi giải mã địa chỉ

Cơ chế Decode Error Interrupt là một tính năng an toàn của thiết kế: khi bridge nhận yêu cầu truy cập đến địa chỉ không thuộc vùng quản lý của bất kỳ APB Slave nào, ngoài việc trả về mã lỗi DECERR cho Master AXI, bridge còn có khả năng phát tín hiệu ngắt để thông báo cho hệ thống về sự kiện này. Tính năng này đặc biệt quan trọng trong các hệ thống nhúng vì nó cho phép phần mềm điều hành phát hiện và xử lý các truy cập địa chỉ sai do lỗi phần mềm.

Bảng 4.6: Kết quả kiểm chứng nhóm E – Cơ chế ngắt Decode Error

STT	Tên Testcase	Mô tả kiểm chứng	Kết quả
1	tc91_enable_decerr_interrupt	DecErrEn=1, gửi giao dịch đến địa chỉ không hợp lệ. Xác minh DecErrIntr assert, DecErrSt=1 và BRESP/RRESP=DECERR.	PASS
2	tc92_disable_decerr_interrupt	DecErrEn=0, gửi giao dịch đến địa chỉ không hợp lệ. Xác minh BRESP=DECERR nhưng DecErrIntr không assert và DecErrSt=0.	PASS
3	tc93_clear_decerr_interrupt	Kích hoạt ngắt, sau đó W1C vào DecErrSt. Xác minh DecErrSt=0 và DecErrIntr deassert sau khi xóa.	PASS
4	tc94_sticky_decerr_interrupt	Gửi nhiều lỗi liên tiếp không xóa giữa chừng. Xác minh DecErrSt=1 được duy trì liên tục,	PASS

		không tự reset.	
--	--	-----------------	--

4.2.6 Nhóm F – Kiểm tra cấu hình lại bản đồ địa chỉ động (Dynamic Address Remap)

Nhóm F kiểm chứng tính năng then chốt phân biệt thiết kế này với các cầu nối AXI-APB thông thường: khả năng lập trình lại bản đồ địa chỉ trong thời gian chạy. Trong các thiết kế truyền thống, vùng địa chỉ của từng APB Slave được cố định tại thời điểm tổng hợp phần cứng và không thể thay đổi sau đó. Thiết kế trong đề tài này loại bỏ ràng buộc đó bằng cách đọc bản đồ địa chỉ từ các thanh ghi BAMS tại thời điểm thực thi, cho phép phần mềm tái cấu hình toàn bộ bản đồ bất kỳ lúc nào trong quá trình vận hành.

Bảng 4.7: Kết quả kiểm chứng nhóm F – Cấu hình lại địa chỉ động

STT	Tên Testcase	Mô tả kiểm chứng	Kết quả
1	tc101_dynamic_remap_basic	Ghi BAMS mới, gửi giao dịch đến vùng địa chỉ mới. Xác minh bridge giải mã đúng theo bản đồ mới.	PASS
2	tc102_old_region_invalid	Sau remap, gửi giao dịch đến vùng địa chỉ cũ. Xác minh DECERR, không có đường truy cập ảo từ bản đồ cũ.	PASS
3	tc103_runtime_remap	Thay đổi BAMS trong khi đang có giao dịch đang xử lý. Xác minh không xảy ra deadlock hoặc giải mã sai.	PASS
4	tc104_random_remap_stress	Ngẫu nhiên hóa BAMS và phát sinh AXI traffic đồng thời. Kiểm tra ổn định bộ giải mã địa chỉ.	PASS
5	tc105_burst_cross_after_remap	Sau remap, phát sinh burst crossing qua ranh giới slave mới. Xác minh PSEL đúng theo bản đồ mới.	PASS
6	tc106_reserved_region_after_remap	Cấu hình BAMS tạo vùng trống, gửi giao dịch đến vùng đó. Xác minh DECERR nhất	PASS

		quán mọi cấu hình.	
--	--	--------------------	--

4.3 KẾT QUẢ REGRESSION

Sau khi hoàn tất phát triển và xác nhận từng testcase riêng lẻ, toàn bộ 70 testcase được chạy hồi quy không giám sát. Mục tiêu của giai đoạn này không phải là phát hiện lỗi mới mà là xác nhận tính ổn định của thiết kế sau quá trình debug: đảm bảo rằng không có lỗi thứ cấp nào được vô tình đưa vào trong quá trình sửa các lỗi đã phát hiện.

Bảng 4.8: Kết quả Regression tổng hợp theo nhóm tính năng

Nhóm tính năng	Tổng TC	PASS	FAIL	SKIP	Tỷ lệ PASS
A – APB Register Configuration	5	5	0	0	100%
B – AXI Write Transaction	27	27	0	0	100%
C – AXI Read Transaction	27	27	0	0	100%
D – AXI Random R/W	1	1	0	0	100%
E – Interrupt Mechanism	4	4	0	0	100%
F – Dynamic Address Remap	6	6	0	0	100%
Tổng cộng	70	70	0	0	100%

4.4 CÁC LỖI PHÁT HIỆN TRONG QUÁ TRÌNH KIỂM CHỨNG

Một trong những thước đo thực chất nhất về hiệu quả của một môi trường kiểm chứng là khả năng phát hiện lỗi thực sự trong thiết kế - không chỉ xác nhận rằng thiết kế hoạt động đúng trong các kịch bản bình thường, mà còn bắt được các sai sót tinh vi chỉ lộ ra trong điều kiện kiểm tra có định hướng. Trong quá trình thực hiện đề tài, môi trường UVM đã phát hiện bốn lỗi thực sự trong mã RTL, trải rộng trên hai module thiết kế khác nhau.

Bug #1 – Sai giá trị mặc định thanh ghi BIR

Trong quá trình chạy testcase tc11_read_default thuộc nhóm A, scoreboard phát hiện giá trị đọc từ thanh ghi BIR ngay sau reset không khớp với đặc tả. Theo thiết kế hình 4.6, bit DecErrEn tại vị trí bit [0] của BIR phải có giá trị mặc định là 1 sau reset. Tuy nhiên, kết quả thực tế ghi nhận giá trị là 0.

Table 2-5. Bridge Interrupt Register (BIR)				
Bit	Name	Type	Default Value	Description
[31:2]	-	Rsvd	0	Reserved
1	DecErrSt	RW1C	0	Interrupt status. DecErrSt is set when receiving a non-existing address, or an address that matches multiple slaves 0: This interrupt has occurred. 1: Writing a '1' to clears this interrupt.
0	DecErrEn	RW	1	Enable or disable DecErrIntr 0: Disable 1: Enable

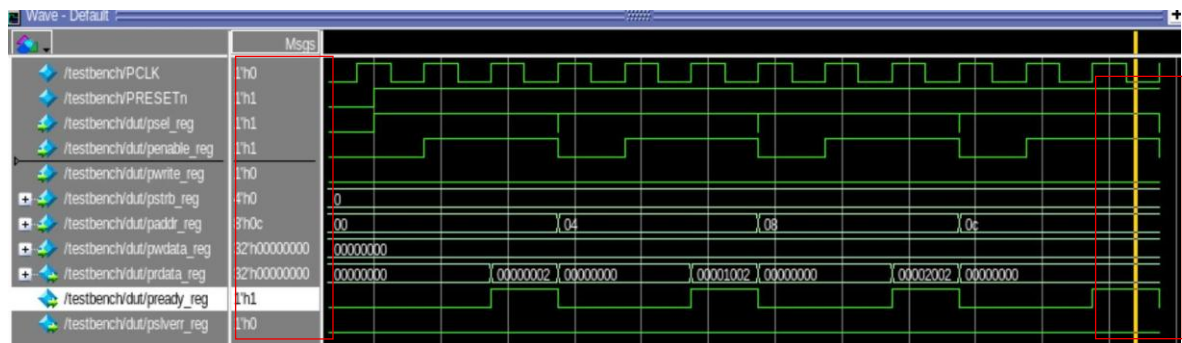
Hình 4.6: Giá trị mặc định thanh ghi BIR

Hình 4.7 bên dưới là thông báo lỗi được Scoreboard ghi nhận, cho thấy giá trị đọc về (actual) là 0x0 trong khi giá trị kỳ vọng (expected) là 0x1.

```
# UVM_INFO ../vip/apb_master_vip/apb_master_monitor.sv(46) @ 0: uvm_test_top.env.apb_master_agt.mon [MON] Transaction get: Addr=c, Write=0, Strb=x, Data=0
# UVM_INFO /usr/local/questasim/verilog_src/uvm-1.2/src/base/uvm_objection.svh(1270) @ 0: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
# UVM_INFO ../tb/scoreboard.sv(125) @ 0: uvm_test_top.env.sb [SCB_MATCH] [Addr: 0x0] READ MATCH: 0x2
# UVM_INFO ../tb/scoreboard.sv(125) @ 0: uvm_test_top.env.sb [SCB_MATCH] [Addr: 0x4] READ MATCH: 0x1002
# UVM_INFO ../tb/scoreboard.sv(125) @ 0: uvm_test_top.env.sb [SCB_MATCH] [Addr: 0x8] READ MATCH: 0x2002
# UVM_ERROR ../tb/scoreboard.sv(123) @ 0: uvm_test_top.env.sb [SCB_RDATA_MISMATCH] [Addr: 0xc] RDATA ERROR! Exp: 0x1, Act: 0x0
# UVM_INFO /usr/local/questasim/verilog_src/uvm-1.2/src/base/uvm_report_server.svh(847) @ 0: reporter [UVM/REPORT/SERVER]
```

Hình 4.7: Scoreboard báo lỗi mismatch giá trị mặc định

Hình 4.8 là dạng sóng mô phỏng tại thời điểm đọc thanh ghi BIR sau reset. Có thể quan sát thấy tín hiệu PRDATA trên bus APB trả về giá trị 0x0, trong khi đặc tả yêu cầu giá trị phải là 0x1 ngay sau kết thúc tín hiệu reset (PRESETn=1).



Hình 4.8: Dạng sóng chi tiết PRDATA tại vùng đọc BIR sau reset (Bug #1)

Hình 4.9 là đoạn mã RTL trong file APB_REGISTER.v gốc chứa lỗi, trong đó câu lệnh gán giá trị reset cho bit DecErrEn được viết sai là 0. Hình 4.10 là đoạn mã sau khi sửa, thay giá trị 0 thành 1.

```

137 wire hit_s1 = (addr >= start_slave_1) && (addr <= end_slave_1);
138 wire hit_s2 = (addr >= start_slave_2) && (addr <= end_slave_2);
139 wire condition_DecErrSt = ~(hit_s0 | hit_s1 | hit_s2); // KHÔNG trúng slave nào → decode error
140
141 reg DecErrSt;
142 reg DecErrEn;
143 reg [31:0] btr;
144
145 wire DecErrSt_next;
146 wire DecErrEn_next;
147 wire DecErrClr;
148 wire DecEnSel;
149
150 always @(posedge PCLK or negedge PRESETn) begin
151     if (!PRESETn) begin
152         DecErrSt <= 1'b0;
153         DecErrEn <= 0;
154     end else begin
155         DecErrSt <= DecErrSt_next;
156         DecErrEn <= DecErrEn_next;
157     end
158 end
159
160 assign DecErrSt_next = DecErrClr ? 1'b0 :
161     condition_DecErrSt ? 1'b1 : DecErrSt;
162 assign DecErrClr = (paddr_reg == BIR_ADDR) && (wr_en == 1) && (pdata_reg[1] == 1'b1) && pstrb_reg[0] == 1'b1;
163
164 assign DecErrEn_next = DecEnSel ? pdata_reg[0] : DecErrEn;
165 assign DecEnSel = (paddr_reg == BIR_ADDR) && (wr_en == 1) && pstrb_reg[0] == 1'b1;
166
167
168 assign DecErrIntr = DecErrSt & DecErrEn;
169
170 always @(*) begin
171     btr = {30'h0, DecErrSt, DecErrEn};
172 end
173
174 // READ ADDR REGISTER
175 always @(*) begin
176     case (rd_en)
177         1'b0: prdata_reg = 32'h0;
178         1'b1: begin
179             case (paddr_reg)
180                 BAMS0_ADDR: prdata_reg = bams0;

```

Hình 4.9: Mã RTL gốc tại dòng 153 – DecErrEn <= 0 (lỗi)

```

137 wire hit_s1 = (addr >= start_slave_1) && (addr <= end_slave_1);
138 wire hit_s2 = (addr >= start_slave_2) && (addr <= end_slave_2);
139 wire condition_DecErrSt = ~(hit_s0 | hit_s1 | hit_s2); // KHÔNG trúng slave nào → decode error
140
141 reg DecErrSt;
142 reg DecErrEn;
143 reg [31:0] btr;
144
145 wire DecErrSt_next;
146 wire DecErrEn_next;
147 wire DecErrClr;
148 wire DecEnSel;
149
150 always @(posedge PCLK or negedge PRESETn) begin
151     if (!PRESETn) begin
152         DecErrSt <= 1'b0;
153         DecErrEn <= 1;
154     end else begin
155         DecErrSt <= DecErrSt_next;
156         DecErrEn <= DecErrEn_next;
157     end
158 end
159
160 assign DecErrSt_next = DecErrClr ? 1'b0 :
161     condition_DecErrSt ? 1'b1 : DecErrSt;
162 assign DecErrClr = (paddr_reg == BIR_ADDR) && (wr_en == 1) && (pdata_reg[1] == 1'b1) && pstrb_reg[0] == 1'b1;
163
164 assign DecErrEn_next = DecEnSel ? pdata_reg[0] : DecErrEn;
165 assign DecEnSel = (paddr_reg == BIR_ADDR) && (wr_en == 1) && pstrb_reg[0] == 1'b1;
166
167
168 assign DecErrIntr = DecErrSt & DecErrEn;
169
170 always @(*) begin
171     btr = {30'h0, DecErrSt, DecErrEn};
172 end
173
174 // READ ADDR REGISTER
175 always @(*) begin
176     case (rd_en)
177         1'b0: prdata_reg = 32'h0;
178         1'b1: begin
179             case (paddr_reg)
180                 BAMS0_ADDR: prdata_reg = bams0;

```

Hình 4.10: Mã RTL sau khi sửa tại dòng 153 – DecErrEn <= 1

Bug #2 – Không thể ghi vào bit [1:0] của thanh ghi BAMS1

Trong quá trình chạy testcase tc13_bams_read_write_access, scoreboard phát hiện rằng sau khi ghi các giá trị khác nhau vào bit [1:0] của thanh ghi BAMS1 tại địa chỉ 0x04, giá trị đọc lại luôn bằng giá trị mặc định 0x2 bất kể đã ghi gì vào đó, theo hình 4.11 đặc tả thanh ghi BAMS1 thì bit[1:0] là bit RW. Hành vi này chỉ xảy ra với BAMS1, còn BAMS0 và BAMS2 hoạt động bình thường.

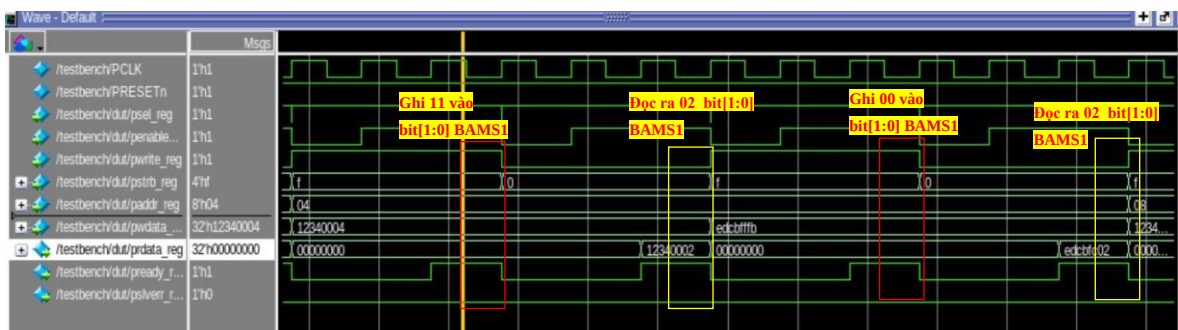
Bit	Name	Type	Default Value	Description
[31:10]	Base	RW	{3'h000;10'h4}	Base address of bits[31:10]
[9:2]	-	Rsvd	0	Reserved
[1:0]	Size	RW	2'b10	Size of address space
				KB select by Unit signal
				00: 1kb <0000_0000 => 0000_03FF>
				01: 2kb <0000_0000 => 0000_07FF>
				10: 4kb <0000_0000 => 0000_0FFF>
				11: 8kb <0000_0000 => 0000_1FFF>

Hình 4.11: Đặc tả thanh ghi BAMS1

Hình 4.12 là thông báo lỗi của Scoreboard, cho thấy giá trị bit [1:0] đọc về luôn là 0x2 bất kể giá trị đã ghi. Hình 4.13 là dạng sóng xác nhận hành vi này: khi ghi 00 vào bit [1:0] thì đọc về vẫn là 10 (0x2), khi ghi 11 thì đọc về cũng vẫn là 10 (0x2).

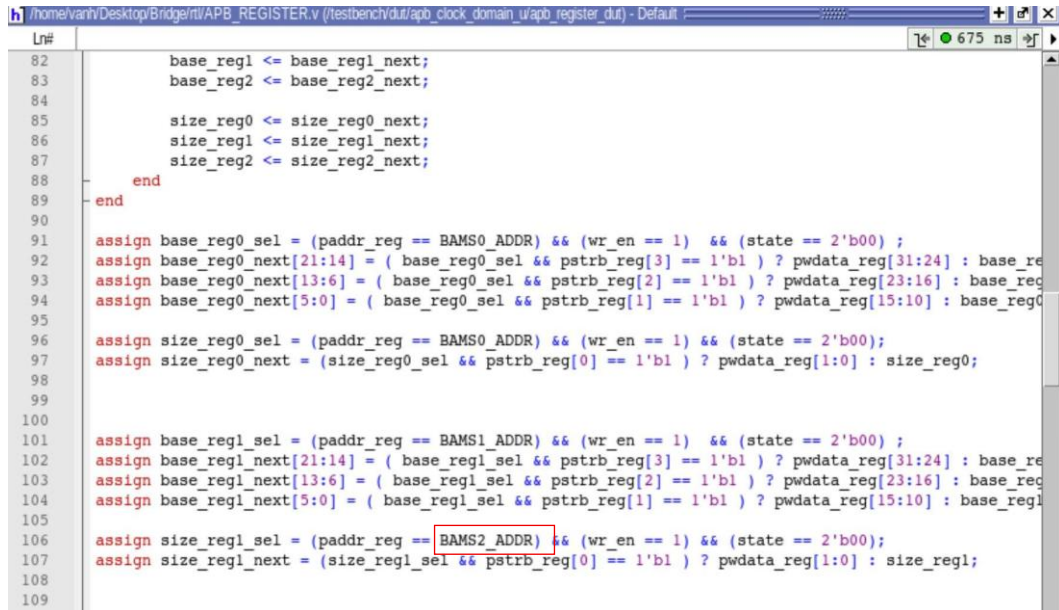
```
# UVM_INFO ../tb/scoreboard.sv(113) @ 1000: uvm_test_top.env.sb [SCB_WRITE_MATCH] [Addr: 0x4] WRITE MATCH | PSTRB: 4'b1111 | Bus
WDATA: 0x12340004 | FINAL REG VAL: 0x12340000
# UVM_ERROR ../tb/scoreboard.sv(123) @ 1000: uvm_test_top.env.sb [SCB_RDATA_MISMATCH] [Addr: 0x4] RDATA ERROR! Exp: 0x12340000, A
ct: 0x12340002
# UVM_INFO ../tb/scoreboard.sv(113) @ 1000: uvm_test_top.env.sb [SCB_WRITE_MATCH] [Addr: 0x4] WRITE MATCH | PSTRB: 4'b1111 | Bus
WDATA: 0xedcbfffb | FINAL REG VAL: 0xedcbfc03
# UVM_ERROR ../tb/scoreboard.sv(123) @ 1000: uvm_test_top.env.sb [SCB_RDATA_MISMATCH] [Addr: 0x4] RDATA ERROR! Exp: 0xedcbfc03, A
ct: 0xedcbfc02
# UVM_INFO ../tb/scoreboard.sv(113) @ 1000: uvm_test_top.env.sb [SCB_WRITE_MATCH] [Addr: 0x8] WRITE MATCH | PSTRB: 4'b1111 | Bus
WDATA: 0x12340008 | FINAL REG VAL: 0x12340000
# UVM_INFO ../tb/scoreboard.sv(125) @ 1000: uvm_test_top.env.sb [SCB_MATCH] [Addr: 0x8] READ MATCH: 0x12340000
```

Hình 4.12: Scoreboard báo lỗi



Hình 4.13: Dạng sóng chi tiết xác nhận ghi 00/11 vào bit [1:0] BAMS1 đọc về vẫn là 10

Hình 4.14 là đoạn mã RTL gốc tại dòng 106, cho thấy điều kiện địa chỉ bị viết sai là BAMS2_ADDR, cần sửa lại với điều kiện đúng là BAMS1_ADDR.



```

Ln#
82      base_reg1 <= base_reg1_next;
83      base_reg2 <= base_reg2_next;
84
85      size_reg0 <= size_reg0_next;
86      size_reg1 <= size_reg1_next;
87      size_reg2 <= size_reg2_next;
88  end
89  end
90
91  assign base_reg0_sel = (paddr_reg == BAMS0_ADDR) && (wr_en == 1) && (state == 2'b00) ;
92  assign base_reg0_next[21:14] = ( base_reg0_sel && pstrb_reg[3] == 1'b1 ) ? pdata_reg[31:24] : base_reg0;
93  assign base_reg0_next[13:6] = ( base_reg0_sel && pstrb_reg[2] == 1'b1 ) ? pdata_reg[23:16] : base_reg0;
94  assign base_reg0_next[5:0] = ( base_reg0_sel && pstrb_reg[1] == 1'b1 ) ? pdata_reg[15:10] : base_reg0;
95
96  assign size_reg0_sel = (paddr_reg == BAMS0_ADDR) && (wr_en == 1) && (state == 2'b00);
97  assign size_reg0_next = (size_reg0_sel && pstrb_reg[0] == 1'b1 ) ? pdata_reg[1:0] : size_reg0;
98
99
100
101  assign base_reg1_sel = (paddr_reg == BAMS1_ADDR) && (wr_en == 1) && (state == 2'b00) ;
102  assign base_reg1_next[21:14] = ( base_reg1_sel && pstrb_reg[3] == 1'b1 ) ? pdata_reg[31:24] : base_reg1;
103  assign base_reg1_next[13:6] = ( base_reg1_sel && pstrb_reg[2] == 1'b1 ) ? pdata_reg[23:16] : base_reg1;
104  assign base_reg1_next[5:0] = ( base_reg1_sel && pstrb_reg[1] == 1'b1 ) ? pdata_reg[15:10] : base_reg1;
105
106  assign size_reg1_sel = (paddr_reg == BAMS2_ADDR) && (wr_en == 1) && (state == 2'b00);
107  assign size_reg1_next = (size_reg1_sel && pstrb_reg[0] == 1'b1 ) ? pdata_reg[1:0] : size_reg1;
108
109

```

Hình 4.14: Mã RTL gốc dòng 106 – điều kiện sai BAMS2_ADDR

Bug #3 – BRESP không phản hồi khi slave báo lỗi PSLVERR

Trong quá trình chạy testcase tc24_psel0_fixed_wr_err, scoreboard phát hiện rằng khi APB Slave báo lỗi PSLVERR=1, tín hiệu BRESP trên kênh B của AXI hoàn toàn không có phản hồi. Giao dịch AXI bị treo do bridge không assert BVALID, khiến phía Master AXI chờ vô thời hạn. Nguyên nhân là lỗi đồng bộ hóa tín hiệu giữa hai miền xung nhịp: tín hiệu bresp_in được cập nhật cùng lúc với PSLVERR trong miền APB nhưng chưa qua tầng đồng bộ hóa trước khi sử dụng trong miền AXI.

Hình 4.15 là bằng chứng lỗi thu thập từ dạng sóng mô phỏng, thể hiện PSLVERR được assert từ phía APB nhưng BVALID trên kênh AXI không bao giờ được kích hoạt. Hình 4.16 và Hình 4.17 lần lượt là mã RTL trước và sau khi sửa lỗi đồng bộ hóa.


```

234 // =====
235 // B CHANNEL (Write Response Channel)
236 // =====
237
238 // ----- BRESP update -----
239 reg [1:0] bresp_in_d1;
240 always @(posedge ACLK or negedge ARESETn)
241 begin
242     if (~ARESETn)
243         bresp_in_d1 <= 2'b00;
244     else
245         bresp_in_d1 <= bresp_in;
246 end
247 assign bresp_tmp = (aw_done) ? bresp_in_d1 : bresp_r;
248
249 // ----- BRESP output (KHÔNG drop về 0) -----
250 assign BRESP = bresp_r;
251
252 // ----- BRESP register -----
253 always @(posedge ACLK or negedge ARESETn)
254 begin
255     if (~ARESETn)
256         bresp_r <= 2'b00;
257     else
258         bresp_r <= bresp_tmp;
259 end
260
261 // ----- BVALID (giữ đến khi handshake) -----
262 reg BVALID_r;
263
264 always @(posedge ACLK or negedge ARESETn)
265 begin
266     if (~ARESETn)
267         BVALID_r <= 1'b0;
268     else if (BVALID_r & BREADY)
269         BVALID_r <= 1'b0;
270     else if (aw_done)
271         BVALID_r <= 1'b1;
272 end
273
274 assign BVALID = BVALID_r;

```

Hình 4.17: Mã RTL sau khi sửa – thêm tầng synchronizer cho bresp_in

Bug #4 – Lỗi AXI Read Handshake gây sai RLAST và mismatch dữ liệu

Trong quá trình kiểm tra các testcase nhóm C, log mô phỏng báo lỗi vi phạm giao thức AXI: giá trị trên kênh R thay đổi khi RVALID=1 nhưng RREADY=0, và scoreboard báo mismatch dữ liệu. Lỗi xuất hiện khi RVALID và RREADY cùng giữ ở mức 1 trong nhiều chu kỳ clock liên tiếp: logic không capture RDATA và RLAST đúng theo từng beat độc lập trong tình huống multi-cycle handshake.

Hình 4.18 là log lỗi giao thức AXI ghi nhận trong quá trình mô phỏng. Hình 4.19 là cho thấy dữ liệu RDATA hoặc tín hiệu RLAST không được cập nhật/capture đúng theo từng beat, dẫn đến việc monitor hoặc scoreboard ghi nhận sai transaction..

```

# ** Error: AXI R channel changed while RVALID=1 and RREADY=0
Time: 10055 ns, Start: 10045 ns, Error: scoreboard output 5

```

Hình 4.18: Dạng sóng kênh R – RDATA thay đổi khi RVALID=1
RREADY=0

```

# UVM_ERROR ../tb/scoreboard.sv(623) @ 45000: uvm_test_top.env.sb [SCB_AXI_RDATA] AXI RDATA mismatch. ARADDR=0x0000001c BeatAddr=0x0000001d Beat=1/5 SizeBytes=1 Exp=0x0000001d
Act=0xbeef0008 PSEL=PSEL0
# UVM_ERROR ../tb/scoreboard.sv(623) @ 45000: uvm_test_top.env.sb [SCB_AXI_RDATA] AXI RDATA mismatch. ARADDR=0x0000001c BeatAddr=0x0000001e Beat=2/5 SizeBytes=1 Exp=0x0000001e
Act=0xbeef0008 PSEL=PSEL0
# UVM_ERROR ../tb/scoreboard.sv(623) @ 45000: uvm_test_top.env.sb [SCB_AXI_RDATA] AXI RDATA mismatch. ARADDR=0x0000001c BeatAddr=0x0000001f Beat=3/5 SizeBytes=1 Exp=0x0000001f
Act=0xbeef0008 PSEL=PSEL0
# UVM_ERROR ../tb/scoreboard.sv(623) @ 45000: uvm_test_top.env.sb [SCB_AXI_RDATA] AXI RDATA mismatch. ARADDR=0x0000001c BeatAddr=0x00000020 Beat=4/5 SizeBytes=1 Exp=0x00000020
Act=0xbeef0008 PSEL=PSEL0
# UVM_ERROR ../tb/scoreboard.sv(623) @ 45000: uvm_test_top.env.sb [SCB_AXI_RDATA] AXI RDATA mismatch. ARADDR=0x0000001c BeatAddr=0x00000021 Beat=5/5 SizeBytes=1 Exp=0x00000021
Act=0xbeef0008 PSEL=PSEL0

```

Hình 4.19: Dữ liệu RDTA bị capture sai

Cần sửa lại thời gian tồn tại handshake (READY & VALID=1) chỉ trong 1 chu kỳ và RLAST trong file AXI_CLOCK_DOMAIN.sv

Chỉnh sửa logic RVALID

Ở phiên bản RTL ban đầu, logic assign RVALID = ~rd_empty; được kết nối trực tiếp vào chân .rden(RVALID) của FIFO.

```

137
138 assign r_data_tmp = (ar_addr [4:2] == 3'b000) ? {24'h00, r_data [7:0]} :
139 (ar_addr [4:2] == 3'b001) ? {16'h00, r_data [15:0]} :
140 (ar_addr [4:2] == 3'b010) ? r_data [31:0] : 32'h00;
141 assign rd_fifo_tmp = {r_data_tmp, rresp_in};
142
143 ASYS_FIFO #(DEPTH_WR, 34) rd_fifo (
144     .wclk      ( ACLK      ),
145     .rclk      ( PCLK      ),
146     .wrstn     ( ARESETn   ),
147     .rrstn     ( PRESETn   ),
148     .wren      ( r_en      ),
149     .rden      ( RVALID    ),
150     .wdata     ( rd_fifo_tmp ),
151     .rdata     ( {RDATA, RRESP} ),
152     .wfull     ( rd_full    ),
153     .rempty    ( rd_empty   ),
154     .almost_empty (      ),
155     .almost_full ( rd_almost_full ),
156 );
157 assign rlast_tmp = rd_trans_done ? rd_trans_done :
158 (RVALID & RREADY) & rlast_r ? 0 : rlast_r;
159 assign RLAST = rd_trans_done | rlast_r;
160 assign RVALID = ~rd_empty;
161
162 always @(posedge ACLK or negedge ARESETn)
163 if (~ARESETn)
164     rlast_r <= 0;
165 else
166     rlast_r <= rlast_tmp;
167

```

Hình 4.20: Mã RTL gốc – chưa chỉnh logic RVALID

Điều này tạo ra hành vi: FIFO còn dữ liệu => RVALID = 1 => FIFO bị đọc ngay

FIFO bị pop chỉ vì có dữ liệu, hoàn toàn không quan tâm Master đã sẵn sàng nhận dữ liệu hay chưa (RREADY).

Hậu quả

Giả sử:

rd_empty = 0

RREADY = 0

=> RVALID = 1 và FIFO vẫn bị đọc.

Kết quả: FIFO bị thay đổi trạng thái nhưng handshake chưa xảy ra. Khi RREADY = 0, RVALID vẫn bị giữ ở mức 1 trong nhiều chu kỳ và FIFO vẫn liên tục bị đọc. Sự chênh lệch này làm mất đồng bộ giữa dữ liệu thực tế trong FIFO và trạng thái giao dịch AXI, gây sai lệch giữa thời điểm dữ liệu được đẩy ra và thời điểm Master thực sự lấy dữ liệu.

Sau khi sửa

```
170 );  
171  
172 // ---- RVALID Generation (1-cycle pulse) ----  
173 reg RVALID_r;  
174  
175 always @(posedge ACLK or negedge ARESETn) begin  
176     if (!ARESETn)  
177         RVALID_r <= 1'b0;  
178     else  
179         RVALID_r <= (!rd_empty) && RREADY;  
180 end  
181  
182 assign RVALID = RVALID_r;  
183  
184 // --- FIFO Read Enable Logic  
185 assign rden = (!rd_empty) && RREADY;  
186
```

Hình 4.21: Mã RTL sau khi sửa logic RVALID

assign rden = (!rd_empty) && RREADY; và RVALID_r <= (!rd_empty) && RREADY;

Nghĩa là: Dữ liệu chỉ được lấy khỏi FIFO khi điều kiện handshake được thỏa mãn.

Trong phiên bản RTL ban đầu, tín hiệu RVALID được tạo trực tiếp từ trạng thái FIFO và FIFO được đọc ngay khi RVALID lên mức 1, khiến dữ liệu có thể bị lấy ra trước khi AXI Master xác nhận bằng tín hiệu RREADY. Sau khi sửa, FIFO chỉ được đọc khi điều kiện handshake RVALID && RREADY được thỏa mãn, nhờ đó loại bỏ hiện tượng RVALID bị giữ ở mức cao trong nhiều chu kỳ và đảm bảo tuân thủ giao thức AXI4.

Chỉnh sửa logic RLAST

Trước khi sửa, rd_trans_done được tạo ở miền PCLK nhưng được sử dụng gần như trực tiếp để sinh RLAST ở miền ACLK. Do không có cơ chế đồng bộ CDC, xung rd_trans_done có thể bị mất, bị kéo dài hoặc xuất hiện sai thời điểm, làm RLAST không xuất hiện chính xác tại beat cuối của burst.

```

137
138 assign r_data_tmp = (ar_addr [4:2] == 3'b000) ? {24'h00, r_data [7:0]} :
139                    (ar_addr [4:2] == 3'b001) ? {16'h00, r_data [15:0]} :
140                    (ar_addr [4:2] == 3'b010) ? r_data [31:0] : 32'h00;
141 assign rd_fifo_tmp = {r_data_tmp, rresp_in};
142
143     ASYS_FIFO #(DEPTH_WR, 34) rd_fifo (
144
145         .wclk      ( ACLK
146         .rclk      ( PCLK
147         .wrstn     ( ARESETn
148         .rrstn     ( PRESETn
149         .wren      ( r_en
150         .rdn       ( RVALID
151         .wdata     ( rd_fifo_tmp
152         .rdata     ( {RDATA, RRESP}
153         .wfull     ( rd_full
154         .empty     ( rd_empty
155         .almost_empty (
156         .almost_full ( rd_almost_full
157
158     );
159 assign rlast_tmp = rd_trans_done
160                  (QUALIN & BDEFANV) & rlast_r ? 0 : rlast_r;
161 assign RLAST = rd_trans_done | rlast_r;
162 assign RVALID = ~rd_empty;
163
164     always @(posedge ACLK or negedge ARESETn)
165     if (~ARESETn)
166         rlast_r <= 0;
167     else
168         rlast_r <= rlast_tmp;
169
170

```

Hình 4.22: Mã RTL gốc – chưa sửa logic RLAST

Sau khi sửa,

// PCLK domain

rd_done_pclk_d <= rd_trans_done;

// Synchronizer

rd_done_sync1 <= rd_done_pclk_d;

rd_done_sync2 <= rd_done_sync1;

// Generate RLAST

assign RLAST = rd_done_sync2 && RVALID_r;

rd_trans_done được đồng bộ từ miền PCLK sang ACLK thông qua bộ synchronizer hai tầng.

RLAST chỉ được phát khi tín hiệu hoàn thành giao dịch đã được đồng bộ thành công và tồn tại dữ liệu hợp lệ (RVALID_r = 1), đảm bảo beat cuối của burst được đánh dấu chính xác theo yêu cầu AXI4.

```

186
187 // ---- Synchronized Output Data and Response ----
188 reg [31:0] rdata_fifo_d;
189 reg [1:0] rresp_fifo_d;
190
191 always @(posedge ACLK or negedge ARESETn) begin
192     if (!ARESETn) begin
193         rdata_fifo_d <= 32'd0;
194         rresp_fifo_d <= 2'd0;
195     end else begin
196         rdata_fifo_d <= rdata_fifo;
197         rresp_fifo_d <= rresp_fifo;
198     end
199 end
200
201 assign RDATA = rdata_fifo_d;
202 assign RRESP = rresp_fifo_d;
203
204 // ---- RLAST Signal Logic ----
205
206 // Delay rd_trans_done in the PCLK domain
207 reg rd_done_pclk_d;
208
209 always @(posedge PCLK or negedge PRESETn) begin
210     if (!PRESETn)
211         rd_done_pclk_d <= 1'b0;
212     else
213         rd_done_pclk_d <= rd_trans_done;
214 end
215
216 // Synchronize rd_trans_done to the ACLK domain
217 reg rd_done_sync1, rd_done_sync2;
218
219 always @(posedge ACLK or negedge ARESETn) begin
220     if (!ARESETn) begin
221         rd_done_sync1 <= 1'b0;
222         rd_done_sync2 <= 1'b0;
223     end else begin
224         rd_done_sync1 <= rd_done_pclk_d;
225         rd_done_sync2 <= rd_done_sync1;
226     end
227 end
228
229 // Final RLAST assignment based on synchronized done and valid status
230 assign RLAST = rd_done_sync2 && RVALID r;

```

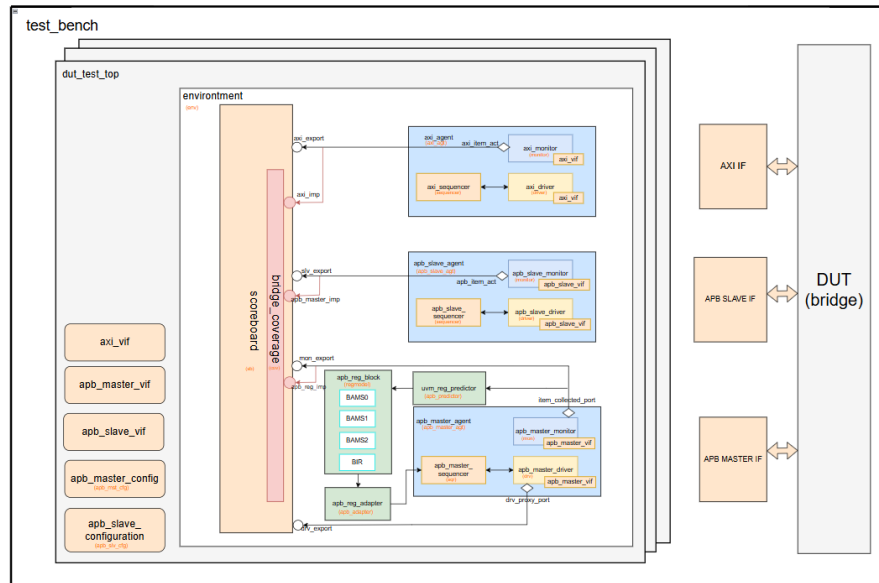
Hình 4.23: Mã RTL sau khi sửa logic RLAST

Trong phiên bản RTL ban đầu, tín hiệu RLAST được tạo từ rd_trans_done phát sinh ở miền PCLK nhưng được sử dụng trực tiếp ở miền ACLK, gây nguy cơ lỗi Clock Domain Crossing. Sau khi bổ sung bộ đồng bộ hai tầng và tạo RLAST từ tín hiệu đã được đồng bộ, tín hiệu beat cuối được phát đúng thời điểm và ổn định theo đặc tả AXI4.

4.5 XÂY DỰNG VÀ PHÂN TÍCH COVERAGE

Sau khi hoàn tất quá trình xây dựng môi trường kiểm chứng và toàn bộ testcase trong Verification Plan đều thực thi thành công, Coverage được sử dụng

như một thước đo để đánh giá mức độ hoàn thành của quá trình verification. Trong đề tài này, việc đánh giá được thực hiện dựa trên hai tiêu chí chính là Functional Coverage và Code Coverage, tương ứng với mức độ bao phủ các mục tiêu chức năng và cấu trúc RTL của thiết kế. Trong môi trường UVM Testbench, khối bridge_coverage được tích hợp cùng Scoreboard để thu thập dữ liệu coverage trong quá trình regression.



Hình 4.24: Kiến trúc UVM Testbench sau khi tích hợp khối Coverage vào Environment

Functional Coverage được thu thập tự động trong quá trình mô phỏng thông qua các covergroup được xây dựng dựa trên Verification Plan. Các covergroup tập trung bao phủ các nhóm chức năng cốt lõi của AXI-to-APB Bridge như AXI read/write path, burst type, transfer size, address decode, APB register access, interrupt handling và dynamic address remap. Ngoài các coverpoint riêng lẻ, hệ thống cũng sử dụng cross coverage để kiểm tra các tổ hợp điều kiện quan trọng nhằm đảm bảo các corner-case trong quá trình verification được kích hoạt đầy đủ.

Sau khi hoàn tất quá trình regression và hợp nhất dữ liệu coverage từ toàn bộ testcase, kết quả Functional Coverage đạt 100% feature coverage trên các nhóm chức năng đã định nghĩa trong Verification Plan. Điều này cho thấy toàn bộ các

hành vi chức năng trọng tâm của DUT đều đã được kích hoạt và kiểm chứng thành công trong phạm vi đề tài.

Name	Coverage	Goal	% of Goal	Status	Include
/env_pkg/bridge_coverage	100.00%				
TYPE axi_cover	100.00%	100	100.00%	✓	
CVP axi_cover:cp_xfer	100.00%	100	100.00%	✓	
CVP axi_cover:cp_size	100.00%	100	100.00%	✓	
CVP axi_cover:cp_brst	100.00%	100	100.00%	✓	
CVP axi_cover:cp_psel	100.00%	100	100.00%	✓	
CVP axi_cover:cp_dec	100.00%	100	100.00%	✓	
CVP axi_cover:cp_cross	100.00%	100	100.00%	✓	
CROSS axi_cover:x_full_traffic	100.00%	100	100.00%	✓	
CROSS axi_cover:x_crossing	100.00%	100	100.00%	✓	
INST axi_domain_coverage	100.00%	100	100.00%	✓	
TYPE apb_reg_cover	100.00%	100	100.00%	✓	
CVP apb_reg_cover:cp_reg	100.00%	100	100.00%	✓	
CVP apb_reg_cover:cp_rw	100.00%	100	100.00%	✓	
CVP apb_reg_cover:cp_strb	100.00%	100	100.00%	✓	
CVP apb_reg_cover:cp_remap	100.00%	100	100.00%	✓	
CROSS apb_reg_cover:x_reg_access	100.00%	100	100.00%	✓	
CROSS apb_reg_cover:x_remap_op	100.00%	100	100.00%	✓	
INST apb_register_coverage	100.00%	100	100.00%	✓	
TYPE apb_master_cover	100.00%	100	100.00%	✓	
CVP apb_master_cover:cp_psel_out	100.00%	100	100.00%	✓	
CVP apb_master_cover:cp_pslverr	100.00%	100	100.00%	✓	
CVP apb_master_cover:cp_decerr	100.00%	100	100.00%	✓	
CVP apb_master_cover:cp_intr	100.00%	100	100.00%	✓	
CROSS apb_master_cover:x_sticky_check	100.00%	100	100.00%	✓	
CROSS apb_master_cover:x_slave_err_matrix	100.00%	100	100.00%	✓	
INST Venv_pkg:bridge_coverage:apb_master_cover	100.00%	100	100.00%	✓	

Hình 4.25: Kết quả tổng hợp Functional Coverage của môi trường UVM Testbench

Instance	Design unit	Design unit type	Top Category	Visibility	Cover Options	Total coverage	Covergroup %	Stmt count	Stmts hit	Stmts missed	Stmt %	Stmt graph	Branch count	Branches hit	Branches missed	Branch %	Branch graph	Toggle nodes	Toggle hit	Toggle %
testbench	Module	DU Instance		+acc=rtl, -cover=bofst	81.79%			525	530	5	97.14%		255	243	12	95.29%		6180	4401	71.21%
apb_master_vif	Interface	DU Instance		+acc=rtl, -cover=bofst	92.77%			14	14	0	100.00%							166	142	85.54%
apb_slave_vif	Interface	DU Instance		+acc=rtl, -cover=bofst	80.41%			23	23	0	100.00%							220	189	85.91%
axi_vif	Interface	DU Instance		+acc=rtl, -cover=bofst	94.89%			36	36	0	100.00%							352	341	96.87%
dut	AXI_to_APB_Module	DU Instance		+acc=rtl, -cover=bofst	81.61%			426	420	6	98.59%		255	243	12	95.29%		5434	3841	70.69%
apb_clock_domain_u	APB_CLOCK_DOMAIN	Module	DU Instance	+acc=rtl, -cover=bofst	83.52%			214	199	15	92.99%		155	144	11	92.90%		3604	2545	70.62%
axi_clock_domain_u	AXI_CLOCK_DOMAIN	Module	DU Instance	+acc=rtl, -cover=bofst	82.60%			221	221	0	100.00%		100	99	1	99.00%		2184	1743	79.85%
wrm_pkg	WrmPkg	VPackage	Package	+acc=rtl, -cover=bofst																
bridge_register_pkg	BridgeRegisterPkg	VPackage	Package	+acc=rtl, -cover=bofst	13.72%			68	28	40	41.17%		40	0	40	0.00%				
apb_master_pkg	ApbMasterPkg	VPackage	Package	+acc=rtl, -cover=bofst	23.22%			196	76	120	38.77%		108	36	72	33.33%				
apb_register_pkg	ApbRegisterPkg	VPackage	Package	+acc=rtl, -cover=bofst	54.34%			60	26	34	43.33%		42	15	27	35.71%				
apb_slave_pkg	ApbSlavePkg	VPackage	Package	+acc=rtl, -cover=bofst	18.15%			378	110	268	29.10%		395	58	337	14.68%				
axi_pkg	AxiPkg	VPackage	Package	+acc=rtl, -cover=bofst	42.09%			627	290	337	46.25%		438	69	369	15.75%				
env_pkg	EnvPkg	VPackage	Package	+acc=rtl, -cover=bofst	69.77%	100.00%		382	285	97	74.60%		315	148	167	46.98%				
seq_pkg	SeqPkg	VPackage	Package	+acc=rtl, -cover=bofst	44.29%			79	57	22	72.15%		42	15	27	35.71%				
test_pkg	TestPkg	VPackage	Package	+acc=rtl, -cover=bofst	41.55%			1524	1229	295	80.64%		1152	443	709	38.45%				

Hình 4.26: Kết quả tổng hợp Code Coverage trên RTL của AXI-to-APB Bridge

Bên cạnh đó, Code Coverage được sử dụng để đánh giá mức độ kích hoạt các dòng lệnh, nhánh điều kiện và biểu thức logic trong thiết kế RTL. Kết quả tổng hợp cho thấy Code Coverage đạt 81.61%, phản ánh phần lớn các luồng xử lý và nhánh điều khiển quan trọng trong hệ thống đã được thực thi trong quá trình verification.

Mặc dù đạt mức bao phủ tương đối cao, kết quả này vẫn chưa đạt mục tiêu trên 90% Code Coverage đã đề ra trong yêu cầu thiết kế. Nguyên nhân chủ yếu đến từ việc một số nhánh logic nằm ngoài phạm vi Verification Plan chưa được kích hoạt trong quá trình regression. Ví dụ, trường AXLEN của giao thức AXI có độ rộng 8 bit và hỗ trợ dải burst length lớn. Tuy nhiên, trong phạm vi đề tài, các testcase chỉ tập trung kiểm chứng các giá trị từ 0 đến 8'd8 theo đúng mục tiêu verification đã xác định. Do đó, các nhánh liên quan đến các giá trị AXLEN ngoài phạm vi này không được thực thi. Bên cạnh đó, một số nhánh xử lý dự phòng hoặc các điều kiện hiếm gặp trong thực tế cũng chưa được đưa vào trọng tâm kiểm chứng của đề tài.

Nhìn chung, việc đạt 100% Functional Coverage và 81.61% Code Coverage cho thấy môi trường verification đã bao phủ đầy đủ các tính năng chức năng trọng tâm, đồng thời đạt mức bao phủ cấu trúc RTL cao trong phạm vi mục tiêu của đề tài. Đây là cơ sở quan trọng để đánh giá quá trình verification đối với Runtime-Configurable AXI-to-APB Bridge đã đạt mức hoàn thiện tốt.

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 KẾT LUẬN

Trong phạm vi đề tài, quy trình Design Verification dựa trên phương pháp Universal Verification Methodology (UVM) đã được nghiên cứu, xây dựng và triển khai thành công trên đối tượng Runtime-Configurable AXI-to-APB Bridge. Thiết kế bridge được kế thừa từ dự án mã nguồn mở và được mở rộng bằng hệ thống thanh ghi cấu hình cho phép thay đổi động bản đồ địa chỉ của các APB Slave trong thời gian chạy, đồng thời bổ sung cơ chế xử lý Decode Error và hệ thống ngắt tương ứng.

Trên cơ sở đó, luận văn đã xây dựng một môi trường kiểm chứng hoàn chỉnh theo hướng tiếp cận công nghiệp, bao gồm Verification Plan (VPlan), Verification IP (VIP), Register Abstraction Layer (RAL), Scoreboard, Functional Coverage cùng hệ thống script phục vụ quá trình mô phỏng, regression và phân tích kết quả. Quy trình kiểm chứng được triển khai theo các bước từ phân tích thiết kế, xác định mục tiêu kiểm chứng, xây dựng môi trường UVM đến đánh giá kết quả thông qua coverage.

Kết quả thực nghiệm cho thấy toàn bộ testcase trong Verification Plan đều thực thi thành công sau quá trình debug và regression. Functional Coverage đạt 100%, cho thấy các mục tiêu chức năng đã được xác định trong phạm vi đề tài đều được kích hoạt và kiểm chứng đầy đủ. Bên cạnh đó, Code Coverage đạt 81.61%, phản ánh phần lớn các luồng xử lý và nhánh logic quan trọng của thiết kế đã được thực thi trong quá trình verification.

Mặc dù chưa đạt mục tiêu trên 90% Code Coverage như kỳ vọng ban đầu, kết quả này chủ yếu xuất phát từ việc một số nhánh logic nằm ngoài phạm vi Verification Plan chưa được kích hoạt, đồng thời quá trình coverage closure và coverage exclusion chưa được thực hiện đầy đủ. Do đó, vẫn còn khả năng tiếp tục mở rộng testcase và hoàn thiện chiến lược coverage nhằm nâng cao mức độ bao phủ của thiết kế trong các nghiên cứu tiếp theo.

Nhìn chung, đề tài đã chứng minh tính khả thi của việc áp dụng quy trình Design Verification dựa trên phương pháp UVM đối với các register-based IP có giao thức truyền thông và hệ thống thanh ghi cấu hình. Các kết quả đạt được có thể được sử dụng làm tài liệu tham khảo cho việc xây dựng và triển khai môi trường kiểm chứng đối với các IP tương tự trong lĩnh vực thiết kế vi mạch số.

5.2 HƯỚNG PHÁT TRIỂN

Bên cạnh các kết quả đã đạt được, đề tài vẫn còn một số hướng phát triển có thể tiếp tục nghiên cứu trong tương lai.

Về mặt thiết kế RTL, Runtime-Configurable AXI-to-APB Bridge có thể được mở rộng nhằm hỗ trợ các tính năng nâng cao của giao thức AXI như outstanding transaction, out-of-order transaction hoặc cơ chế exclusive access. Bên cạnh đó, hệ thống runtime address remap hiện tại có thể được phát triển theo hướng hỗ trợ số lượng APB Slave linh hoạt hơn và các cơ chế ánh xạ địa chỉ phức tạp hơn nhằm đáp ứng nhu cầu tích hợp trong các hệ thống SoC quy mô lớn.

Về mặt kiểm chứng, VPlan có thể được mở rộng nhằm bao phủ nhiều tình huống kiểm chứng hơn, đặc biệt là các trường hợp liên quan đến không gian địa chỉ, dữ liệu truyền và các tổ hợp giao dịch phức tạp chưa nằm trong phạm vi đề tài hiện tại. Việc mở rộng phạm vi kiểm chứng sẽ góp phần nâng cao Code Coverage và tiến tới hoàn thành mục tiêu coverage đã đề ra.

Ngoài ra, môi trường UVM Testbench có thể tiếp tục được cải tiến bằng cách bổ sung stress testcase, protocol corner-case, assertion-based verification (ABV) hoặc fault injection nhằm tăng khả năng phát hiện lỗi và nâng cao độ tin cậy của quá trình verification.

Cuối cùng, quy trình Design Verification được xây dựng trong đề tài có thể được áp dụng và đánh giá trên các register-based IP khác như UART, SPI, I2C, GPIO hoặc Timer. Đây cũng là hướng phát triển quan trọng nhằm đánh giá khả năng tái sử dụng và tính tổng quát của quy trình verification đã đề xuất.

TÀI LIỆU THAM KHẢO

- [1] Accellera Systems Initiative, Universal Verification Methodology (UVM) 1.2 User's Guide. Accellera Systems Initiative, 2015.
- [2] M. Horn, Universal Verification Methodology (UVM) Cookbook. Siemens Verification Academy, 2018.
- [3] A. R. Babu, P. Anand, Y. Kim, and S. Jadhav, "SystemVerilog versus UVM-Based Verification of AXI-Lite Arbitration," in Proceedings of IEEE SoutheastCon 2023, Apr. 2023, pp. 350–357.
- [4] S. A. Pharate et al., "Verification of AMBA-APB Protocol Using UVM," International Journal of Innovative Research in Electrical, Electronics, Instrumentation and Control Engineering (IJIREEICE), vol. 12, 2024.
- [5] ARM Limited, AMBA AXI Protocol Specification, ARM IHI 0022E, Feb. 2013.
- [6] ARM Limited, AMBA APB Protocol Specification, ARM IHI 0024E, 2023.
- [7] V. S. R. V. Anil Kumar, "Design and Implementation of AXI–APB Bridge Based on AMBA 4.0," International Journal for Scientific Research & Development, 2014.
- [8] K. S. P. Sarekokku, "Design and Implementation of APB Bridge Based on AMBA AXI 4.0," International Journal of Engineering Research & Technology (IJERT), 2012.
- [9] M. Efil, "Design and Verification of AXI2APB Bridge Communication Protocol," B.Sc. Thesis, Abdullah Gul University.
- [10] K. Usha and T. Ashok Kumar, "AXI to APB Interface Design Using Verilog," IOSR Journal of Electronics and Communication Engineering (IOSR-JECE), 2013.
- [11] Accellera Systems Initiative, "Universal Verification Methodology (UVM) Working Group." [Online]. Available:

<https://www.accellera.org/activities/working-groups/uvm>. [Accessed: Jun. 19, 2026].

[12] J. Ye, Y. Hu, K. Xu, D. Pan, Q. Chen, J. Zhou, S. Zhao, X. Fang, X. Wang, N. Guan, and Z. Jiang, “From Concept to Practice: an Automated LLM-aided UVM Machine for RTL Verification,” arXiv preprint arXiv:2504.19959, Apr. 2025. [Online]. Available: <https://arxiv.org/abs/2504.19959>. [Accessed: Jun. 19, 2026].

[13] N. D. K. Vy, “Design and Verify AXI4-to-APB4 Bridge using UVM,” GitHub Repository. [Online]. Available: https://github.com/ngdkvy/DA2_Design-and-Verify-AXI4-to-APB4-bridge-using-UVM. [Accessed: Jun. 19, 2026].

[14] ICTC, “Công việc của kỹ sư kiểm tra thiết kế chip (Design Verification Engineer) – Bí mật đằng sau việc đảm bảo thành công cho silicon.” [Online]. Available: <https://ictc.edu.vn/cong-viec-cua-ky-su-kiem-tra-thiet-ke-chip-dv-bi-mat-dang-sau-viec-dam-bao-thanh-cong-cho-silicon/>. [Accessed: Jun. 19, 2026].

[15] Synopsys, “What is a System on a Chip (SoC)?,” [Online]. Available: <https://www.synopsys.com/glossary/what-is-a-system-on-a-chip.html> [Accessed: Jun. 19, 2026].

[16] H. Nguyen, “Verification IP (VIP) Là Gì?,” ICTC – IC Training Center Vietnam, Jul. 31, 2024. [Online]. Available: <https://ictc.edu.vn/verification-ip-vip-la-gi/>. [Accessed: Jun. 19, 2026].

[17] D. Le, “Vai Trò Của IP Và Các Loại IP Phổ Biến Trong Vi Mạch,” ICTC – IC Training Center Vietnam, Jan. 18, 2025. [Online]. Available: <https://ictc.edu.vn/vai-tro-cua-ip-va-cac-loai-ip-pho-bien-trong-vi-mach/>. [Accessed: Jun. 19, 2026].